SCALAR DESIGN SYSTEM

HOW IT WORKS · DESIGN TOKENS · COMPONENT REFERENCE

VERSION	@scalar/design-system 1.1.0
SOURCE	Prism-v1 · de84207 · 2026-10-12 15:08:17 -0600
PART I	How the system works: layers, pipeline, token and component recipes, component behaviour, 3-tier navigation, implementation, AI agents
COVERAGE	186 components · 968 documented props · 180 semantic colours (Light + Dark) · 142 primitives · 60 scale, elevation & motion tokens · 80 type tokens × 3 modes
GENERATED	19 October 2026

Generated from the live code layer: token values from **`src/tokens/tokens.json`** (extracted from the Figma library **`Scalar_Design_System-v1.1`**), component documentation from the JSDoc in each component's source, Figma traceability from **`docs/components.md`**, and previews rendered from the published Storybook stories. Light mode throughout (rule R14).

SCALAR DESIGN SYSTEM · V1.1

CONTENTS 1.1

I HOW THE SYSTEM WORKS 4

01 The problem and the four layers 5
02 One source of truth 6
03 Tokens are named by role 7
04 Token recipe: pick by job, never by eye 8
05 How a component is built 12
06 Choosing a component by intent 14
07 Behaviour of the main components 17
08 The 3-tier navigation system 22
09 Implementation examples 28
10 Building with an AI agent 30
11 Getting started and quality 32

II SYSTEM RULES 33

III DESIGN TOKENS 36

Semantic colour 37
Scales 44
Typography 47
Primitive colour — reference only 49

IV COMPONENTS 55

02 · Core (5) 56
03 · Avatar (1) 60
04 · Button (3) 61
05 · Chip (1) 65
06 · Input (2) 66
07 · Checkbox (2) 68
08 · Form Controls (6) 70
09 · Header & Menus (25) 77
10 · Navigation (8) 97
11 · Table & Cells (11) 103
12 · Charts (6) 116
13 · Summary Card (2) 123
14 · Feedback (4) 125
15 · Accordion & Drawer (6) 129
16 · Modals (4) 136
17 · Global Search (5) 139
18 · Template (0 components) 145
20 · Menus & Actions (11) 145
21 · Form Patterns (13) 156
22 · Grid Patterns (10) 171
23 · Files & Documents (7) 181
24 · Feedback Patterns (5) 187
25 · Overlays & Panels (4) 191
26 · Settings & Admin (13) 195
Workboard (4) 205
Added by the code layer 212
Other exports 220

COVERAGE · AT A GLANCE

BY THE NUMBERS

186

components, each mapped from Figma to a typed React export

968

documented props

180

semantic colours, each with a Light and a Dark value

142

primitive colour steps, reference only

60

scale, elevation & motion tokens

80×3

type tokens across Desktop, Desktop Large and Mobile modes

4

layers: tokens, components, templates, screens

Source: coverage line of the generated reference, 19 October 2026 · @scalar/design-system 1.1.0. Layer count from section 01.

HOW THE SYSTEM WORKS

This part explains the logic behind the reference that follows: how the layers fit together, how tokens and components are chosen, how the main components behave, how the 3-tier navigation is organised, and how a developer or an AI agent builds a screen. The token and component reference later in this document is the *what*; this part is the *why* and the *how*.

01 THE PROBLEM AND THE FOUR LAYERS

Without a system, every screen invents its own blue. A hex here, a one-off padding there, a button rebuilt per feature. Each decision is reasonable on the day; together they are the drift that review keeps chasing. The system replaces those decisions with names: five near-identical blues become one token, `--color-text-brand`, which is correct in Light and Dark and changes everywhere when Design changes it.

`@scalar/design-system` is one package in four layers. Each layer is built only from the one below, and a team can stop at any layer.

LAYER	WHAT IT IS	WHERE IT LIVES
1 · Tokens	Colour, type, spacing, radius, elevation and motion, in Light and Dark.	`src/styles/tokens.css`
2 · Components	Buttons, inputs, tables, grids, menus, charts, overlays. Typed, tested, accessible.	186 React components
3 · Templates	`PageTemplate`: navigation, a body slot and the docked drawer, wired once.	`template/PageTemplate.tsx`
4 · Screens	Real Scalar pages already coded from the layers below. Copy one before drawing from scratch.	103 frames in `apps/product`

02 ONE SOURCE OF TRUTH

Values flow one way. Figma owns the values; code owns the behaviour.

01 FIGMA
— a variable changes in the token library (`Scalar_Design_System-v1.1`).

02 `TOKENS.CSS`
— the one hand-kept token file mirrors it.

03 GENERATED
— `npm run gen:tokens` writes `tokens.json`, the typed TypeScript API and `docs/tokens.md`.

04 EVERYWHERE
— every component and every screen picks the change up; `npm run verify` checks types, token usage, lint, tests and that every product frame still builds.

```
$ npm run gen:tokens
wrote tokens.json, generated.ts, docs/tokens.md
$ npm run verify
types · token lint · eslint · tests · product frames
```

If a value looks wrong, fix it in Figma and regenerate. Never patch a value inside a component: the fix would reach one place instead of every place.

03 TOKENS ARE NAMED BY ROLE

FAMILY	COUNT	NOTE
Semantic colour	180	Each has a Light and a Dark value. The only colours product code may use.
Type	80 × 3	Desktop (1440), Desktop Large (1920) and Mobile (393) modes.
Scale, elevation, motion	60	Spacing, radius, sizing, breakpoints, elevation, duration, easing, z-index.
Primitive colour	142	Reference only, for tracing and audit. Never used directly (R1).

The same token name resolves correctly in both themes. `bg.warning` stays yellow in Dark on purpose: near-black text clears contrast on it, white never does (R4). PRISM tokens say what an item *is* (a firm, a company, a document), never what the user may do with it (R11).

04 TOKEN RECIPE: PICK BY JOB, NEVER BY EYE

The full colour recipe from `AI-GUIDE.md` §2:

Never pick a colour by eye. Find the row that matches the **job**.

THE JOB.	USETHE JOB	NEVER USEUSE
Body text on a page	`color.text.primary`	a primitive, `#0f172a`
Supporting / secondary text	`color.text.secondary`	opacity on primary
Labels, captions, metadata	`color.text.tertiary`	a lighter grey you picked
Text on a filled brand surface	`color.text.onBrand`	`color.text.primary` (R3)
A control sitting **on the brand bar** (chip, search, notification)	`color.bg.onBrand`	`bg.brandSubtle` — a light chip on navy
The value inside such a control	`color.text.onBrandSubtle`	`text.tertiary`
Its placeholder or field-label prefix	`color.text.onBrandMuted`	`text.disabled`
An **inactive** top-level nav label on the bar	`color.text.onBrandInactive`	`text.onBrand` for every item alike
The active nav item's underline	`color.stroke.onBrand`	a `bg.brandSubtle` chip (R3)
Icons inside bar controls	`color.icon.onBrandSubtle` / `.onBrandMuted` / `.onBrandInactive`	`icon.primary`
Text on a filled **yellow** surface	`color.text.onWarning` (near-black)	white (R4)
A primary action's fill	`color.bg.brand` + `text.onBrand`	`bg.brandSubtle`
A destructive action's fill	`color.bg.negative` + `text.onNegative`	brand recoloured by hand
An **unread** indicator dot	`color.bg.unread`	`bg.negative` — unread is not an error
A **group band** row in a data grid (VIP Fund, Holding Co.)	`color.bg.groupHeader` + `text.onGroupHeader`	`bg.onBrand` — it inverts to pale blue in Dark
Alternate-row striping in a dense data grid	`color.bg.rowStripe`	`bg.subtle` — it is not the grid's striping token
The structural rule in a striped grid (under the header, above totals)	`color.stroke.gridRule`	`stroke.default`
The **row-label** column of a grid (line items, subtotals, totals)	`color.text.rowLabel`	`text.primary`

THE JOB.	USETHE JOB	NEVER USEUSE
A grid **table title**, column-group label or single- row column header	`color.text.gridHeader`	`text.primary`
A file type badge (PDF, DOCX, XLSX…)	`color.file.<kind>.background` + `.text`	`bg.negative` for a PDF — a format is not a status
A **product** tile or mark (Intelligence, Valuations, Waterfalls, Documents)	`color.product.<key>.background` + `.icon`	status tints, or the AI family for Waterfalls
A status **tint** (pill, badge, alert ground)	`color.bg.*Subtle` + matching `color.text.*`	a saturated fill with 12px white text
A card or panel surface	`color.bg.surface`	`bg.page`
A surface lifted above another surface	`color.bg.surfaceRaised`	a lighter hex
The page itself	`color.bg.page`	`bg.surface`
A zebra stripe / hover ground	`color.bg.subtle`	opacity
A **container** edge (card, divider, table frame)	`color.stroke.default` / `.subtle` / `.divider`	`stroke.control`
An **interactive** edge (input, checkbox, anything clickable)	`color.stroke.control`	`stroke.default` (fails 3:1) (R5)
A keyboard focus ring	`color.stroke.focus`, 2px, **outside**	a coloured box-shadow
An icon beside body text	`color.icon.secondary`	`text.tertiary`
An inline link	`color.text.link` + permanent underline	`text.brand` with no underline (R8)
A chart series	`chart.series[i]`, assigned in order	cycling, or hand-picked hexes
"What kind of thing is this?" in search / nav	a PRISM token — see §6	`text.brand`
A scrim behind a modal	`color.overlay.scrim`	`rgba(0,0,0,.5)`

Hover and pressed are tokens, not filters. `bg.brandHover`, `bg.brandPressed`. Never `filter: brightness()` or an opacity change.

SPACE, SIZE AND TYPE

Three scales that never cross (R2): `space.*` is for gaps and padding only, `size.*` is for widths and heights only, and type is chosen by **role** first, then by **step**. The full recipes from `AI-GUIDE.md` §3 and §4:

Two scales that must never be crossed (R2).

`space.*` — gap and padding only.

TOKEN	VALUE	TYPICAL USE
`space['2xs']`	2px	icon-to-text in dense grid furniture
`space.xs`	4px	inside a chip or badge
`space.s`	8px	between a label and its control
`space.m`	12px	card padding, menu row padding
`space.l`	16px	between related blocks
`space.xl`	24px	between sections
`space['2xl']`	32px	around an empty state
`space['3xl']` · `space['4xl']`	48 · 64px	page-level rhythm

`size.*` — width and height only.

TOKEN	VALUE	USE
`size.icon.xs … xl`	12 · 16 · 20 · 24 · 32	every icon. Never hand-size one.
`size.control.s / m / l`	32 · 40 · 48	button and field heights
`size.avatar.xs … xl`	24 · 32 · 40 · 56 · 80	every avatar
`size.iconWell.m / l`	40 · 56	a container holding one glyph (Product Tile mark, Dropzone well) — not `size.avatar`
`size.progressRing.m`	48	Progress Ring diameter
`size.row.compact`	26px	data-grid row min-height
`size.row.header`	40px	data-grid header row height
`size.divider.pinned`	4px	the divider before a pinned Total column (Grid Column Divider, Type = Pinned)
`size.target.minimum`	**44px**	the tap target floor
`size.target.dense`	24px	documented exception: dense grid only

`radius.*` — `2xs` 2 (grid cells) · `xs` 4 (buttons, chips, inputs-in-bar) · `s` 8 (fields, cards, modals) · `m` 16 (large panels) · `full` (avatars, pills).

Targets. Every interactive control reaches `size.target.minimum` (44px) even when the drawn box is smaller. The exception is dense grid furniture, which cannot and is documented as such in `docs/known-gaps.md`.

Pick a **role** by job, then a **step** by prominence. Never pick a role because it happened to be the right size.

ROLE	WHAT IT IS FOR	STEPS
`display`	**Marketing only.** Never in product UI.	s · m · l
`heading`	Section and page titles, table column headers	**xs** · s · m · l · xl · 2xl · 3xl · 4xl · 5xl
`text`	Body copy, table cell content	s · m · l · xl · 2xl
`label`	Form labels, button labels, compact UI furniture	s · m · l
`link`	Inline links — tracks body text without a heading's tracking	s · m · l
`overline`	Uppercase group labels. **Owns +0.8px tracking.**	s · m

```
// Preferred — the semantic components:
<Heading level={1} step="3xl">Valuation summary</Heading>
<Text step="m" tone="secondary">Last reviewed 14 months ago.</Text>
<Overline>Companies</Overline>
// When you need the raw CSS (e.g. for a third-party component):
<div style={type.style('heading', '2xl', 'semiBold')} />
```

The floor is 12px (R10) for everything a user reads. `heading.xs` is the one step below it, at 10px — added for the v1.1 data-grid chrome, and scoped to column headers and in-cell labels. Do not reach for it to make something fit: if a design shows 10px or 11px *prose*, raise it to 12 and expect the layout to reflow.

Never set `letterSpacing`. The Overline role carries its own tracking, and setting it by hand detaches the step.

05 HOW A COMPONENT IS BUILT

FIGMA VARIANTS BECOME PROPS

Figma models every combination as a discrete variant; code collapses each variant *property* into a prop and lets the browser own interaction states. `Button` is 225 Figma variants (Style × Size × State × Type) and three props in code:

```
<Button variant="secondary" tone="negative" size="s">
  Delete version
</Button>
```

`variant` says how much attention the action deserves; `tone` says what kind of action it is; hover, pressed and focus are CSS. A destructive action is `tone="negative"` at whatever prominence it needs, never a primary button recoloured by hand. Across the library: 186 components, 968 documented props, 154 Figma component sets with 1,260 variants, all traced in `docs/components.md`.

ANATOMY

Every component is built the same way:

FILE	ROLE
`Button.tsx`	Typed props and the JSDoc contract: purpose, Figma mapping, accessibility.
`components.css`	One class (`.scalar-button`) that reads semantic tokens only.
`Button.stories.tsx`	Every variant and state in Storybook, with an accessibility panel.
`Button.test.tsx`	Behaviour and accessibility tests run by `verify`.
`index.ts`	The public export from `@scalar/design-system`.

```
.scalar-button {
  --btn-fill: var(--color-bg-brand);
  --btn-fill-hover: var(--color-bg-brand-hover);
  --btn-on: var(--color-text-on-brand);
  gap: var(--space-s);
  border-radius: var(--radius-xs);
  transition: background-color var(--duration-fast) var(--easing-standard);
}
.scalar-button:focus-visible { outline: 2px solid var(--color-stroke-focus); }
```

A tone (positive, warning, negative) only swaps the local `--btn-*` variables. No component hard-codes a value, which is why a token change reaches every component without touching component code.

SIX CONVENTIONS EVERY COMPONENT SHARES

01 **VARIANTS ARE PROPS.**
A Figma variant property becomes a prop. Hover, pressed and focus are CSS, not props. `forcedState` exists only for documentation and visual tests.

02 **NATIVE PROPS PASS THROUGH.**
`ref`, `id`, `className`, `aria-*` and `data-*` land on the element you expect.

03 **`ASCHILD`.**
Render your own element, such as a router `Link`, with the component's classes, behaviour and ref.

04 **CONTROLLED OR UNCONTROLLED.**
`value` + `onChange`, or `defaultValue`; `open` + `onOpenChange` for overlays.

05 **NAMES ARE REQUIRED.**
`ButtonIcon`, `Tabs`, `DataGrid` and `SegmentedControl` need a `label` (or `aria-label`).

06 **TRIGGERS OWN STATE.**
Menus and panels are surfaces; the trigger owns open state and position.

```
<Button asChild variant="secondary"><Link to="/valuations">Valuations</Link></Button>
<ButtonIcon label="More actions" ref={ref} data-testid="row-menu">…</ButtonIcon>
```

06 CHOOSING A COMPONENT BY INTENT

Find the intent, not the shape. Most choices come down to one question:

QUESTION	ANSWER → COMPONENT
Does it act or go somewhere?	Acts → `Button` · goes → `Link`
How many options?	Up to 7 → `Radio` · more → `Select` · a list that grows → `ComboboxPanel`
When does it apply?	At once → `Switch` · on Save → `Checkbox`
Must the user act on the message?	Yes → `Alert` · already done → `Toast`
Keep the page visible?	Yes → `Drawer` · no → `Modal`
Can the figures be edited?	Yes → `Table` · read-only → `DataGrid`

The full intent table from `AI-GUIDE.md` §5:

Find the **intent**, not the shape.

INTENT	COMPONENT	NOT THIS
Run an action	`Button`	`Link`
Navigate somewhere	`Link`	`Button`
Run an action with an unambiguous glyph only	`ButtonIcon` (`label` required)	`Button` with no text
Start a generative action	`AIButton` / `AITool`	a purple `Button`
Ask for one short value	`FormField` + `Input`	a bare `Input`
Ask for long free text	`FormField` + `Textarea`	a tall `Input`
Pick one of **≤7** options	`Radio` group	`Select`
Pick one of **>7** options	`Select`	a long `Radio` group
Toggle a setting that applies **at once**	`Switch`	`Checkbox`
Toggle a setting that needs **Save**	`Checkbox`	`Switch`
Pick a date near today	`DatePicker`	a typed field alone
Pick a far-off date	a typed field **plus** `DatePicker`	`DatePicker` alone
State a fact about an item	`Chip`	`Button`
Show a count or marker	`Badge`	`Chip`

INTENT	COMPONENT	NOT THIS
Show the state of a valuation's **work**	`ModalStatus`	`Chip`
Show the state of a **deal**	`ValuationStatus`	`ModalStatus`
Switch between peer views of one thing	`Tabs`	`Stepper`
Move through an ordered process	`Stepper`	`Tabs`
Page through records	`Pagination`	`Tabs`
Show where the page sits in the hierarchy	`Breadcrumb`	a back button
Message the user must act on	`Alert` (inline, persistent)	`Toast`
Acknowledge something that already happened	`Toast` (transient)	`Alert`
Focused task, page stays visible	`Drawer`	`Modal`
Task needing full attention, must finish first	`Modal`	`Drawer`
Secondary detail on a dense page	`Accordion`	a `Modal`
Surface with nothing in it	`EmptyState`	a blank div
Content not loaded yet (first load, known shape)	`Skeleton`	a spinner
Progress of a task with a known total	`ProgressBar` (determinate)	indeterminate
A dense grid of figures, read-only	`DataGrid` + `Row` + `Cell`	an HTML `<table>`
A spreadsheet-like table: selection, resize, formula view, editing	`Table`	a product-specific table or a fork of `Table`
Explain the control under the pointer	`Tooltip`	placeholder text
Search everything, from anywhere	`GlobalSearch`	a page filter
Compare magnitude across categories	`BarChart`	a line chart
Show change over time	`LineChart`	a bar chart
Show composition **and** a meaningful total	`BarChart type="stacked"`	a donut over time
Show one composition's split	`DonutChart` (≤8 slices)	a pie
Show how a figure becomes another figure	`WaterfallChart`	a bar chart
Page / row / column actions behind a ⋮	`ContextMenu` + `MenuItem`	a row of icon buttons
A main action with alternatives	`SplitButton`	two adjacent Buttons
The page's one floating "add" action	`Fab` (+ `SpeedDial` for a choice)	a fixed primary Button
Switch between 2–4 views of the same content	`SegmentedControl`	`Tabs`, or `ToolSwitch` (product switch only)
Saved views / scenarios / notes as tabs	`ViewTabBar` + `ViewTab`	`Tabs` with a hand-rolled kebab
A page-level or modal form field	`FloatingLabelInput` / `FloatingLabelSelect`	`FormField` (keep that for dense forms)

INTENT	COMPONENT	NOT THIS
Pick from a list that can exceed ~8 items	`ComboboxPanel`	`Select`
A number with a unit, or an integer count	`NumberField`	`Input type="number"`
Several short values (domains, names)	`TagInput`	a comma-separated `Input`
A value the user must copy elsewhere	`CopyField`	a read-only `Input`
A list of like rows the user adds/removes	`RepeatableRow` + tertiary "+ Add"	a grid
Upload a file	`Dropzone`	a bare file input
Confirm a consequential action	`ConfirmationDialog`	`window.confirm` or an `Alert`
A page-level message spanning the page	`Banner`	`Alert` (boxed, in-section)
Work with no known duration	`Spinner`	`ProgressBar` indeterminate
A count-based goal (3/5 answered)	`ProgressRing`	a `Badge`
A preference that applies at once	`SettingRow`	`CheckboxItem`
Row hierarchy in a financial statement	`RowLabelCell` + `GridValueCell`	bold text in a `Cell`
A file in a list	`FileRow` + `FileTypeBadge`	a `Link` with an icon
A label/value pair in a description list	`KeyValueRow`	a two-column `DataGrid`
The open menu of a select / picker trigger (≤ ~8 options)	`SelectMenu` + `SelectMenuOption`	a hand-rolled list of `MenuItem`s
Pending tasks on a grid row	`TaskPill` (`label` required)	a coloured dot or a bare `Badge`
The step an item is on next, to tick off in place	`ActionPill` (+ `StepMenu` behind its caret)	a `SplitButton` or a coloured `Chip`
Show a workflow step that cannot be acted on	`ActionTag`	an `ActionPill` with no handlers
A due date nudged a day at a time	`DueDatePill`	an `Input type="date"` alone
Frame and zoom an uploaded logo or avatar	`ImageCropField` (+ `Dropzone` for the upload)	an `<img>` with a native range input

07 BEHAVIOUR OF THE MAIN COMPONENTS

Everything in this chapter comes with the component. A team that rebuilds a control has to rediscover each of these behaviours. A manual VoiceOver and NVDA pass is still outstanding, so accessibility is strong but not yet certified.

BUILT IN	WHAT IT MEANS
Keyboard	Arrow keys, Home/End and Esc in menus, tabs, dialogs and the date picker.
Focus	Opening moves focus in; closing returns it to the control that opened it.
Screen readers	Correct roles and states; every chart carries a data table behind it.
Contrast	Fill and text tokens ship as pairs, so a filled control is legible by default.
Composable	`ref`, `className`, `data-*` and `asChild` work everywhere.
States	Loading, disabled, selected and error are props, announced to assistive tech.

ACTIONS — BUTTON, BUTTONICON, SPLITBUTTON

COMPONENT	PROP OR INPUT	BEHAVIOUR
`Button`	`variant`	Weight: primary, secondary or tertiary.
`tone`	Meaning: main, positive, warning, negative. `warning` pairs yellow with near-black text (R4).	
`size`	xs, s, m, l draw at 24, 32, 40, 48px; `s` is dense table use only.	
`loading`	Spinner replaces the leading icon; stays focusable (`aria-disabled`), `aria-busy`, announces “Loading”.	
`selected` · `asChild`	Toggle exposed as `aria-pressed` · renders your link as the button.	
`ButtonIcon`	`label` (required)	Becomes `aria-label`. All sizes meet the 24px target rule; pair with `Tooltip` if the icon is unclear.
`SplitButto n`	main half	Runs the main action; the caret is a separate hit target.
caret · ↓	Opens the menu and moves focus to the first item.	
Esc · Tab · outside	Closes; focus returns to the caret.	

FORMS — FORMFIELD, INPUT, SELECT, COMBOBOXPANEL

COMPONENT	PROP OR INPUT	BEHAVIOUR
`FormField`	`label` · `required`	Wires `id`, `required`, `aria-invalid` and `aria-describedby` into the control; merges an existing describedby.
`state="error"`	The helper text becomes the error and is read with the field.	
`Input`	standalone	Needs its own `aria-label`. Native props land on the `<input>`; `className` on the wrapper.
`Select`	over ~7 options	Native `<select>`: keyboard and screen reader behaviour come from the browser.
`Radio` · `Switch` · `Checkbox`	choice	Up to 7 visible options → Radio. Applies at once → Switch. Waits for Save → Checkbox.
`ComboboxPanel`	typing	Focus stays in the search input (`aria-activedescendant`). You pass already-filtered items.
↑ ↓ · Enter	Moves the active option, skipping disabled ones; Enter selects.	
Esc	First clears the query, then calls `onEscape`.	
`autoFocus`	Off by default; pass it when the panel mounts on open.	

DATA — TABLE AND DATAGRID

`Table` is the one spreadsheet-like table for the whole platform: Valuations, Waterfalls, Financials, Cap Table, Companies, Reports and Workboard all render it, and differences come only from `columns`, `rows`, `cells` and props — never a fork. It is an ARIA `grid` with 26px rows, 40px headers and a sticky title column. `DataGrid` is the read-only layout shell (ARIA `table`).

COMPONENT	KEY OR INPUT	BEHAVIOUR
`Table`	Tab · arrows	One tab stop; arrows move the active cell (roving focus).
Shift + arrows	Extends a rectangular selection, drawn as one outline.	
Ctrl/⌘ + arrows	Jumps to the edge of the data.	
Ctrl/⌘ + Space · Shift + Space	Selects the column · the row.	
Enter / F2 · Esc	Edits an editable cell · cancels, then clears the selection.	
Ctrl/⌘ + C	Copies the selection as tab-separated values.	
drag a column edge	Resizes; never narrower than its longest value.	
`cell.kind`	Sets text colour and editability; fill precedence is error › selected › pressed › hover › zebra › default.	

COMPONENT	KEY OR INPUT	BEHAVIOUR
`DataGrid`	`label` (required)	`role="table"`; cells sit in the normal Tab order. Never pass `role="grid"`.
`maxHeight`	The header sticks while the body scrolls; wide data scrolls sideways inside the grid.	
header row	Defines the column tracks; rows are subgrids, so header and body never drift.	

OVERLAYS — MODAL, CONFIRMATIONDIALOG, DRAWER

COMPONENT	EVENT OR PROP	BEHAVIOUR
`Modal`	open	Traps focus and locks page scroll; the scrim is a sibling.
Esc	Closes the top-most modal only.	
close	Focus returns to whatever opened it.	
`size` · `role`	s, m, l, xl · `role="alertdialog"` for urgent content.	
`ConfirmationDialog`	scrim click	Does not dismiss: the user must choose.
`destructive`	`role="alertdialog"`; opens with focus on Cancel, the safe choice. Built on `Modal`.	
`Drawer`	open	Focus moves in; the page stays visible behind.
Esc · close	Closes; focus returns to the opener.	
`modal={false}`	No scrim; Tab and outside clicks leave the drawer.	

Whole attention, must finish first → `Modal`. Focused task with context → `Drawer`.

TABS, MENUS AND PICKERS

COMPONENTDATA — TABLE AND DATAGRID	KEY OR PROP	BEHAVIOURTABLE
`Tabs`	← / →	Moves focus and selects the tab at once (automatic activation).
Home / End · Tab	First or last tab · Tab leaves the set.	
`value`	Wires a `TabItem` to the `TabPanel` with the same value.	
`ContextMenu`	↑ ↓ · Home/End · type	Moves through items; typing jumps to a match.
→ / ←	Opens or closes an inline submenu.	
Esc · Tab	Calls `onClose`; the trigger owns open state and position.	

COMPONENTDATA — TABLE AND DATAGRID	KEY OR PROP	BEHAVIOURTABLE
`DatePicker`	Tab · arrows	One tab stop; ← → a day, ↑ ↓ a week; Home/End week start or end.
PgUp / PgDn (+ Shift)	A month, or a year with Shift.	
Enter / Space · Esc	Chooses the date · calls `onEscape`.	
layout	Always six week rows, so the popover never changes height. Pair it with a typed date field.	

FEEDBACK — ALERT, TOAST, TOOLTIP

COMPONENT	ASPECT	BEHAVIOUR
`Alert`	lifetime	Inline; stays until the condition changes. For messages the user must act on.
`tone`	`negative` is `role="alert"` (assertive); info, positive and warning are `role="status"`.	
content	Say what happened and what to do next.	
`Toast`	lifetime	Transient; reports something already done. Never the only copy of information.
`ToastViewport`	Mount toasts in it: a live region that exists before they arrive.	
timer	The library ships none; when you add one, pause it on hover.	
`Tooltip`	focus or hover	Opens on keyboard focus too, not hover alone.
Esc	Closes it from anywhere while open.	
content	Read with the control via `aria-describedby`; never essential information.	

PAGE LEVEL — GLOBALSEARCH AND PAGETEMPLATE

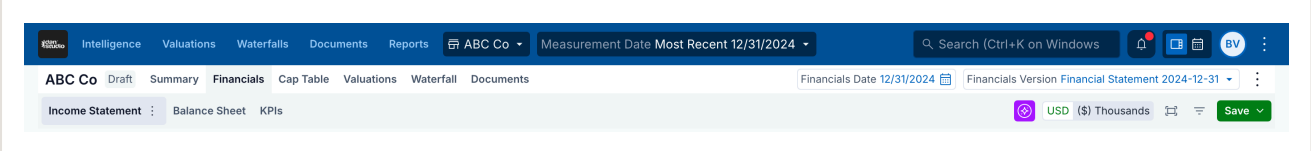
COMPONENT	KEY OR SLOT	BEHAVIOUR
`GlobalSearch`	⌘K	The app opens it (a modal dialog); focus goes to the search input.
↑ / ↓	Moves the highlight; focus stays in the input.	
Tab · Backspace	Pushes the highlighted result onto the scope · pops it.	
scope chips	Three at most: Firm › Company › Document.	
Esc	Closes from any scope depth; focus returns to the opener.	
`PageTemplate`	slots	`navigation`, `companyInfo`, `subNavigation`, `children`, `drawer` — each optional.
body slot	The page's single `<main>`; scrolls on its own so the chrome stays put.	
styling	Never restyle a slot from outside; change the component you pass in.	

08 THE 3-TIER NAVIGATION SYSTEM

Source: `docs/navigation-3-tier-system.md` in Prism-v1, written from the New-Navigation Figma file. Items the spec marks as open are listed at the end of this chapter; do not invent behaviour for them.

MENTAL MODEL

Every authenticated screen renders the same vertical stack: Tier 1, then Tier 2, then Tier 3, then content. Each tier answers one question and narrows the scope of the one above. Set context at the top, then trust every table below to reflect it.

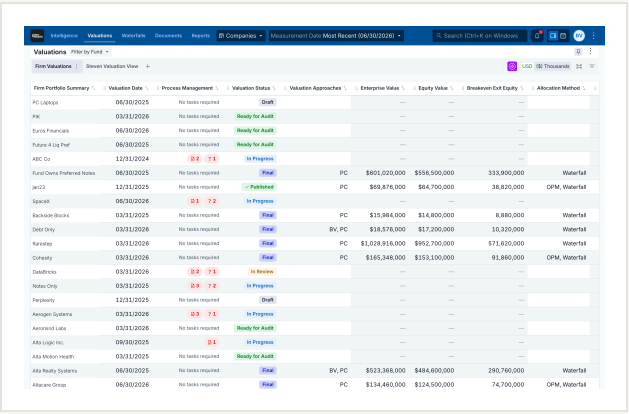


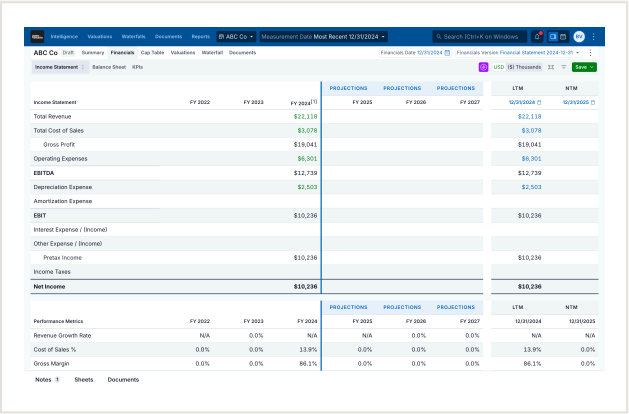
TIER	COMPONENT	QUESTION IT ANSWERS	SCOPE	HEIGHT	CHANGES WHEN…
1	Primary Menu	Where am I, and in what context?	Global	48px, full bleed	Never structurally; only its state.
2	Company info	Which entity, and which area of it?	Firm or company	32px, inset 16px	The page or Tier 2 tab changes.
3	Tertiary Menu	Which table, and what can I do to it?	Table or view	40px, inset 16px	The Tier 3 view tab changes.

Tiers stack with no gap; Tiers 2 and 3 share the 16px inset. Tiers 2 and 3 are optional per screen: hide them rather than render an empty bar. Total chrome is 48px (Tier 1 only), 80px (Tiers 1–2) or 120px (all three).

IN THE PRODUCT

The same three tiers at firm level and at company level, from the coded product app:





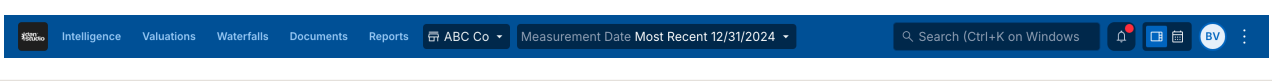
On the firm Valuations page (left) the Company selector reads *Companies*, so the Valuations section link is active, Tier 2 shows the page title with Filter by Fund, and Tier 3 holds the view tabs and toolbar. On ABC Co's Financials page (right) the selector names the company, so no section link is active; Tier 2 becomes the company bar with its shared tab row and entity selectors.

GLOBAL CONTEXT

All data below Tier 1 is a function of two values that live in Tier 1: **Company scope** and **Measurement Date**.

- **Scope picks the pages.** *Companies* shows firm-level pages; one company shows that company's pages.
- **The date persists** across sections, companies and tabs, until the user changes it. It can be a date, *Most Recent*, or *Most Recent* with a date.
- **Change means re-fetch.** Changing either value re-derives all data on screen, keeping the user's Tier 2 and Tier 3 position wherever it still exists.
- **One source.** Read both from shared context; never keep a local copy.

TIER 1 — PRIMARY MENU



- **Left · navigate:** firm logo, then five section links in fixed order — Intelligence, Valuations, Waterfalls, Documents, Reports.
- **Centre · context:** Company selector (*Companies* or one company) and Measurement Date.
- **Right · utilities:** Search (Ctrl+K from anywhere), Notifications with an unread count, the app switcher (Portfolio Valuations ↔ Workboard, staff only), Avatar, and ⋮.
- **Active link:** zero or one. None on the Homepage, All Companies or any company page.
- **Fixed structure:** never add, remove or reorder items for one page; a new Tier 1 element needs Design approval.
- **⋮ Global Action Menu:** firm group (firm name, Switch Firm, User Management, Comp Groups, Audit Logs, Firm Settings), user group (name, Account Settings, Feedback), Sign Out. Switch Firm appears only when the user can reach more than one firm.

In code: `PrimaryMenu`, `MainMenuItem`, `CompanyDropdown`, `SearchBar`, `Notification`, `ToolSwitch`, `Avatar`.

TIER 2 — COMPANY INFO

Two patterns share one bar.

Firm pages (scope *Companies*). Left: page title, optional Filter by Fund, then tabs. Right: page actions, Pin and ⋮.



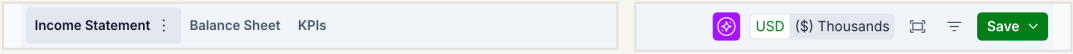
Company pages (one company). Left: company name, status badge and the shared tab row — Summary, Financials, Cap Table, Valuations, Waterfall, Documentation. Right: entity selectors that follow the active tab.



ACTIVE COMPANY TAB	RIGHT-ZONE CONTROLS
Summary · Valuations · Waterfall	Equity Value · Unrealized Firm Total · Valuation Version ▾ · ⋮
Financials	Financials Date (calendar) · Financials Version ▾ · ⋮
Cap Table	Cap Table version ▾ · ⋮

Information Labels come in three modes: a read-only KPI (green value), a selectable value (blue, with ▾ or a calendar) and a version-only selector. A selectable label is entity-wide: changing it changes every Tier 3 table. Exactly one tab is active when tabs are present. The Homepage hides Tier 2.

TIER 3 — TERTIARY MENU



View tabs (left). Exactly one is active, filled and with a ⋮ for view management (rename, reorder, duplicate). An optional "+" adds a view; what it adds is page-specific.

Toolbar (right), in fixed order — omit a control the page doesn't support, never reorder:

01 **AI**
— always present when Tier 3 is visible; scoped to the current table.

02 **CURRENCY AND UNITS**
— applies to every table on the page, so per-table currency selectors are removed.

03 **AUTOFIT**
— only if the table supports it.

04 **FILTER**
— column filtering for the current table.

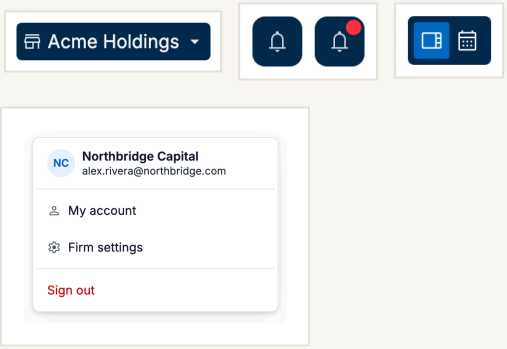
05 **PRIMARY ACTION**
— `Save ▾` on Company Valuations, Cap Table and Financials; `Save Notes & Documents ▾` on Waterfalls; `Bulk Actions` on firm Valuations; none on Intelligence and Company Summary.

06 ⋮
— table-level management.

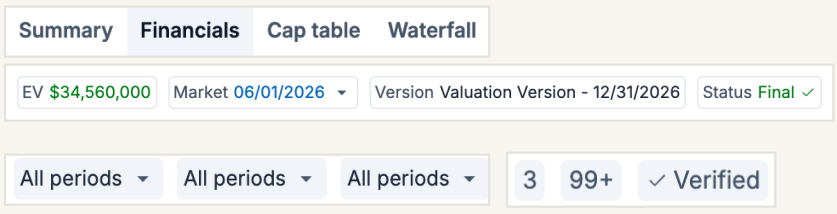
Show Tier 3 only when the page body is a table or a switchable view.

THE PARTS

Every bar is composed from shared components; never hand-build one.



Tier 1: `CompanyDropdown` · `Notification` · `ToolSwitch` · `UserMenu`.



Tier 2: `SecondaryMenu` · `InformationLabel` · `FilterDropdown` · `Badge`.



Tier 3: `TertiaryMenu` · `AITool` · `CurrencySelector`, plus standard `ButtonIcon`s and a `Button` for the primary action.

WHERE A NEW CONTROL GOES

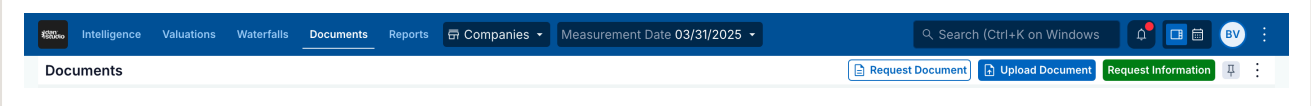
Place a control by the scope it affects.

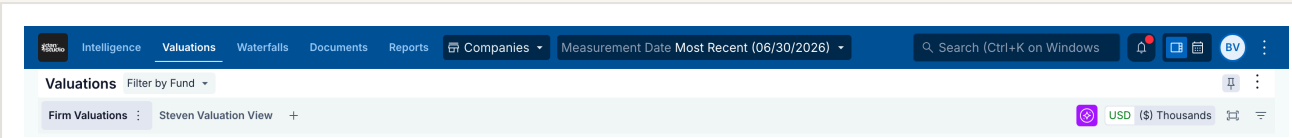
IF THE ELEMENT…	PUT IT IN	EXAMPLE
Changes context for the whole product	Tier 1 — requires Design approval	—
Applies to the whole entity	Tier 2, right zone	Valuation Version, Financials Date
Narrows a firm-level page by fund	Tier 2, Filter dropdown beside the title	Filter by Fund
Is a page-level action with no table	Tier 2, right-zone Button	Upload Document
Switches between areas of an entity	Tier 2 tab	Cap Table
Switches between tables inside an area	Tier 3 view tab	Balance Sheet
Affects only the current table	Tier 3 toolbar	Filter, Autofit
Commits changes to the current table	Tier 3 primary action	Save ▾
Is a rarely used management action	The ⋮ of the tier whose scope it affects	Rename view → Tier 3 ⋮

Never put table-level controls in Tier 2 or entity-level selectors in Tier 3; never render an empty bar; keep ⋮ as the right-most slot of any bar that has one.

SCREEN MAP

SCREEN	TIER 1 ACTIVE LINK	COMPANY SELECTOR	TIER 2SHOW TIER 3 ONLY WHEN THE PAGE BODY IS A TABLE OR A SWITCHABLE VIEW.	TIER 3THE PARTS
Homepage Dashboard	—	Companies	hidden	hidden
Intelligence · Valuations	matching link	Companies	shown	shown
Waterfalls	Waterfalls	Companies	shown	Phase 2 only
Documents · Reports	matching link	Companies	shown	hidden
All Companies	—	Companies	shown	hidden
Company landing	—	company name	shown	hidden
Company › Summary … Waterfall	—	company name	shown	shown





IN CODE

In `PageTemplate` the tiers map to slots: `navigation` ← Tier 1 (`PrimaryMenu`), `companyInfo` ← Tier 2 (`CompanyInfo` + `SecondaryMenu`), `subNavigation` ← Tier 3 (`TertiaryMenu`). The spec proposes a data-driven contract so a new screen is a configuration rather than new markup (a proposal; adapt names to the codebase):

```
interface ScreenNavConfig {
  tier1: { activeSection?: Section };
  tier2?: Tier2Config;   // undefined = hidden
  tier3?: Tier3Config;   // undefined = hidden
}
interface Tier3Config {
  views: { id: string; label: string }[];
  activeViewId: string;
  canAddView?: boolean;  // renders "+"
  toolbar: { ai: true; currency?: boolean; autofit?: boolean;
             filter?: boolean; primaryAction?: ButtonConfig };
}
```

Open — ask Design, don't invent: Pin behaviour on firm Tier 2 · the Company › Documentation tab · whether the company landing defaults to Summary · what “+” adds on Company Summary and Waterfalls · AI tool behaviour · All Companies right-zone actions.

09 IMPLEMENTATION EXAMPLES

A PRODUCT SCREEN

Every screen starts in `PageTemplate`. Pass the tiers into their slots and put the page in `children`; the body slot is already inset and scrolls on its own. The fastest route to a new screen is to copy the nearest of the 103 frames in `apps/product`.

```
<PageTemplate
  navigation={<PrimaryMenu logo={…} end={…}>
    <MainMenuItem current>Valuations</MainMenuItem>
  </PrimaryMenu>}
  companyInfo={<CompanyInfo … />}
  subNavigation={<TertiaryMenu>…</TertiaryMenu>}
  drawer={<WorkspaceDrawer … />}
>
  <Tabs label="Valuation views">
    <TabItem active>Summary</TabItem> …
  </Tabs>
  <Table tableId="val" title="Valuation Summary" … />
</PageTemplate>
```

A FORM IN A MODAL

Thirteen lines, no styling written. Focus is trapped, Esc closes and focus returns; label, required and error are wired to the input; the error is read with the field; `loading` keeps the button focusable and announces it.

```
<Modal open={open} onClose={close} title="New comp group" size="s"
  footer={<>
    <Button variant="tertiary" onClick={close}>Cancel</Button>
    <Button onClick={save} loading={saving}>Create</Button>
  </>}>
  <FormField label="Group name" required
    state={error ? 'error' : 'default'}
    helperText={error ?? 'Shown in reports.'}>
    <Input value={name} onChange={e => setName(e.target.value)} />
  </FormField>
</Modal>
```

For a destructive confirmation use `ConfirmationDialog` rather than building one on `Modal`.

A VALUATION TABLE

Describe the data; the Table does the rest. `kind` sets the colour and editability (`readable`, `editable` in blue, `sourced` in green, `total`, `draft`); `format` sets how the value prints (`currency`, `perShare`, `percent`, `rate`, `multiple`, `count`, `text`). An `error` string on a cell gives it the error fill, `aria-invalid` and a read-out message.

```
const columns: TableColumnDef[] = [
  { id: 'ev', header: 'Equity Value', group: 'Valuation' },
];
const rows: TableRowDef[] = [
  { id: 'sub', label: 'Subscription', level: 1, kind: 'default' },
  { id: 'tot', label: 'Total revenue', level: 0, kind: 'total' },
];
const cells = {
  'sub:ev': { id: 'EV.SUB', kind: 'sourced', value: 401139375, format: 'currency' },
};
<Table tableId="val" title="Valuation Summary"
  columns={columns} rows={rows} cells={cells}
  onCellCommit={(id, value) => save(id, value)} />
```

10 BUILDING WITH AN AI AGENT

The guide is written so an agent can follow it step by step:

01 READ THE CONTRACT.
The consuming repo's `CLAUDE.md` points the agent to `AI-GUIDE.md`.

02 PICK BY INTENT.
The component recipe (chapter 6).

03 FILL BY JOB.
The colour, space and type recipes (chapter 4).

04 COMPOSE.
In `PageTemplate`, starting from the nearest coded screen.

05 VERIFY.
`npm run verify`, then the definition of done below.

Paste into the consuming repo's `CLAUDE.md`:

```
## UI
All product UI is built from `@scalar/design-system`. Before writing any
component, read `node_modules/@scalar/design-system/AI-GUIDE.md` — it is the
contract. Never hand-write a colour, spacing value or font size.
```

ANTI-PATTERNS

The token linter reads CSS and JSX `style` objects alike, so value mistakes fail `verify`; structural mistakes are caught in review. From `AI-GUIDE.md` §8:

✗ DON'T	✓ DO
`style={{ color: '#0268c1' }}`	`style={{ color: color.text.brand }}`
`style={{ padding: 12 }}`	`style={{ padding: space.m }}`
`<Cell><Link href=…>{name}</Link></Cell>` as a row's first cell	`<RowHeader href=…>{name}</RowHeader>`
A screen that assembles `PrimaryMenu` + `CompanyInfo` + `WorkspaceDrawer` itself	`<PageTemplate …>{content}</PageTemplate>`
Shipping the `BodySlot` placeholder, or wrapping the body in extra padding / `max-width` / a scroll box	Real `children`; the slot already scrolls and is inset
`style={{ fontSize: 13 }}`	a `type.style()` role/step, min 12px (`heading.xs` 10px only for data-grid chrome)

✗ DON'T	✓ DO
`width: space.l`	`width: size.icon.s` (R2)
`background: bg.brand; color: text.primary`	`color: text.onBrand` (R3)
white text on `bg.warning`	`text.onWarning` (R4)
`border: 1px solid stroke.default` on an input	`stroke.control` (R5)
a red dot as the only error signal	a dot **plus** words or an icon (R8)
`filter: brightness(0.9)` for hover	`bg.brandHover`
`letterSpacing: '0.8px'` for uppercase	the `overline` role (R10)
a `<table>` for a dense figure grid	`DataGrid` + `Row` + `Cell`
a `Button` that navigates	`Link`
a `Link` that submits or deletes	`Button`
`role="heading"` on `Typography`	`variant="heading"` — `role` is the DOM attribute
PRISM colour to show "you can't open this"	PRISM says *what it is*, never permission (R11)

DEFINITION OF DONE

From `AI-GUIDE.md` §9 — the same list for people and agents:

- [ ] The screen is built in `PageTemplate` and no `BodySlot` placeholder is left.

```
npm run verify # typecheck + token-contract lint
```

The linter reads both CSS **and** JSX `style={{ … }}` objects, so it catches a raw hex, a hard-coded gap, type set from a sizing token, and the `space` / `size` scales being crossed — wherever you wrote them.

Then check by hand:

- [ ] No raw hex, `rgb()`, or hard-coded px in anything you wrote.
- [ ] Every filled surface uses its `on*` text token (R3).
- [ ] Every interactive edge uses `stroke.control`, not `stroke.default` (R5).
- [ ] Every interactive control reaches 44px, or is documented dense grid.
- [ ] Every status carries words or an icon, not just colour (R8).
- [ ] Focus is visible on every control, 2px `stroke.focus`, outside the box.
- [ ] It reads correctly in **both** Light and Dark.
- [ ] No text below 12px (R10) — the only exception is `heading.xs` (10px) in data-grid column headers and in-cell labels.
- [ ] Charts with ≥3 series carry direct labels.

11 GETTING STARTED AND QUALITY

Setup — three steps happen once per app:

```
// 1 · install, pinned to a tag
npm install github:Bayardo88/Prism-v1#v1.1.0
// 2 · styles once, 3 · provider once
import '@scalar/design-system/styles.css';
<ScalarProvider mode="light" viewport="auto"><App /></ScalarProvider>
// 4 · build
<Card title="Valuations"><Button>Save</Button></Card>
```

Works with Next.js, Vite and CRA; `viewport="auto"` is safe during server rendering.

Quality you can check: 406 Storybook stories with an accessibility panel and a Light/Dark toggle; one `npm run verify` gate run locally and on every pull request; this reference, covering every token and component; and every Figma component set traced to a React export in `docs/components.md`.

Reach for the component first. Read the guide, browse Storybook, copy a screen from `apps/product`, run the gate. If something is missing, open an issue instead of forking a component.

II

SYSTEM RULES

These are the system's rules (R1–R13 from the Figma contract, plus R14 for code), restated for code. Breaking one is a bug, not a style preference.

14

rules · R1–R13 from the Figma contract, plus R14 for code

SYSTEM RULES · R1–R14

BREAKING ONE IS A BUG

01

RULE R1

Never use a `--primitive-*` token in product code. The primitive collections are published and visible in Figma and in `tokens.css`, but they are off-limits to developers and AI agents — **only the Semantic collections are for use.** Primitives have no mode and will not respond to the theme.

02

RULE R2

Colour from `color.*`. Type from `type.style()` or a `scalar-type-*` class. **Gap and padding** from `space.*`. Radius from `radius.*`. **Width and height** from `size.*`. Never cross `space` and `size`.

03

RULE R3

Every filled surface has a matching on-colour. `bg.brand` pairs with `text.onBrand` and `icon.onBrand`. `text.primary` **on a filled surface is a contrast bug.**

04

RULE R4

Yellow is the exception. `bg.warning` is identical in both modes and `text.onWarning` is near-black in both. White can never clear AA on yellow.

05

RULE R5

`stroke.default`, `stroke.subtle` and `stroke.divider` are non-interactive **container** edges (~1.2–1.7:1, allowed — WCAG 1.4.11 covers components, not containers). Anything clickable, focusable or typable takes `stroke.control`, which clears 3:1.

06

RULE R6

The type ramp has three modes: Desktop (1440, default), Desktop Large (1920), Mobile (393). Set it on `ScalarProvider`. **Never hand-resize type.**

07

RULE R7

Weight is orthogonal to size. Changing weight never moves size, line-height or tracking.

08

RULE R8

Colour alone never carries meaning. Status needs an icon or words. Chart series need direct labels. Links need a permanent underline.

09

RULE R9

Elevation is a ladder: `raised` < `overlay` < `modal`. Shadow colours are token-bound and deepen automatically in Dark.

10

RULE R10

Text floor is 12px for everything a user reads. `heading.xs` (10px) is the single sanctioned exception, for data-grid chrome only — column headers and in-cell labels, never prose, form labels or helper text. Display is marketing-only. Uppercase labels use the Overline role, which owns the +0.8px tracking — never set `letterSpacing` yourself.

12

RULE R12

33 of the 78 type tokens are identical across all three modes. That is intentional. Do not "fix" them.

14

RULE R14

Pages are always light mode, even when the machine's system style is dark. `ScalarProvider` defaults to `mode="light"` and the stylesheet does not follow `prefers-color-scheme`. Never ship `mode="dark"`, never set `data- theme="dark"` on a page, never add your own dark styling. Dark exists for the library only.

11

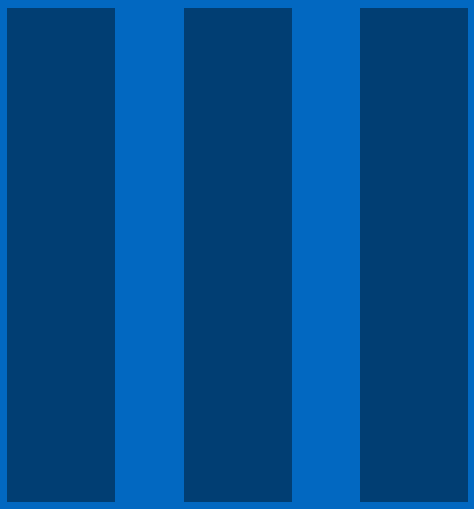
RULE R11

PRISM says what an item **represents** — never access, availability or permission.

13

RULE R13

If you add a `bg.*` or a PRISM concept, re-run the contrast matrix against Page, Surface, Surface Raised and Subtle before shipping.



DESIGN TOKENS

180
semantic colour tokens (Light + Dark)

142
primitive colour steps (reference only)

60
scale, elevation and motion tokens

80
typography tokens × 3 viewport modes

Generated from `src/tokens/tokens.json`, which is generated from `src/styles/tokens.css`, which was extracted from the live Figma library `anrnTIJKgu27zV224h7vON (Scalar_Design_System-v1.1)`.

DESIGN TOKENS

SEMANTIC COLOUR

Every token below responds to the theme. Use these, never the primitives.

TEXT

On a filled surface always use an `on*` token — `text-primary` on a fill is a contrast bug (rule R3).

TOKEN	LIGHTSRC/TOKENS/TOKENS.JSON	DARK, WHICH IS GENERATED FROM
--color-text-ai	#a614ff	#d692ff
--color-text-brand	#0268c1	#68b1f1
--color-text-brand-hover	#02539a	#9acbf6
--color-text-brand-pressed	#013e73	#cde5fa
--color-text-disabled	#94a3b8	#475569
--color-text-editable	#0268c1	#68b1f1
--color-text-grid-header	#01294c	#cde5fa
--color-text-inverse	#f8fafc	#0f172a
--color-text-link	#2a3ef4	#9ea7fa
--color-text-link-hover	#0a1cc2	#ced3fd
--color-text-link-pressed	#071592	#e9ebfe
--color-text-negative	#cb0000	#ff999c
--color-text-negative-hover	#9c0000	#ffc8c9
--color-text-negative-pressed	#6c0003	#ffe6e7
--color-text-on-ai	#ffffff	#0f172a
--color-text-on-brand	#ffffff	#0f172a
--color-text-on-brand-inactive	#9acbf6	#013e73
--color-text-on-brand-muted	#94a3b8	#475569
--color-text-on-brand-subtle	#f1f5f9	#1e293b
--color-text-on-disabled	#475569	#cbd5e1
--color-text-on-group-header	#ffffff	#ffffff
--color-text-on-negative	#ffffff	#0f172a

TOKEN	LIGHTSRC/TOKENS/TOKENS.JSON	DARK, WHICH IS GENERATED FROM
--color-text-on-positive	#ffffff	#0f172a
--color-text-on-warning	#0f172a	#0f172a
--color-text-positive	#007e17	#31c37b
--color-text-positive-hover	#006012	#7ed6a5
--color-text-positive-pressed	#00420f	#bbe8ce
--color-text-primary	#0f172a	#f8fafc
--color-text-row-label	#013e73	#9acbf6
--color-text-secondary	#1e293b	#e3e8f0
--color-text-sourced	#007e17	#31c37b
--color-text-tertiary	#475569	#cbd5e1
--color-text-warning	#996600	#ffcc66
--color-text-warning-hover	#664400	#ffdd99
--color-text-warning-pressed	#332200	#ffeecc

BACKGROUND

TOKEN--COLOR-TEXT-ON-WARNING	LIGHT	DARK#0F172A
--color-bg-ai	#a614ff	#c668ff
--color-bg-ai-subtle	#fbf4ff	#5d0095
--color-bg-brand	#0268c1	#3597ed
--color-bg-brand-hover	#02539a	#68b1f1
--color-bg-brand-pressed	#013e73	#9acbf6
--color-bg-brand-subtle	#e9f3fd	#01294c
--color-bg-disabled	#e3e8f0	#334155
--color-bg-group-header	#01294c	#013e73
--color-bg-inverse	#0f172a	#f8fafc
--color-bg-negative	#cb0000	#ff2f3d
--color-bg-negative-hover	#9c0000	#ff686e
--color-bg-negative-pressed	#6c0003	#ff999c
--color-bg-negative-subtle	#ffe6e7	#3b0104
--color-bg-on-brand	#01294c	#cde5fa
--color-bg-page	#f8fafc	#0f172a
--color-bg-positive	#007e17	#00b04f
--color-bg-positive-hover	#006012	#31c37b

TOKEN--COLOR-TEXT-ON-WARNING	LIGHT	DARK#0F172A
--color-bg-positive-pressed	#00420f	#7ed6a5
--color-bg-positive-subtle	#e0f5e9	#002409
--color-bg-row-stripe	#f8fafc	#334155
--color-bg-subtle	#f1f5f9	#1e293b
--color-bg-surface	#ffffff	#1e293b
--color-bg-surface-raised	#ffffff	#334155
--color-bg-unread	#ff2f3d	#cb0000
--color-bg-warning	#ffbb33	#ffbb33
--color-bg-warning-hover	#ffaa00	#ffcc66
--color-bg-warning-pressed	#cc8800	#ffdd99
--color-bg-warning-subtle	#fff7e8	#332200

STROKE

`default`, `subtle` and `divider` are non-interactive container edges. Anything clickable, focusable or typable takes `control` (rule R5).

TOKEN	LIGHT	DARK
--color-stroke-ai	#a614ff	#d692ff
--color-stroke-brand	#0268c1	#3597ed
--color-stroke-brand-hover	#02539a	#68b1f1
--color-stroke-brand-pressed	#013e73	#9acbf6
--color-stroke-control	#64748b	#94a3b8
--color-stroke-default	#cbd5e1	#334155
--color-stroke-disabled	#e3e8f0	#334155
--color-stroke-divider	#e3e8f0	#334155
--color-stroke-focus	#037de8	#68b1f1
--color-stroke-grid-rule	#94a3b8	#64748b
--color-stroke-negative	#cb0000	#ff2f3d
--color-stroke-negative-hover	#9c0000	#ff686e
--color-stroke-negative-pressed	#6c0003	#ff999c
--color-stroke-on-brand	#ffffff	#0f172a
--color-stroke-positive	#007e17	#00b04f
--color-stroke-positive-hover	#006012	#31c37b
--color-stroke-positive-pressed	#00420f	#7ed6a5
--color-stroke-strong	#475569	#cbd5e1
--color-stroke-subtle	#e3e8f0	#1e293b

TOKEN	LIGHT	DARK
--color-stroke-warning	#996600	#ffcc66
--color-stroke-warning-hover	#664400	#ffdd99
--color-stroke-warning-pressed	#332200	#ffeecc

ICON

TOKEN--COLOR-BG-WARNING-HOVER	LIGHT	DARK#FFAA00
--color-icon-ai	#a614ff	#d692ff
--color-icon-brand	#0268c1	#3597ed
--color-icon-brand-hover	#02539a	#68b1f1
--color-icon-brand-pressed	#013e73	#9acbf6
--color-icon-disabled	#94a3b8	#475569
--color-icon-inverse	#f8fafc	#0f172a
--color-icon-negative	#cb0000	#ff2f3d
--color-icon-negative-hover	#9c0000	#ff686e
--color-icon-negative-pressed	#6c0003	#ff999c
--color-icon-on-brand	#ffffff	#0f172a
--color-icon-on-brand-inactive	#cbd5e1	#334155
--color-icon-on-brand-muted	#94a3b8	#475569
--color-icon-on-brand-subtle	#f1f5f9	#1e293b
--color-icon-positive	#007e17	#00b04f
--color-icon-positive-hover	#006012	#31c37b
--color-icon-positive-pressed	#00420f	#7ed6a5
--color-icon-primary	#0f172a	#f8fafc
--color-icon-secondary	#475569	#94a3b8
--color-icon-warning	#996600	#ffcc66
--color-icon-warning-hover	#664400	#ffdd99
--color-icon-warning-pressed	#332200	#ffeecc

OVERLAY & SHADOW

TOKEN	LIGHT	DARK
--color-overlay-medium	#00000080	#ffffff80
--color-overlay-scrim	#00000080	#00000099
--color-overlay-strong	#000000b2	#ffffffb2
--color-overlay-subtle	#0000004d	#ffffff4d

SHADOW

TOKEN	LIGHT	DARK
--color-shadow-ambient	#00000014	#00000066
--color-shadow-key	#00000029	#00000099

CHART

Every pair separates by at least ΔE2000 6 under normal vision, deuteranopia, protanopia and tritanopia (enforced by `npm run lint:contrast`).

TOKEN	LIGHT	DARK
--color-chart-axis	#64748b	#64748b
--color-chart-grid	#e3e8f0	#1e293b
--color-chart-negative	#cb0000	#ff2f3d
--color-chart-positive	#007e17	#00b04f
--color-chart-series-1	#8e1242	#f088bd
--color-chart-series-1-subtle	#edd9e1	#6e4d6c
--color-chart-series-2	#1b8cfc	#b4ef40
--color-chart-series-2-subtle	#dbedff	#57743d
--color-chart-series-3	#968e23	#2c9f59
--color-chart-series-3-subtle	#eeeddc	#235646
--color-chart-series-4	#1936c0	#aa6aff
--color-chart-series-4-subtle	#dadff5	#534285
--color-chart-series-5	#e05f8d	#4fdb9f
--color-chart-series-5-subtle	#fae5ed	#316d61
--color-chart-series-6	#715c09	#f4c4ff
--color-chart-series-6-subtle	#e8e5d8	#6f6485
--color-chart-series-7	#33786c	#8fb93c
--color-chart-series-7-subtle	#dee9e7	#49603b
--color-chart-series-8	#748156	#b57b8c
--color-chart-series-8-subtle	#e9ebe4	#57485a

TOKEN	LIGHT	DARK
--color-chart-total	#0f172a	#f8fafc

PRISM

Semantic colour for **what an item represents** — never access, availability or permission (rule R11). `primary` is graphic only, `background` is the tint, `text` is the only member safe for small text.

TOKEN	LIGHT#235646	DARK
--color-entity-firm-primary	#cc8800	#ffbb33
--color-entity-firm-background	#ffeecc	#332200
--color-entity-firm-text	#664400	#ffcc66
--color-entity-company-primary	#0268c1	#3597ed
--color-entity-company-background	#cde5fa	#01294c
--color-entity-company-text	#02539a	#68b1f1
--color-data-document-primary	#0a1cc2	#3d4ff5
--color-data-document-background	#ced3fd	#020731
--color-data-document-text	#0a1cc2	#9ea7fa
--color-data-version-primary	#007e17	#00b04f
--color-data-version-background	#bbe8ce	#002409
--color-data-version-text	#006012	#31c37b
--color-data-measurement-date-primary	#0891b2	#22d3ee
--color-data-measurement-date-background	#cffafe	#164e63
--color-data-measurement-date-text	#0e7490	#67e8f9
--color-destination-page-primary	#be185d	#ea589e
--color-destination-page-background	#fce7f3	#2f0613
--color-destination-page-text	#be185d	#f6b7d9
--color-command-firm-action-primary	#a614ff	#c668ff
--color-command-firm-action-background	#f5e6ff	#5d0095
--color-command-firm-action-text	#9100e9	#d692ff
--color-command-company-action-primary	#808135	#c2c26b
--color-command-company-action-background	#f3f3dd	#303012
--color-command-company-action-text	#666629	#d3d392
--color-utility-neutral-primary	#475569	#94a3b8
--color-utility-neutral-background	#f1f5f9	#0f172a
--color-utility-neutral-text	#475569	#cbd5e1

FILE TYPE

Format identity for the File Type Badge — **never status**. A PDF is red because it is a PDF; never use `bg.negative` for it. `text` is the extension label on its own `background`.

TOKEN	LIGHT	DARK
--color-file-generic-background	#f1f5f9	#0f172a
--color-file-generic-text	#475569	#cbd5e1
--color-file-image-background	#ffe0c4	#3c1b00
--color-file-image-text	#a04900	#ffac5c
--color-file-pdf-background	#ffc8c9	#3b0104
--color-file-pdf-text	#9c0000	#ff686e
--color-file-sheet-background	#bbe8ce	#002409
--color-file-sheet-text	#006012	#31c37b
--color-file-text-background	#e3e8f0	#1e293b
--color-file-text-text	#1e293b	#e3e8f0
--color-file-word-background	#ced3fd	#020731
--color-file-word-text	#071592	#6d7bf8

PRODUCT IDENTITY

One tint + icon colour per Scalar product (Product Tile). Identity, not status; Waterfalls does not borrow the AI family. `icon` is graphic only — titles on the tint use `text-primary`.

TOKEN	LIGHT	DARK
--color-product-documents-background	#fff1e4	#3c1b00
--color-product-documents-icon	#d26000	#ffac5c
--color-product-intelligence-background	#e9ebfe	#020731
--color-product-intelligence-icon	#0a1cc2	#6d7bf8
--color-product-valuations-background	#e0f5e9	#002409
--color-product-valuations-icon	#007e17	#31c37b
--color-product-waterfalls-background	#e9faf4	#03402a
--color-product-waterfalls-icon	#058355	#6bdcb3

DESIGN TOKENS

SCALES

SPACING — GAP AND PADDING ONLY

TOKEN	VALUE
--space-none	0px
--space-2xs	2px
--space-xs	4px
--space-s	8px
--space-m	12px
--space-l	16px
--space-xl	24px
--space-2xl	32px
--space-3xl	48px
--space-4xl	64px

RADIUS

TOKEN	VALUE
--radius-none	0px
--radius-2xs	2px
--radius-xs	4px
--radius-s	8px
--radius-m	16px
--radius-l	24px
--radius-xl	32px
--radius-full	9999px

SIZING — WIDTH AND HEIGHT ONLY

TOKEN4PX	VALUE
--size-icon-xs	12px
--size-icon-s	16px
--size-icon-m	20px
--size-icon-l	24px
--size-icon-xl	32px
--size-control-s	32px
--size-control-m	40px
--size-control-l	48px
--size-avatar-xs	24px
--size-avatar-s	32px
--size-avatar-m	40px
--size-avatar-l	56px
--size-avatar-xl	80px
--size-row-compact	26px
--size-row-header	40px
--size-divider-pinned	4px
--size-icon-well-m	40px
--size-icon-well-l	56px
--size-progress-ring-m	48px
--size-target-dense	24px
--size-target-minimum	44px
--size-target-comfortable	48px

BREAKPOINTS

TOKEN	VALUE
--breakpoint-mobile	393px
--breakpoint-tablet-small	768px
--breakpoint-tablet	1024px
--breakpoint-desktop	1440px
--breakpoint-desktop-large	1920px

ELEVATION

TOKEN	VALUE
--elevation-raised	`0 1px 3px 0 var(--color-shadow-ambient), 0 1px 2px 0 var(--color-shadow-key)`
--elevation-overlay	`0 8px 24px -4px var(--color-shadow-ambient), 0 2px 6px -1px var(--color-shadow-key)`
--elevation-modal	`0 24px 48px -12px var(--color-shadow-ambient), 0 8px 16px -4px var(--color-shadow-key)`

MOTION — DURATION

TOKEN	VALUE
--duration-instant	100ms
--duration-fast	150ms
--duration-moderate	250ms

MOTION — EASING

TOKEN	VALUE
--easing-standard	cubic-bezier(0.2, 0, 0, 1)
--easing-decelerate	cubic-bezier(0, 0, 0, 1)

Z-INDEX LADDER

TOKEN48PX	VALUE
--z-sticky	100
--z-drawer	200
--z-scrim	300
--z-modal	400
--z-overlay	500
--z-toast	600
--z-tooltip	700

DESIGN TOKENS

TYPOGRAPHY

Three modes (rule R6). Desktop is the default. Weight is orthogonal to size (R7). The floor is 12px (R10); `heading.xs` (10px) is the one sanctioned exception, for data-grid chrome only.

Values are `size/line-height` in px.

DISPLAY

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`s`	40px/44px	48px/52px	24px/32px	-1.5px
`m`	48px/52px	56px/60px	28px/36px	-1.5px
`l`	56px/60px	64px/68px	32px/40px	-1.5px

HEADING

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`xs`	12px/16px	12px/16px	12px/16px	0px
`s`	12px/16px	14px/18px	12px/16px	0px
`m`	14px/18px	16px/20px	14px/18px	0px
`l`	16px/20px	18px/24px	16px/20px	-0.2px
`xl`	18px/24px	20px/26px	18px/24px	-0.2px
`2xl`	20px/26px	24px/32px	20px/26px	-0.5px
`3xl`	24px/32px	28px/36px	24px/32px	-0.5px
`4xl`	28px/36px	32px/40px	24px/32px	-1px
`5xl`	32px/40px	40px/48px	28px/36px	-1px

TEXT

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`s`	12px/16px	14px/20px	12px/16px	0px
`m`	14px/20px	16px/24px	14px/20px	0px
`l`	16px/24px	18px/26px	16px/24px	0px
`xl`	18px/26px	20px/28px	18px/26px	0px
`2xl`	20px/28px	24px/32px	20px/28px	0px

LABEL

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`s`	12px/16px	14px/18px	12px/16px	0px
`m`	14px/18px	16px/20px	14px/18px	0px
`l`	16px/20px	18px/24px	16px/20px	0px

LINK

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`s`	12px/16px	14px/20px	12px/16px	0px
`m`	14px/20px	16px/24px	14px/20px	0px
`l`	16px/24px	18px/26px	16px/24px	0px

OVERLINE

STEP	DESKTOP (1440)	DESKTOP LARGE (1920)	MOBILE (393)	TRACKING
`s`	12px/16px	12px/16px	12px/16px	0.8px
`m`	14px/18px	14px/18px	14px/18px	0.8px

DESIGN TOKENS

PRIMITIVE COLOUR — REFERENCE ONLY

> **Rule R1: never bind product UI to a primitive.** They have no mode and will not respond to the theme. They are published here so the semantic aliases can be traced and audited.

NEUTRAL

TOKEN	VALUE
--primitive-neutral-white	#ffffff
--primitive-neutral-black	#000000
--primitive-neutral-50	#f8fafc
--primitive-neutral-100	#f1f5f9
--primitive-neutral-200	#e3e8f0
--primitive-neutral-300	#cbd5e1
--primitive-neutral-400	#94a3b8
--primitive-neutral-500	#64748b
--primitive-neutral-600	#475569
--primitive-neutral-700	#334155
--primitive-neutral-800	#1e293b
--primitive-neutral-900	#0f172a
--primitive-neutral-alpha-white-30	#ffffff4d
--primitive-neutral-alpha-white-50	#ffffff80
--primitive-neutral-alpha-white-70	#ffffffb2
--primitive-neutral-alpha-black-08	#00000014
--primitive-neutral-alpha-black-16	#00000029
--primitive-neutral-alpha-black-30	#0000004d
--primitive-neutral-alpha-black-40	#00000066
--primitive-neutral-alpha-black-50	#00000080

TOKEN	VALUE
--primitive-neutral-alpha-black-60	#00000099
--primitive-neutral-alpha-black-70	#000000b2

BRAND

TOKEN	VALUE
--primitive-brand-50	#e9f3fd
--primitive-brand-100	#cde5fa
--primitive-brand-200	#9acbf6
--primitive-brand-300	#68b1f1
--primitive-brand-400	#3597ed
--primitive-brand-500	#037de8
--primitive-brand-600	#0268c1
--primitive-brand-700	#02539a
--primitive-brand-800	#013e73
--primitive-brand-900	#01294c

BLUE

TOKEN	VALUE
--primitive-blue-50	#e9ebfe
--primitive-blue-100	#ced3fd
--primitive-blue-200	#9ea7fa
--primitive-blue-300	#6d7bf8
--primitive-blue-400	#3d4ff5
--primitive-blue-500	#2a3ef4
--primitive-blue-600	#0a1cc2
--primitive-blue-700	#071592
--primitive-blue-800	#050e61
--primitive-blue-900	#020731

GREEN

TOKENTOKEN	VALUEVALUE
--primitive-green-50	#e0f5e9
--primitive-green-100	#bbe8ce
--primitive-green-200	#7ed6a5

TOKENTOKEN	VALUEVALUE
--primitive-green-300	#31c37b
--primitive-green-400	#00b04f
--primitive-green-500	#009c1c
--primitive-green-600	#007e17
--primitive-green-700	#006012
--primitive-green-800	#00420f
--primitive-green-900	#002409

RED

TOKEN	VALUE
--primitive-red-50	#ffe6e7
--primitive-red-100	#ffc8c9
--primitive-red-200	#ff999c
--primitive-red-300	#ff686e
--primitive-red-400	#ff2f3d
--primitive-red-500	#fb0000
--primitive-red-600	#cb0000
--primitive-red-700	#9c0000
--primitive-red-800	#6c0003
--primitive-red-900	#3b0104

YELLOW

TOKEN	VALUE
--primitive-yellow-50	#fff7e8
--primitive-yellow-100	#ffeecc
--primitive-yellow-200	#ffdd99
--primitive-yellow-300	#ffcc66
--primitive-yellow-400	#ffbb33
--primitive-yellow-500	#ffaa00
--primitive-yellow-600	#cc8800
--primitive-yellow-700	#996600
--primitive-yellow-800	#664400
--primitive-yellow-900	#332200

ORANGE

TOKEN	VALUE
--primitive-orange-50	#fff1e4
--primitive-orange-100	#ffe0c4
--primitive-orange-200	#ffc790
--primitive-orange-300	#ffac5c
--primitive-orange-400	#ff9211
--primitive-orange-500	#ff7700
--primitive-orange-600	#d26000
--primitive-orange-700	#a04900
--primitive-orange-800	#6f3300
--primitive-orange-900	#3c1b00

TEAL

TOKEN	VALUE
--primitive-teal-50	#e9faf4
--primitive-teal-100	#cef3e6
--primitive-teal-200	#9ce8cd
--primitive-teal-300	#6bdcb3
--primitive-teal-400	#39d19a
--primitive-teal-500	#08c581
--primitive-teal-600	#07a46b
--primitive-teal-700	#058355
--primitive-teal-800	#046140
--primitive-teal-900	#03402a

CYAN

TOKEN	VALUE
--primitive-cyan-50	#e9fdff
--primitive-cyan-100	#cffafe
--primitive-cyan-200	#a5f3fc
--primitive-cyan-300	#67e8f9
--primitive-cyan-400	#22d3ee
--primitive-cyan-500	#06b6d4
--primitive-cyan-600	#0891b2
--primitive-cyan-700	#0e7490

TOKEN	VALUE
--primitive-cyan-800	#155e75
--primitive-cyan-900	#164e63

PURPLE

TOKEN	VALUE
--primitive-purple-50	#f1e9ff
--primitive-purple-100	#dfcfff
--primitive-purple-200	#c2a8ff
--primitive-purple-300	#a77fff
--primitive-purple-400	#8a58ff
--primitive-purple-500	#6e2fff
--primitive-purple-600	#5826d6
--primitive-purple-700	#441da3
--primitive-purple-800	#2e1571
--primitive-purple-900	#1a0b3e

PINK

TOKEN	VALUE
--primitive-pink-50	#fef4fa
--primitive-pink-100	#fce7f3
--primitive-pink-200	#f6b7d9
--primitive-pink-300	#f088bd
--primitive-pink-400	#ea589e
--primitive-pink-500	#e4287c
--primitive-pink-600	#be185d
--primitive-pink-700	#8e1242
--primitive-pink-800	#5f0c29
--primitive-pink-900	#2f0613

OLIVE

TOKEN	VALUE
--primitive-olive-50	#fafaf0
--primitive-olive-100	#f3f3dd
--primitive-olive-200	#e4e4b9
--primitive-olive-300	#d3d392

TOKEN	VALUE
--primitive-olive-400	#c2c26b
--primitive-olive-500	#aaaa46
--primitive-olive-600	#808135
--primitive-olive-700	#666629
--primitive-olive-800	#49491d
--primitive-olive-900	#303012

AI

TOKEN	VALUE
--primitive-ai-50	#fbf4ff
--primitive-ai-100	#f5e6ff
--primitive-ai-200	#e6bcff
--primitive-ai-300	#d692ff
--primitive-ai-400	#c668ff
--primitive-ai-500	#b63eff
--primitive-ai-600	#a614ff
--primitive-ai-700	#9100e9
--primitive-ai-800	#7700bf
--primitive-ai-900	#5d0095

IV

COMPONENTS

Every component in the Figma components file (`Z4MtKOfkNEzhMYJzN1q3kR`, 154 component sets and 1,260 variants across 23 pages; last checked against the live file on 2026-10-05) mapped to its React export.

READING THE REFERENCE

How the Figma variant matrix becomes props. Figma models every combination as a discrete variant — `Button` alone is 225. React collapses that: each Figma variant *property* becomes a prop, and the interaction states (Hover, Pressed, Focus) become CSS pseudo-classes rather than props, because the browser already owns them. So `Button`'s Style × Size × State × Type matrix becomes `variant` × `size` × `tone` plus `:hover` / `:active` / `:focus-visible`.

Legend: = direct mapping · ⊕ several Figma sets merged into one export · + added by the code layer, with no Figma counterpart.

02 CORE

COMPONENTS · 5 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Divider	Orientation	= `Divider`	`core/Divider.tsx`
Link	Size × State	= `Link`	`core/Link.tsx`
Tooltip	Position	= `Tooltip` (+ `open` / `onOpenChange` for a controlled or forced-open tooltip)	`core/Tooltip.tsx`
Scrim	—	= `Scrim`	`core/Scrim.tsx`
Empty State	Type	= `EmptyState`	`core/EmptyState.tsx`

02 · CORE

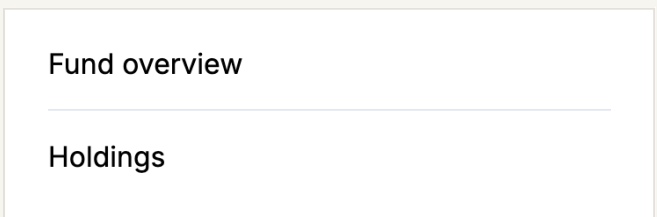
Divider

REACT `Divider` FILE `src/components/core/Divider.tsx` FIGMA `Divider` VARIANTS `Orientation`

Divider — a rule between sections or inline items.

Token: Stroke/Divider at 1px. Deliberately low contrast (~1.4:1) because it is a non-interactive container edge, so WCAG 1.4.11 does not apply (rule R5).

Use a divider only where whitespace alone fails to group. Two dividers stacked with nothing between them is a spacing problem, not a divider one.



Storybook · Core/Primitives › Divider Horizontal

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`orientation`	`"horizontal"` | `"vertical"`	`horizontal`	—
`decorative`	`boolean`	—	Purely visual: removes the separator from the accessibility tree. Use it when the rule only adds spacing and the grouping is already conveyed by the markup.

02 · CORE

Link

REACT Link FILE src/components/core/Link.tsx FIGMA Link VARIANTS Size × State

Link — inline navigation inside running text.

The Link type role exists so link metrics track body text without inheriting a heading's tracking. The underline is permanent, not a hover reveal: colour alone never signals that something is a link (rule R8), and a link that only underlines on hover is invisible to anyone not already pointing at it.

A link navigates. If it submits, deletes or saves, it is a Button wearing the wrong clothes.

`asChild` renders your router link instead of an `<a>`: `<Link asChild><RouterLink to="/x">Valuations</RouterLink> </Link>`. `target="_blank"` defaults `rel` to `noopener noreferrer`.



Storybook · Core/Primitives › Link Sizes

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`size`	`"s"` | `"m"` | `"l"`	`m`	Link/S · Link/M · Link/L.
`asChild`	`boolean`	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

02 · CORE

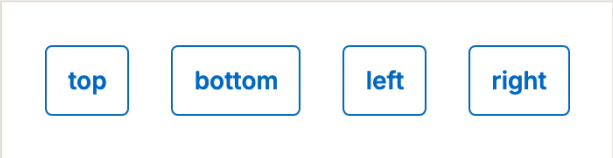
Tooltip

REACT Tooltip FILE src/components/core/Tooltip.tsx FIGMA Tooltip VARIANTS Position

Tooltip — a short explanation of the control under the pointer.

A tooltip explains; it never holds the only copy of something. Anything a user must read to complete a task belongs in the layout, not behind a hover.

Accessibility: reachable by keyboard focus, not hover alone. Escape closes it from anywhere while it is open. `aria- describedby` is set on the trigger itself (the child element), so assistive tech reads the text with the control. The `ref` points at the positioning wrapper.



Storybook · Core/Primitives › Tooltip Positions

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
childrenrequired	`ReactElement<any, string | JSXElementConstructor<any>>`	—	The trigger. Must be focusable — a tooltip is never hover-only.
contentrequired	`ReactNode`	—	The explanation. Keep it short; this is not a place to store content.
position	`"bottom" | "top" | "left" | "right"`	`top`	The side the tooltip sits on, so the arrow points back at its trigger.
open	`boolean`	—	Controlled visibility. `true` forces the tooltip open (a guided tour, a documentation frame), `false` keeps it shut; omit it for the normal hover / focus behaviour.
onOpenChange	`((open: boolean) => void)`	—	Called with the hover / focus / Escape intent, controlled or not.
className	`string`	—	Class name for the tooltip bubble. Extra root props (`ref`, `data-*`) land on the positioning wrapper.

02 · CORE

Scrim

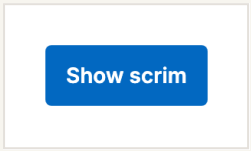
REACT Scrim FILE src/components/core/Scrim.tsx FIGMA Scrim

Scrim — the dimming layer behind a modal.

Token: Overlay/Scrim — 50% black in Light, 60% in Dark, so the layer deepens rather than inverting.

Always present under Elevation/Modal (rule R9). Render it as a sibling of the dialog, never a child: a scrim inside the dialog cannot cover the page behind it.

Accessibility: the scrim is a pointer-only convenience and is `aria-hidden`. The dialog it sits under owns Escape and its own Close button.



Storybook · Core/Primitives › Scrim Behind Modal

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
onDismiss	`(() => void)`	—	Called when the scrim is clicked. Wire this to dismiss only when the dialog has no unsaved state. A caller's own `onClick` still runs first.

02 · CORE

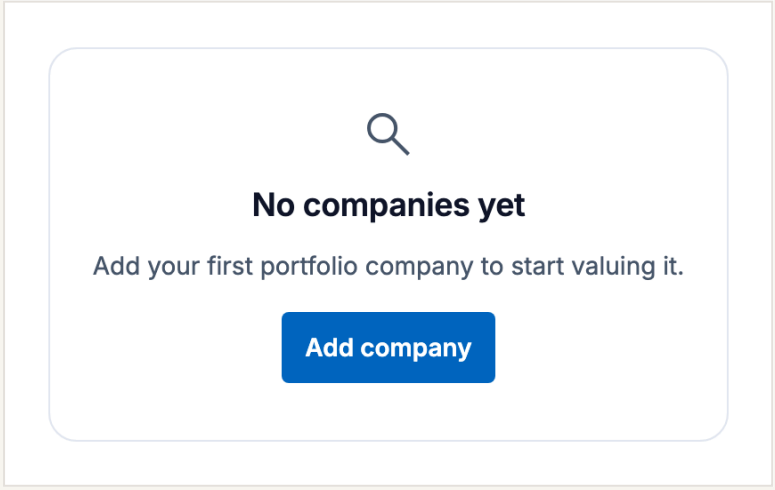
EmptyState

REACT EmptyState FILE src/components/core/EmptyState.tsx FIGMA Empty State VARIANTS Type

Empty State — what a surface shows when it has nothing to show.

An empty state is the most-read screen in a new account. Write the copy as guidance, not apology — never a bare "Something went wrong".

It has no live-region role by default: a static empty state must not shout at page load, and it holds buttons, which an alert should not. When an error appears after a user action, pass `role="status"` (or announce it elsewhere).



Storybook · Core/Primitives › Empty No Data

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
type	"no-data" | "no-results" | "error"	no-data	—
headingLevel	1 | 2 | 3 | 4 | 5 | 6	3	The heading level, so the title fits the page outline. Default 3.
body	ReactNode	—	—
icon	ReactNode	—	An icon from SDS_Main icons, wrapped in `Icon`.
actions	ReactNode	—	One or two Button instances carrying the way forward.

03

COMPONENTS · 1 COMPONENT

AVATAR

FIGMA	VARIANTS	REACT	FILE
Avatar	Size × With Image	= `Avatar`	`avatar/Avatar.tsx`

03 · AVATAR

Avatar

REACT Avatar FILE src/components/avatar/Avatar.tsx FIGMA Avatar VARIANTS Size × With Image

Avatar — a person or company identity mark.

Initials scale with the ramp rather than being hand-set: Heading/S at XS and S, Text/M at M, Text/2XL at L, Heading/3XL at XL.

Storybook · Core/Avatar › Initials

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`size`	`"xs"` | `"s"` | `"m"` | `"l"` | `"xl"`	`s`	Bound to Semantic: Sizing/Avatar. Never hand-size an avatar — pick a step.
`src`	`string`	—	Image URL. Without one the avatar falls back to initials.
`alt`	`string`	—	Describes the person or company, not the picture. Falls back to `initials` for the accessible name. With neither, the avatar is decorative and hidden from assistive technology (use that only beside the person's name).
`initials`	`string`	—	Usually two characters. Rendered uppercase. Shown when there is no `src` or the image fails to load.

04

COMPONENTS · 3 COMPONENTS

BUTTON

FIGMA	VARIANTS	REACT	FILE
Button	Style × Size × State × Type (225)	= `Button`	`button/Button.tsx`
Button_Icon	same matrix (225)	= `ButtonIcon`	`button/ButtonIcon.tsx`
AI-Button	—	= `AIButton`	`button/AIButton.tsx`

04 · BUTTON

Button

`REACT` Button `FILE` src/components/button/Button.tsx `FIGMA` Button
`VARIANTS` Style × Size × State × Type (225)

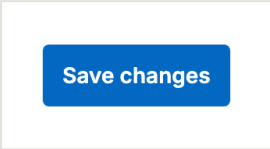
Button — the primary action control.

The Figma set is 225 variants: Style × Size × State × Type. In code, Style is `variant`, Type is `tone`, Size is `size`, and the interaction states are CSS pseudo-classes rather than props — which is why there is no `state`.

Disabled is modelled as a Type in Figma, not a State, because a disabled control has no hover, pressed, selected or focus. Here it is the native `disabled` attribute, which has the same effect.

`asChild` renders the single child element (an `<a>`, a router Link) with the button classes, props and ref merged on. `leadingIcon`, `trailingIcon` and the loading spinner are inserted inside that child.

Accessibility: `tone="warning"` pairs Background/Warning with Text/On Warning (near-black) — white can never clear AA on yellow (rule R4). The drawn control is 24/32/40/48px (xs/s/m/l); only `xs` falls below the 32px dense floor, and all sizes meet WCAG 2.5.8 (24px minimum).



Storybook · Core/Button › Primary

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
variant	`"primary"` | `"secondary"` | `"tertiary"`	`primary`	—
tone	`"main"` | `"positive"` | `"warning"` | `"negative"`	`main`	—
size	`"xs"` | `"s"` | `"m"` | `"l"`	`m`	—
loading	`boolean`	`false`	Swaps the leading icon for a spinner and blocks interaction. The button stays focusable (`aria-disabled`, not `disabled`), is marked `aria-busy` and announces "Loading" through a status region.
selected	`boolean`	—	Persistent toggle state. Renders as `aria-pressed`.
leadingIcon	`ReactNode`	—	—
trailingIcon	`ReactNode`	—	—
forcedState	`"hover"` | `"pressed"`	—	Forces a visual state. For documentation and visual tests only.
asChild	`boolean`	`false`	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

04 · BUTTON

ButtonIcon

REACT ButtonIcon FILE src/components/button/ButtonIcon.tsx FIGMA Button_Icon
VARIANTS same matrix (225)

Button_Icon — Button with an icon and no label.

The same token contract as Button (including `asChild`); only the label is gone. Use it only where the icon is unambiguous on its own — close, more, expand. Everything else takes a visible label.

Accessibility: an accessible name is required by the type (`label`, `aria-label` or `aria-labelledby`); `label` is the primary form and becomes `aria-label`. The drawn control is the target: 24px (`xs`), 32px (`s`), 40px (`m`), 48px (`l`). WCAG 2.5.8 needs 24px, so every size passes; the 44px target token is not applied here — use `m` or `l` for touch-first surfaces. Pair with `Tooltip` where the icon alone may be unclear.



Storybook · Core/ButtonIcon › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
iconrequired	ReactNode	—	The glyph. Wrap it in `Icon` so it is tinted and sized from the ramp.
label	string	—	—
aria-labelledby	string	—	—
aria-label	string	—	—
variant	"primary" | "secondary" | "tertiary"	—	—
tone	"main" | "positive" | "warning" | "negative"	—	—
selected	boolean	—	Persistent toggle state. Renders as `aria-pressed`.
size	"xs" | "s" | "m" | "l"	—	—
loading	boolean	—	Swaps the leading icon for a spinner and blocks interaction. The button stays focusable (`aria-disabled`, not `disabled`), is marked `aria-busy` and announces "Loading" through a status region.
forcedState	"hover" | "pressed"	—	Forces a visual state. For documentation and visual tests only.

PROP	TYPE	DEFAULT	DESCRIPTION
asChild	boolean	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

04 · BUTTON

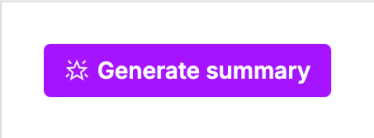
AIButton

REACT AIButton FILE src/components/button/AIButton.tsx FIGMA AI-Button

AI-Button — the entry point to an AI action.

Reserved for genuinely generative actions. It is not a decorative treatment for ordinary buttons: the purple always means "a model produced this". Supports `loading`, `trailingIcon` and `asChild` like Button; it has no variant/size axes by design (one AI treatment).

Tokens: Background/AI with Text/On AI and Icon/On AI. The gradient stroke in Figma is the one intentional exception in the set and has no Dark-mode counterpart — verify on Background/Page in Dark before shipping.



Storybook · Core/AIButton › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
showIcon	boolean	true	Shows the AI mark. On by default — it is what makes the purple legible.
loading	boolean	false	Blocks interaction and announces "Loading"; the button stays focusable.
trailingIcon	ReactNode	—	Extra glyph after the label.
asChild	boolean	false	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

05 CHIP

COMPONENTS · 1 COMPONENT

FIGMA	VARIANTS	REACT	FILE
Chip	Style × Size	= `Chip`	`chip/Chip.tsx`

05 · CHIP

Chip

REACT Chip FILE src/components/chip/Chip.tsx FIGMA Chip VARIANTS Style × Size

Chip — a compact, removable label that states a fact about the item it sits on.

Static by default (a `span`). Pass `onClick` or `selected` and the label becomes a toggle button (Enter/Space, `aria-pressed`); pass `onRemove` and a separate remove button (24px target) is added. `asChild` renders the single child (e.g. a link) as the chip; the remove control is not rendered then.

For the scope stack in Global Search use `SearchScopeChip` instead — that one carries the PRISM type colours, which this component deliberately does not.

Storybook · Core/Chip › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`styleVariant`	`"positive" | "default" | "warning" | "negative" | "info" | "ai"`	`info`	`ai` marks a count or label a model produced. Like every style it is a tint.
`size`	`"s" | "m" | "l"`	`s`	—
`leadingIcon`	`ReactNode`	—	A glyph or Avatar shown before the label (decorative — wrap glyphs in `Icon`).
`onClick`	`((event: MouseEvent<HTMLButtonElement, MouseEvent>) => void)`	—	Makes the label a toggle button (`aria-pressed`) so the chip can act as a filter. `onClick` is typed on the label button, not the root span, which is why the root's own `onClick` is omitted: an unfocusable span with a click handler is not keyboard reachable.
`selected`	`boolean`	—	Toggle state; passing it (or `onClick`) makes the label a `<button aria-pressed>`.
`onRemove`	`(() => void)`	—	Fires when the remove control is pressed, or Backspace/Delete is pressed on the focused label button. Omit it to hide the control.
`removeLabel`	`string`	—	Accessible name for the remove control. Without it the name is "Remove" plus the chip's own text (also for non-string children), so it is always unique.
`asChild`	`boolean`	`false`	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

06

COMPONENTS · 2 COMPONENTS

INPUT

FIGMA	VARIANTS	REACT	FILE
input	State	= `Input`	`input/Input.tsx`
Form Field	State	= `FormField`	`input/FormField.tsx`

06 · INPUT

Input

REACT Input FILE src/components/input/Input.tsx FIGMA input VARIANTS State

Input — the bare text field.

The field alone, with no label or helper. Use `FormField` for anything a person has to fill in: a naked input with no label is an accessibility failure in almost every context. Inside a `FormField` it takes its `id`, `required`, error state and `aria-describedby` (hint / error) automatically; standalone, give it an `aria-label` / `aria-labelledby` and pass your own `aria-describedby`.

`ref`, `id`, `name`, `value` / `defaultValue` and every other native prop land on the `<input>` (controlled or uncontrolled); `className` and `data-state` land on the wrapper. `error` state sets `aria-invalid`.



Storybook · Forms/Input › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`state`	`"disabled"` | `"default"` | `"error"`	—	Hover and focus are CSS states; only error and disabled are props.
`leadingIcon`	`ReactNode`	—	A glyph shown before the text. Wrap it in `Icon`.
`trailingIcon`	`ReactNode`	—	—
`className`	`string`	—	Lands on the wrapper `div` (the drawn field). Use `controlClassName` for the `<input>`.
`controlClassName`	`string`	—	Lands on the `<input>` itself.

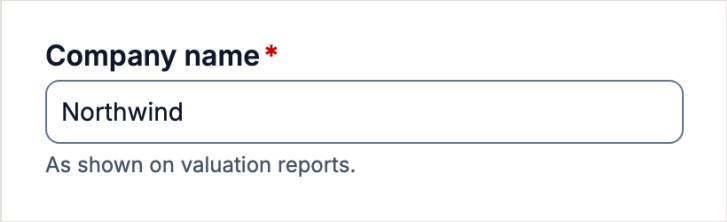
06 · INPUT

FormField

REACT FormField FILE src/components/input/FormField.tsx FIGMA Form Field VARIANTS State

Form Field — label, control, helper and error as one unit.

This is the default way to ask for input. It wires `id`, `required`, `aria-invalid` and `aria-describedby` (hint, or error when `state="error"`) through to the control — via context for the package's own controls and via props for custom ones — and merges rather than replaces a describedby the control already has. Extra native props and `ref` land on the wrapper `div`.



Storybook · Forms/Input › Form Field Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
childrenrequired	`ReactElement<{ id?: string | undefined; state?: FieldState | undefined; 'aria-describedby'?: string | undefined; }, string | JSXElementCo…`	—	The control: an Input, Select or Textarea (they read the field from context), or any single element that accepts `id`, `state` and `aria-describedby`.
label	`ReactNode`	—	The label. Permanent — never a placeholder standing in for one, which disappears exactly when the user needs it.
helperText	`ReactNode`	—	Helper text. In `error` state this becomes the error message and takes Text/Negative. Say what is wrong and how to fix it, never just "Invalid".
state	`"disabled"` | `"default"` | `"error"`	`default`	—
required	`boolean`	`false`	Marks the control `required` (native) and draws a decorative asterisk.
className	`string`	—	—

07

COMPONENTS · 2 COMPONENTS

CHECKBOX

FIGMA	VARIANTS	REACT	FILE
Checkbox Item	Status × Size	= `CheckboxItem`	`checkbox/CheckboxItem.tsx`
CheckBox	—	= `Checkbox`	`checkbox/Checkbox.tsx`

07 · CHECKBOX

CheckboxItem

REACT CheckboxItem FILE src/components/checkbox/CheckboxItem.tsx FIGMA Checkbox Item
VARIANTS Status × Size

Checkbox Item — the checkbox control on its own.

Controlled (`checked` + `onChange`) or uncontrolled (`defaultChecked`), as a native checkbox. It has no label of its own: give it `aria-label` or `aria-labelledby`, or use `Checkbox`, whose wrapping label also carries the 44px target the drawn 16–24px box cannot reach.



Storybook · Forms/Checkbox › Bare Item

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`size`	`"s"` | `"m"`	`m`	M is 24px, S is 16px. The drawn box, not the target.
`indeterminate`	`boolean`	`false`	For a parent whose children are partly selected. A display state — never something the user selects directly. Exposed natively as "mixed".
`invalid`	`boolean`	—	Draws the error edge and sets `aria-invalid`. Point `aria-describedby` at the error text.
`className`	`string`	—	Lands on the drawn box `span`, not the `<input>` (which takes `ref` and every other native prop).

07 · CHECKBOX

Checkbox

REACT Checkbox FILE src/components/checkbox/Checkbox.tsx FIGMA CheckBox

CheckBox — checkbox with its label.

The label is part of the target: clicking it toggles the box. The wrapper also carries the 44px minimum target the drawn box cannot reach on its own. `description` is read as the checkbox's description. Controlled or uncontrolled (native). For an invalid state pass `invalid` and an `aria-describedby` pointing at the error text.



Storybook · Forms/Checkbox › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`description`	`ReactNode`	—	Secondary text under the label; wired to the input as its description.
`className`	`string`	—	Lands on the drawn box `span`, not the `<input>` (which takes `ref` and every other native prop). Lands on the wrapping `<label>`; `ref` and every other native prop land on the `<input>`.
`size`	`"s" | "m"`	—	M is 24px, S is 16px. The drawn box, not the target.
`indeterminate`	`boolean`	—	For a parent whose children are partly selected. A display state — never something the user selects directly. Exposed natively as "mixed".
`invalid`	`boolean`	—	Draws the error edge and sets `aria-invalid`. Point `aria-describedby` at the error text.

08

COMPONENTS · 6 COMPONENTS

FORM CONTROLS

FIGMA	VARIANTS	REACT	FILE
Radio	Status × Size	= `Radio`	`form-controls/Radio.tsx`
Switch	State × Size	= `Switch`	`form-controls/Switch.tsx`
Select	State	= `Select`	`input/Select.tsx`
Textarea	State	= `Textarea`	`input/Textarea.tsx`
Calendar Day	State	= `CalendarDay`	`form-controls/CalendarDay.tsx`
Date Picker	—	= `DatePicker`	`form-controls/DatePicker.tsx`

08 · FORM CONTROLS

Radio

REACT Radio FILE src/components/form-controls/Radio.tsx FIGMA Radio VARIANTS Status × Size

Radio — one choice from a set of mutually exclusive options.

Radios are never used alone: a single radio that cannot be unselected is a checkbox. Always two or more, always with one preselected unless the question genuinely has no default. Wrap them in `RadioGroup` for the group name, shared `name`, selection state and error; the keyboard model (arrows move and select, Tab enters once) is the native one for same-`name` radios.

Mirrors the Checkbox variant model minus Indeterminate, which has no meaning for an exclusive choice.



Storybook · Forms/Controls › Radio Standalone

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`className`	`string`	—	Lands on the wrapping `<label>`; `ref` and every other native prop land on the `<input>`.
`size`	`"s" | "m"`	—	—
`invalid`	`boolean`	—	Draws the error edge. `radio` has no `aria-invalid`; put the error on `RadioGroup` (`error`), which exposes it.

08 · FORM CONTROLS

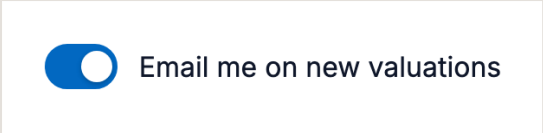
Switch

REACT Switch FILE src/components/form-controls/Switch.tsx FIGMA Switch VARIANTS State × Size

Switch — turns a setting on or off, taking effect immediately.

Use a switch only when the change applies at once. If the setting needs a Save, use a Checkbox: a switch that does not take effect until submitted lies about what it did. Label what the switch controls — never repeat its on/off state.

A native `<input type="checkbox" role="switch">`: Space toggles, and the checked state is exposed as the switch's on/off value (no `aria-checked` needed, and none that could drift). Controlled (`checked` + `onChange`) or uncontrolled (`defaultChecked`).



Storybook · Forms/Controls › Switch Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`className`	`string`	—	Lands on the wrapping `<label>`; `ref` and every other native prop land on the `<input>`.
`description`	`ReactNode`	—	Secondary text under the label; wired to the switch as its description.
`size`	`"s" | "m"`	`m`	M is 40×24, S is 28×16.
`invalid`	`boolean`	—	Draws `aria-invalid` for parity with Checkbox. Point `aria-describedby` at the error text.

08 · FORM CONTROLS

Select

REACT Select FILE src/components/input/Select.tsx FIGMA Select VARIANTS State

Select — choose one option from a list that is too long to show inline.

Built to the same spec as Input (native `<select>`: keyboard and screen reader behaviour come from the browser). Use Select above roughly seven options; below that, Radio shows every choice at once and costs one fewer interaction. Inside a `FormField` it is labelled and described automatically; standalone, pass `aria-label`. `ref` and native props land on the `<select>`; `className` on the wrapper.

Note on icons: glyphs from SDS_Main icons default to Text/On Brand (white) and are invisible on a light surface. Every glyph here is re-tinted to Text/Secondary, and Text/Disabled when the control is off. Any component that imports an icon must do the same.



Storybook · Forms/Input › Select Field

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`state`	`"disabled"` | `"default"` | `"error"`	—	—
`controlClassName`	`string`	—	Lands on the `<select>` itself; `className` lands on the wrapper.

---

08 · FORM CONTROLS

Textarea

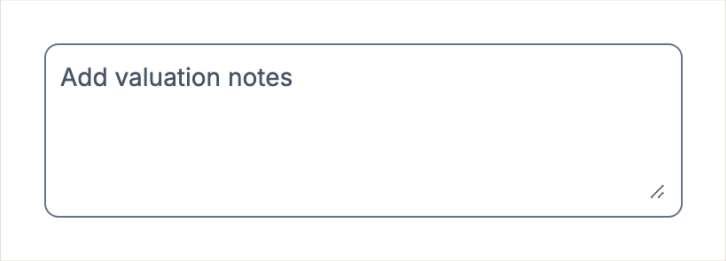
REACT Textarea FILE src/components/input/Textarea.tsx FIGMA Textarea VARIANTS State

Textarea — free text longer than a single line.

Tokens and geometry match Input, with the text pinned to the top edge.

Size the default height to the expected answer. A three-line box invites three lines; a box taller than the answer needs reads as a demand for more.

Pair with `FormField` for a label, helper/error text and `required`; inside one it is wired automatically. Standalone, pass `aria- label` and your own `aria-describedby` (e.g. a character counter). `ref` and native props land on the `<textarea>`; `className` on the wrapper.



Storybook · Forms/Input › Textarea Field

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`state`	`"disabled"` | `"default"` | `"error"`	—	—
`resizable`	`boolean`	`true`	Whether the box can be dragged taller. If a field is not resizable, turn this off rather than leaving the grip as decoration.
`controlClassName`	`string`	—	Lands on the `<textarea>` itself; `className` lands on the wrapper.

08 · FORM CONTROLS

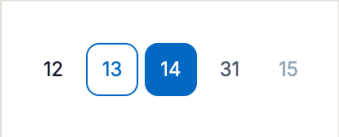
CalendarDay

REACT CalendarDay FILE src/components/form-controls/CalendarDay.tsx FIGMA Calendar Day VARIANTS State

Calendar Day — a single day cell inside Date Picker.

Today and Selected are never drawn the same way: a user who opens the picker on a date they already chose has to be able to see both facts. Selected fills Background/Brand; Today is an outline in Stroke/Brand with no fill.

Accessibility: the accessible name is the full date (from `date`), with "today" appended; selection is `aria-selected` on the gridcell when `cell` is set, otherwise `aria-pressed` on the button. Roving tab stops are managed by `DatePicker` through `tabIndex`.



Storybook · Forms/Controls › Calendar Days

PROPS

PROP	TYPESTORYBOOK · FORMS/INPUT › TEXTAREA FIELD	DEFAULTPROP	DESCRIPTIONPROP
dayrequired	`string | number`	—	The visible day number.
date	`Date`	—	The full date this cell stands for. Gives the button an accessible name such as "Monday, March 5, 2026" (plus "today") instead of a bare "5", and a `data- date` hook. Without it, supply your own `aria- label`.
locale	`string`	`en-US`	Locale for the spoken date. Default `en-US`.
todayLabel	`string`	`today`	Spoken suffix for today's cell. Default "today" — override to localise.
selected	`boolean`	—	The currently chosen date.
today	`boolean`	—	Today's date. Drawn as an outline, never as a fill.
outside	`boolean`	—	A day belonging to the neighbouring month.
cell	`boolean`	`false`	Wrap the button in a `role="gridcell"` that carries `aria-selected` (and `aria-disabled`). Set by `DatePicker`; leave off when using the day on its own, where the button exposes `aria-pressed` instead.
children	`ReactNode`	—	Extra content after the number, e.g. an event marker. Decorative unless labelled.

DatePicker

REACT DatePicker FILE src/components/form-controls/DatePicker.tsx FIGMA Date Picker

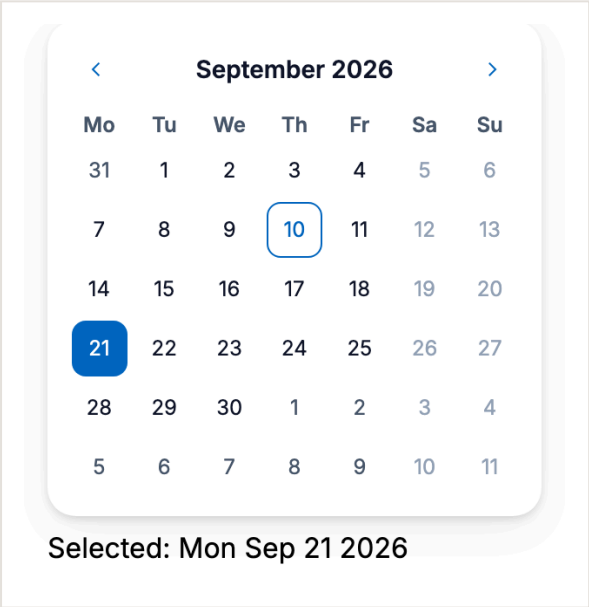
Date Picker — a month grid for choosing a date, shown in an overlay.

Six week rows always, so the popover never changes height between months and the controls under it never move.

Always pair with a typed date field. A picker is faster for dates near today and slower for everything else — a valuation dated three years back should be typed, not clicked to.

Accessibility (WAI-ARIA date-picker grid): `role="grid"` > `row` > `gridcell`

> `button`, named by the month heading, which is a polite live region so month changes are announced. Each day's name is its full date. The grid is a single tab stop (roving tabindex); inside it: Left/Right move a day, Up/Down a week, Home/End to the start/end of the week, Page Up/Down a month, Shift+Page Up/Down a year, Enter/Space choose. Escape calls `onEscape`. Controlled (`value`) or uncontrolled (`defaultValue`).



Storybook · Forms/Controls › Date Picker Default

PROPS

PROP	TYPEPROP	DEFAULTTYPE	DESCRIPTIONDEFAULT
value	Date	—	The chosen date (controlled).
defaultValue	Date	—	The initially chosen date (uncontrolled).
onChange	((date: Date) => void)	—	—
month	Date	—	The month on view. Uncontrolled if omitted.
onMonthChange	((month: Date) => void)	—	—

PROP	TYPEPROP	DEFAULTTYPE	DESCRIPTIONDEFAULT
isDisabled	((date: Date) => boolean)	—	Returns true for a date that cannot be chosen. Disabled days stay focusable (aria-disabled) but cannot be picked.
weekStartsOn	0 | 1	1	0 = Sunday. Defaults to Monday.
locale	string	en-US	—
today	Date	—	"Today" for the highlight and the initial month. Pass it from the server or a test to avoid a hydration mismatch around midnight; defaults to new Date().
onEscape	(() => void)	—	Called when Escape is pressed inside the picker. The host owns open state: close the overlay and return focus to the field here.
autoFocus	boolean	false	Move focus to the selected (or today's) day on mount. Off by default so a picker rendered inline or in a gallery never steals focus; turn it on when the picker opens in an overlay.
className	string	—	—

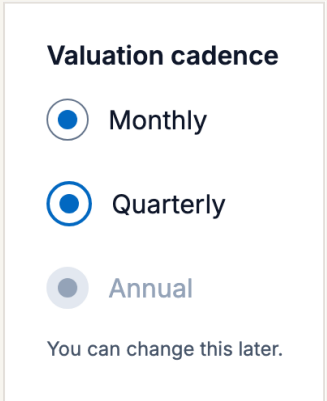
08 · FORM CONTROLS

RadioGroup

REACT RadioGroup FILE src/components/form-controls/Radio.tsx

RadioGroup — a labelled set of mutually exclusive Radios.

Renders role="radiogroup" named by label, with hint / error as its description, a shared name, and controlled (value) or uncontrolled (defaultValue) selection. Arrow keys move and select between radios and Tab enters/leaves the group — that is native radio behaviour, since every radio shares one name.



Storybook · Forms/Controls › Radio Group Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`name`	`string`	—	Shared `name` of the radios. Generated when omitted.
`value`	`string`	—	Selected value (controlled).
`defaultValue`	`string`	—	Initially selected value (uncontrolled).
`onValueChange`	`((value: string) => void)`	—	—
`label`	`ReactNode`	—	Visible group label (the question). Without it, pass `aria-label` / `aria-labelledby`.
`hint`	`ReactNode`	—	Helper text under the group; read as its description.
`error`	`ReactNode`	—	Error text; sets `aria-invalid` on the group and draws each radio's error edge.
`required`	`boolean`	—	—
`disabled`	`boolean`	—	—
`size`	`"s" | "m"`	—	—

09

COMPONENTS · 25 COMPONENTS

HEADER & MENUS

FIGMA	VARIANTSX	REACTMAIN-MENU-HORIZONTAL-ITEM	FILESTATE
Primary Menu	—	= `PrimaryMenu`	`header/Chrome.ts x`
Main-Menu-horizontal-item	State	= `MainMenuItem`	`header/Chrome.ts x`
Secondary Menu	State	= `SecondaryMenu` + `SecondaryMenuItem`	`header/Chrome.ts x`
Tertiary Menu	—	= `TertiaryMenu`	`header/Chrome.ts x`
Tertiary Menu Item	State	= `TertiaryMenuItem`	`header/Chrome.ts x`
Company info	—	= `CompanyInfo`	`header/Chrome.ts x`
Submenu Item	State	= `SubmenuItem`	`header/Menus.tsx`
Company Dropdown	—	= `CompanyDropdown`	`header/Controls. tsx`

FIGMA	VARIANTSX	REACTMAIN-MENU-HORIZONTAL-ITEM	FILESTATE
Company Dropdown/Default	—	⊕ `CompanyDropdownPanel`	`header/Menus.tsx`
Company Dropdown/Pinned Company	—	⊕ `CompanyDropdownPanel` (`pinned` entries)	`header/Menus.tsx`
Captable sub-menu	State	⊕ `SectionSubMenu`	`header/Menus.tsx`
Valuations sub-menu	State	⊕ `SectionSubMenu`	`header/Menus.tsx`
Filter dropdown	—	= `FilterDropdown`	`header/Controls. tsx`
Search bar	—	= `SearchBar`	`header/Controls. tsx`
Notification	—	= `Notification`	`header/Controls. tsx`
Badge	—	= `Badge`	`header/Controls. tsx`
Combo tag	—	= `ComboTag`	`header/Controls. tsx`
Currency Selector	—	= `CurrencySelector`	`header/Controls. tsx`
Selector	`Drop-Down`, `Picker label`	= `Selector` (`dropdown`; omit `label` to hide it)	`header/Controls. tsx`
Information Label	`Has drop down`, `Long Value`, `L1–L3`	= `InformationLabel` (`dropdown`, `longValue`; omit `label` / `value` to hide)	`header/Controls. tsx`
Valuation Info	Open (None · Values · Dates · Both)	= `ValuationInfo` (`open` / `defaultOpen` / `onOpenChange`)	`header/Valuation Info.tsx`
Valuation	State	⊕ `ToolSwitch`	`header/Controls. tsx`
Workboard	State	⊕ `ToolSwitch`	`header/Controls. tsx`
Tool-switch	State	⊕ `ToolSwitch`	`header/Controls. tsx`
AI tool	State	= `AITool`	`header/Controls. tsx`

Merges. `Company Dropdown/Default` and `/Pinned Company` differ only by whether a pinned group is present, so one component takes `companies` with a `pinned` flag. `Captable sub-menu` and `Valuations sub-menu` share one token contract by design "so the two sections navigate identically" — one component takes the section's items. `Valuation`, `Workboard` and `Tool-switch` are one control in three pieces; exactly one side is always selected, so a single `value` / `onChange` pair models it correctly.

09 · HEADER & MENUS

PrimaryMenu

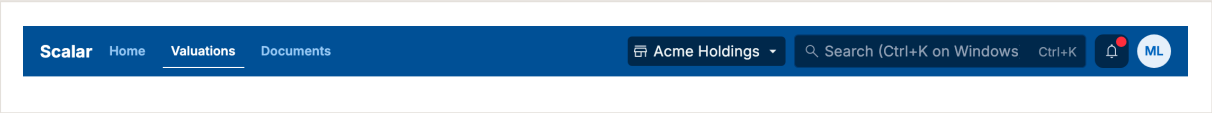
REACT PrimaryMenu FILE src/components/header/Chrome.tsx FIGMA Primary Menu

Primary Menu — the top-level application bar.

Composed from MainMenuItem, CompanyDropdown, SearchBar, Notification and Avatar — change those, not this.

It spans the viewport, so it is one of the two components that legitimately use a full-bleed width. Set the viewport type mode on `ScalarProvider` to move it between breakpoints rather than hand-resizing type.

Accessibility: a banner landmark containing a labelled `<nav>`; the current item carries `aria-current="page"`.



Storybook · Header/Chrome › Primary

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`logo`	`ReactNode`	—	The Scalar mark.
`children`	`ReactNode`	—	MainMenuItem instances.
`end`	`ReactNode`	—	Company switcher, search, notifications, avatar.
`navLabel`	`string`	`Primary`	Accessible name of the primary `<nav>` landmark. Default "Primary".
`as`	`"header" | "div"`	`header`	Root element. `header` (default) is the page banner landmark; pass `div` when the page already has a banner (e.g. a PageTaskHeader) to avoid two.

09 · HEADER & MENUS

MainMenuItem

REACT MainMenuItem FILE src/components/header/Chrome.tsx FIGMA Main-Menu-horizontal-item
VARIANTS State

Main-Menu-horizontal-item — one item in the primary navigation.

PROPS

PROP	TYPEAITOOL	DEFAULT	DESCRIPTIONHEADER/CONTROLS.
icon	ReactNode	—	—
current	boolean	—	The section the user is in (`aria-current="page"`).
expanded	boolean	—	Set only for an item that opens a dropdown: it is then a disclosure button and carries `aria-expanded`. Leave undefined for a plain link. Pass `aria-controls` to name the panel.
onClick	MouseEventHandler<HTMLElement>	—	—
asChild	boolean	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

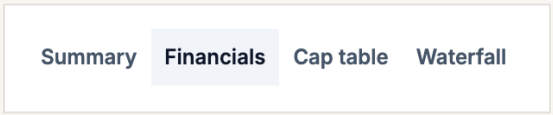
09 · HEADER & MENUS

SecondaryMenu

REACT SecondaryMenu FILE src/components/header/Chrome.tsx FIGMA Secondary Menu VARIANTS State

Secondary Menu — the second navigation tier, scoped to the selected company.

Sits directly under Primary Menu and changes when the company changes. A labelled `<nav>` landmark.



Storybook · Header/Chrome › Secondary

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
label	string	Tertiary	Accessible name of the `<nav>` landmark. Default "Secondary".

09 · HEADER & MENUS

SecondaryMenuItem

REACT SecondaryMenuItem FILE src/components/header/Chrome.tsx FIGMA Secondary Menu VARIANTS State

One item in the second navigation tier.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
current	boolean	—	The page the user is on (`aria-current="page"`).
onClick	MouseEventHandler<HTMLElement>	—	—
asChild	boolean	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

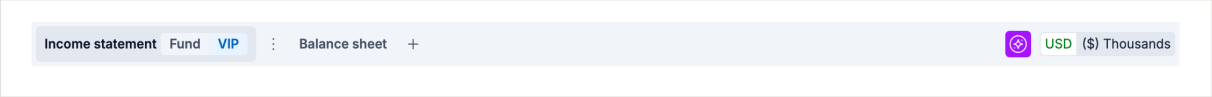
09 · HEADER & MENUS

TertiaryMenu

REACT TertiaryMenu FILE src/components/header/Chrome.tsx FIGMA Tertiary Menu

Tertiary Menu — the third navigation tier, scoped to the selected section.

Three tiers is the limit. A fourth level belongs in the page body, not the chrome. A labelled `<nav>` landmark.



Storybook · Header/Chrome › Tertiary

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
children	ReactNode	—	TertiaryMenuItem instances.
onAdd	(() => void)	—	Shows the + that adds a view; called when it is pressed.
end	ReactNode	—	The page toolbar on the right: AI tool, currency, table tools, primary action.
label	string	Tertiary	Accessible name of the `<nav>` landmark. Default "Tertiary".
addLabel	string	Add view	Accessible name of the + button. Default "Add view".

09 · HEADER & MENUS

TertiaryMenuItem

REACT TertiaryMenuItem FILE src/components/header/Chrome.tsx FIGMA Tertiary Menu Item VARIANTS State

Tertiary Menu Item — one item in the third navigation tier.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
icon	ReactNode	—	—
current	boolean	—	The view the user is on (`aria-current="page"`).
tags	ReactNode	—	Combo tags shown beside the label.
menu	boolean	—	Show the kebab for the view's actions. Defaults to on for the current item.
onMenuClick	MouseEventHandler<HTMLButtonElem ent>	—	Makes the kebab a real, separately focusable button that opens the view's actions. Without it the kebab is a decorative affordance (`aria-hidden`).
menuLabel	string	View actions	Accessible name of the kebab button. Default "View actions".
onClick	MouseEventHandler<HTMLElement>	—	—
asChild	boolean	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

09 · HEADER & MENUS

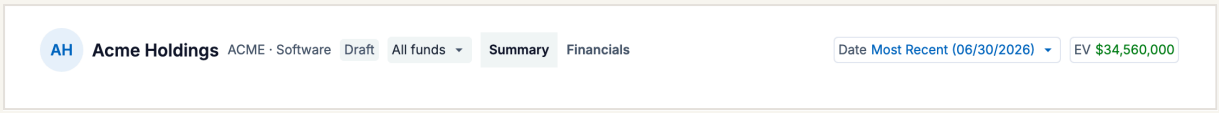
CompanyInfo

REACT CompanyInfo FILE src/components/header/Chrome.tsx FIGMA Company info

Company info — the Secondary Menu bar under the primary bar.

Composed from Badge, FilterDropdown, SecondaryMenu, InformationLabel and Selector. On a firm page it carries the page title; on a company it carries the company name and its tab row.

Accessibility: the title is a heading (level 1 by default) so the page has an outline; pass `titleLevel` when another heading already owns level 1.



Storybook · Header/Chrome › Company Info Bar

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
namerequired	ReactNode	—	The page or company title.
avatar	ReactNode	—	An Avatar instance, for a company with its own mark.
titleLevel	false | 1 | 2 | 3 | 4 | 5 | 6	1	Heading level of the title (default 1, the page title). `false` renders a plain span.
meta	ReactNode	—	Ticker, sector and other metadata beside the name.
status	ReactNode	—	Badge beside the name, e.g. Draft.
filter	ReactNode	—	A FilterDropdown beside the name, e.g. Filter by Fund.
children	ReactNode	—	The tab row: SecondaryMenu with its items.
end	ReactNode	—	The page's selectors and actions, pinned right.

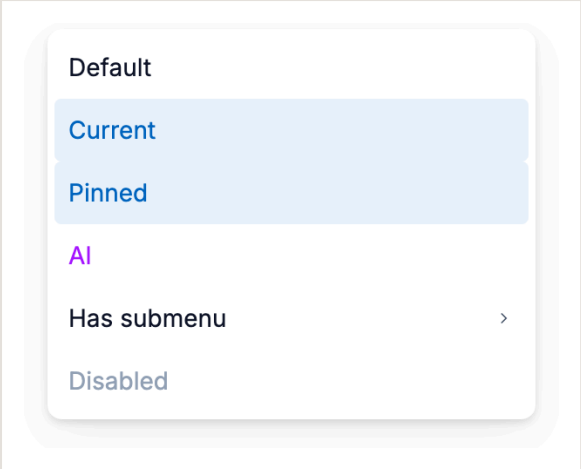
09 · HEADER & MENUS

SubmenuItem

REACT SubmenuItem FILE src/components/header/Menus.tsx FIGMA Submenu Item VARIANTS State

Submenu Item — one row in a dropdown menu.

Its role follows the MenuPanel it sits in: a plain link/button in a `navigation` panel (`aria-current`), `menuitem` in a `menu`, `option` (`aria-selected`) in a `listbox`. Outside any panel it is a plain row. The `pinned` / `ai` states are read out as well as shown.



Storybook · Header/Menus › Item States

PROPS

PROP	TYPEREACTNODE	DEFAULT—	DESCRIPTIONTHE PAGE OR COMPANY TITLE.
onClick	MouseEventHandler<HTMLElement>	—	—
icon	ReactNode	—	—
hasSubmenu	boolean	—	Draws the trailing chevron for an item that opens a further level.
expanded	boolean	—	Whether that further level is open (`aria-expanded`). Only meaningful with `hasSubmenu`.
current	boolean	—	The item the user is on.
state	`"default"` | `"ai"` | `"pinned"`	default	`pinned` keeps a favourite at the top; `ai` marks a generative option. Announced as text too.
asChild	boolean	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

09 · HEADER & MENUS

CompanyDropdown

REACT CompanyDropdown FILE src/components/header/Controls.tsx FIGMA Company Dropdown

Company Dropdown — the company switcher in the primary bar.

The trigger only; the screen owns the panel (see CompanyDropdownPanel, a listbox). It announces `aria- haspopup="listbox"`; name the panel with `aria-controls`.



Storybook · Header/Controls › Company Dropdown Closed

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
expanded	boolean	false	The switcher's panel is open (`aria-expanded`). Default false.

09 · HEADER & MENUS

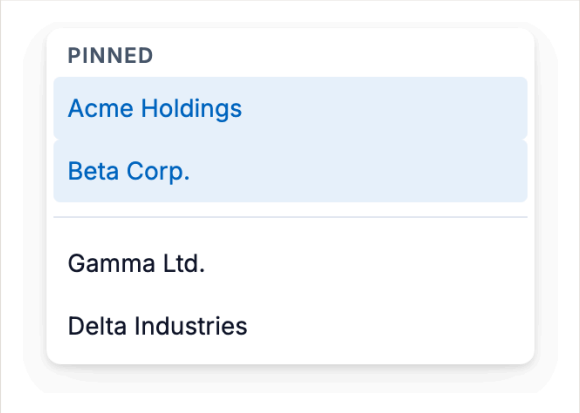
CompanyDropdownPanel

REACT CompanyDropdownPanel FILE src/components/header/Menus.tsx FIGMA Company Dropdown/Default

Company Dropdown panel — the open company switcher.

Choosing a company is a selection, so this is a `listbox` of `option`s (`aria-selected` on the current one) with Up/Down/Home/End, type-ahead, Esc and Tab (via `onClose`). Pass `initialFocus` when the screen opens it so focus moves in and returns to the trigger on close.

Pinned entries are shown as their own group separated by a Stroke/Divider above the full list; with none pinned this is the plain scrollable list.



Storybook · Header/Menus › Company Switcher

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
companiesrequired	`CompanyOption[]`	—	—
currentId	`string`	—	—
onSelect	`((id: string) => void)`	—	—
label	`string`	`Companies`	Accessible name. Default "Companies".
onClose	`(() => void)`	—	`menu` / `listbox` only. Called on Esc, Tab and (with `initialFocus`) an outside click.
initialFocus	`boolean`	`false`	`menu` / `listbox` only. Moves focus onto the current (else first) row when mounted and returns it to the opener on unmount. Opt-in so a panel rendered statically in a gallery never steals focus.

09 · HEADER & MENUS

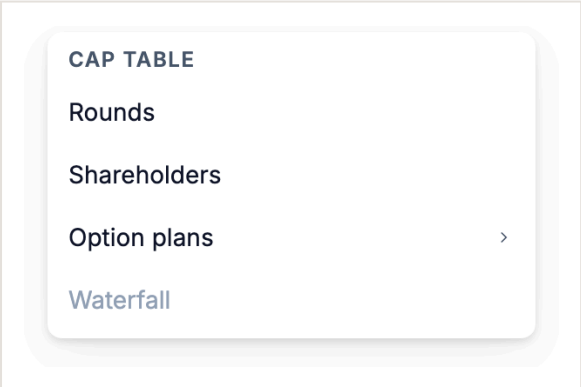
SectionSubMenu

REACT SectionSubMenu FILE src/components/header/Menus.tsx FIGMA Captable sub-menu VARIANTS State

Section sub-menu — the navigation panel for one product section.

Figma ships this twice, as "Captable sub-menu" and "Valuations sub-menu", which share a single token contract so the two sections navigate identically. One component covers both: pass the section's own items.

It is a navigation disclosure — a labelled `<nav>` of links/buttons, not an ARIA menu — so Tab walks the rows and the current one has `aria-current`.



Storybook · Header/Menus › Section Menu

PROPS

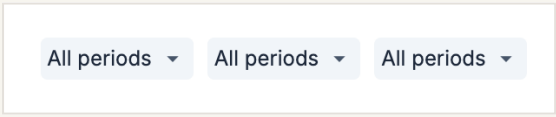
PROP	TYPE	DEFAULT	DESCRIPTION
itemsrequired	{ id: string; label: ReactNode; hasSubmenu?: boolean | undefined; disabled?: boolean | undefined; } []	—	—
onClose	(() => void)	—	`menu` / `listbox` only. Called on Esc, Tab and (with `initialFocus`) an outside click.
initialFocus	boolean	false	`menu` / `listbox` only. Moves focus onto the current (else first) row when mounted and returns it to the opener on unmount. Opt-in so a panel rendered statically in a gallery never steals focus.
label	string	Companies	The section this menu belongs to, e.g. "Cap table" or "Valuations". Also the nav's accessible name.
currentId	string	—	—
onSelect	((id: string) => void)	—	—

09 · HEADER & MENUS

FilterDropdown

REACT FilterDropdown FILE src/components/header/Controls.tsx FIGMA Filter dropdown

Filter dropdown — a filter control in a toolbar. The trigger only; the screen owns the listbox.



Storybook · Header/Controls › Filter Dropdowns

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
children	ReactNode	—	The value in force, not the filter's name. A filter reading "All periods" tells the user more than one reading "Period".
label	string	Ask AI	The filter's name, read out before the value ("Period: All periods") but not shown.
expanded	boolean	false	Its listbox is open (`aria-expanded`). Default false.

09 · HEADER & MENUS

SearchBar

REACT SearchBar FILE src/components/header/Controls.tsx FIGMA Search bar

Search bar — global search in the primary bar.

Its placeholder names the shortcut ("Search (Ctrl+K on Windows)"). Filters as you type; it does not submit.

The ref, `className`'s siblings and every other prop (`aria-*`, `data-testid`, `value`…) go to the `<input>` so the Ctrl+K handler can focus it; `className` styles the wrapper, which is a `role="search"` landmark. The input is named "Search" by default (override with `aria-label` / `aria-labelledby`); a placeholder is not a name.



Storybook · Header/Controls › Search

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
shortcut	string	—	An optional keyboard-shortcut hint shown at the right edge.
inputClassName	string	—	Class for the styled wrapper (`className` is applied there too — it is the visual root).

09 · HEADER & MENUS

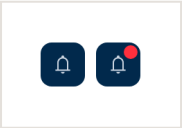
Notification

REACT Notification FILE src/components/header/Controls.tsx FIGMA Notification

Notification — the notification bell with an unread mark.

Accessibility: the glyph is Icon/Primary, not Icon/On Brand. On Background/Brand Subtle, Icon/On Brand resolves to 1.12:1 in Light and 1.21:1 in Dark, because ground and token invert together; Icon/Primary holds 15.91:1 and 14.11:1.

The dot is decorative, so the unread state is also carried in the accessible name rather than by the red alone. Announce *new* arrivals from the screen with its own live region; this control is static.



Storybook · Header/Controls › Notification States

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
unread	boolean	—	Unread state. The dot means unread, not urgent, and carries no count.
label	string	Ask AI	Accessible name when everything is read. Default "Notifications".
unreadLabel	string	Notifications, unread	Accessible name when unread. Default "Notifications, unread".

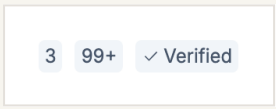
09 · HEADER & MENUS

Badge

REACT Badge FILE src/components/header/Controls.tsx FIGMA Badge

Badge — a small count or status marker.

Raised from 10px to the 12px floor (rule R10). A badge over 99 shows "99+" rather than growing. A bare count reads as a number: give it context with `aria-label` (e.g. "3 unread") or place it inside a labelled control.



Storybook · Header/Controls › Badges

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
icon	ReactNode	caret	—

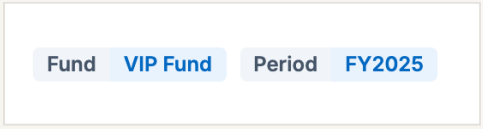
09 · HEADER & MENUS

ComboTag

REACT ComboTag FILE src/components/header/Controls.tsx FIGMA Combo tag

Combo tag — a paired label and value chip.

Use it where a filter shows both its dimension and its value.



Storybook · Header/Controls › Combo Tags

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	ReactNode	—	The value in force.
label	ReactNode	Ask AI	The dimension.

09 · HEADER & MENUS

CurrencySelector

REACT CurrencySelector FILE src/components/header/Controls.tsx FIGMA Currency Selector

Currency Selector — the display-currency switcher.

Currency is a display concern. Switching it must never imply a conversion has been recorded against the data.

It announces `aria-haspopup="menu"` only when it can open something (an `onClick` or `expanded` is supplied); the screen owns the picker.



Storybook · Header/Controls › Currency Selectors

PROPS

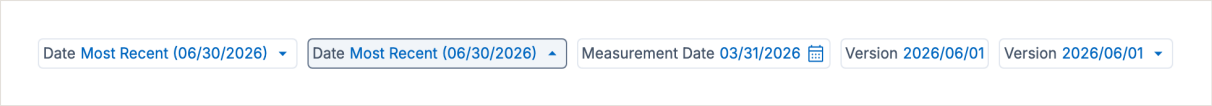
PROP	TYPE	DEFAULT	DESCRIPTION
children	ReactNode	—	The display unit, e.g. "($) Thousands".
currency	ReactNode	—	The currency code, shown in the first part of the chip, e.g. "USD".
expanded	boolean	false	Its picker is open (`aria-expanded`). Omit for a read-only display.

09 · HEADER & MENUS

Selector

REACT Selector FILE src/components/header/Controls.tsx FIGMA Selector
VARIANTS `Drop-Down`, `Picker label`

Selector — a compact labelled value picker ("Date Most Recent (06/30/2026) ▾"). The trigger only: the screen owns the menu it opens. `aria-haspopup="menu"` is set only when there is a dropdown and something to open (`onClick` or `expanded`).



Storybook · Header/Controls › Selectors

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	`ReactNode`	—	The current value.
label	`ReactNode`	`Ask AI`	What is being picked, e.g. "Measurement Date".
surface	`"brand"` | `"surface"`	`brand`	`brand` (default) sits on the navy Primary Menu bar; `surface` sits on a light page or company header (Financials Date, Financials Version).
expanded	`boolean`	`false`	Its menu is open: sets `aria-expanded` and flips the chevron.
icon	`"caret"` | `"calendar"`	`caret`	The trailing glyph: a dropdown caret (default) or a calendar for date pickers.
dropdown	`boolean`	`true`	Figma `Drop-Down`: show the trailing glyph. Default true; false for a read-only value.

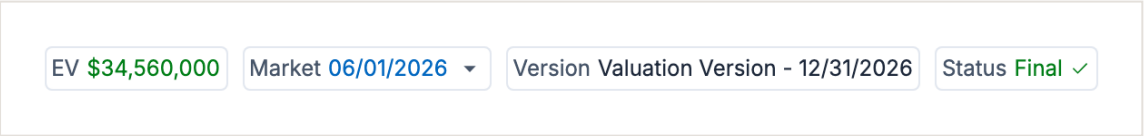
09 · HEADER & MENUS

InformationLabel

REACT InformationLabel FILE src/components/header/Controls.tsx FIGMA Information Label

Variants `Has drop down`, `Long Value`, `L1`–`L3`

Information Label — a bordered label and value with an inline status marker. A screen-reader-only ": " separates label from value so they read as a pair.



Storybook · Header/Controls › Information Labels

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`	`ReactNode`	`Ask AI`	Figma `L1` / `Label`. Omit to hide it.
`value`	`ReactNode`	—	Figma `L2` / `Amount`. Omit to hide it.
`longValue`	`ReactNode`	—	Figma `L3` / `Long Value`: a longer third text, e.g. "Valuation Version - 12/31/2026". Omit to hide it.
`dropdown`	`boolean`	`true`	Figma `Has drop down`: a trailing caret, for a value the user can change. The screen owns the menu.
`marker`	`ReactNode`	—	A status marker. It reinforces the words; it never replaces them (R8).
`tone`	`"positive"` | `"brand"`	`positive`	`positive` (default) for amounts; `brand` for a value the user can pick.

09 · HEADER & MENUS

ValuationInfo

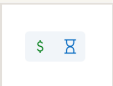
REACT ValuationInfo FILE src/components/header/ValuationInfo.tsx FIGMA Valuation Info

Variants Open (None · Values · Dates · Both)

Valuation Info — the collapsible valuation context at the right end of Company info on company valuation pages.

Two 24px toggles on Background/Subtle. The `$` toggle (Icon/Positive) reveals Equity Value and Unrealized Firm Total; the hourglass toggle (Icon/Brand) reveals the Market date and Version pickers. Figma `Open` = None · Values · Dates · Both.

Accessibility: the toggles are icon-only, so each carries a constant accessible name ("Valuation values", "Market date and version") plus `aria-expanded` and `aria-controls`; the pickers carry `aria-haspopup` and `aria-expanded`.



Storybook · Header/ValuationInfo › Collapsed

PROPS

PROPDESCRIPTION	TYPELABEL	DEFAULT	DESCRIPTIONREACTNODE
`open`	`"none"` | `"both"` | `"values"` | `"dates"`	—	Controlled state. Omit to let the component own it.
`defaultOpen`	`"none"` | `"both"` | `"values"` | `"dates"`	`none`	Initial state when uncontrolled. Default `none` — both collapsed, the resting state.

PROP	DESCRIPTION	TYPE	LABEL	DEFAULT	DESCRIPTION	REACTNODE
`onOpenChange`		`((next: ValuationInfoOpen) => void)`		—	—	
`equityValue`		`ReactNode`		—	Equity Value, e.g. "$34,560,000".	
`unrealizedFirmTotal`		`ReactNode`		—	Unrealized Firm Total, e.g. "$48,871,695".	
`marketDate`		`ReactNode`		—	Market date, e.g. "06/01/2026".	
`version`		`ReactNode`		—	Valuation version, e.g. "2026/06/01".	
`onMarketDateClick`		`(() => void)`		—	The Market date / Version pickers are triggers only — the screen owns the menus they open.	
`onVersionClick`		`(() => void)`		—	—	
`marketDateExpanded`		`boolean`		—	The Market date menu is open (`aria-expanded` on its picker).	
`versionExpanded`		`boolean`		—	The Version menu is open (`aria-expanded` on its picker).	

09 · HEADER & MENUS

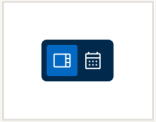
ToolSwitch

REACT ToolSwitch FILE src/components/header/Controls.tsx FIGMA Valuation VARIANTS State

Tool-switch — the paired Valuation / Workboard control.

Exactly one side is always selected; there is no unselected state.

Accessibility: a `radiogroup` of two `radio`s (`aria-checked`) with one tab stop. Left/Right/Up/Down and Home/End move to and select the neighbour (selection follows focus), per the APG radio-group pattern.



Storybook · Header/Controls › Tools

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	`"valuations" | "workboard"`	—	—
onChange	`((next: ToolSwitchValue) => void)`	—	—

09 · HEADER & MENUS

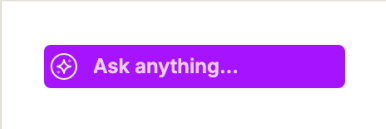
AITool

REACT AITool FILE src/components/header/Controls.tsx FIGMA AI tool VARIANTS State

AI tool — the AI assistant entry point in the chrome.

Reserved for genuinely generative actions. The gradient stroke in Figma is the intentional exception and has no Dark-mode counterpart.

The ref and the remaining native props go to the interactive element — the `<button>` in `trigger`, the `<input>` in `input` (where `className` styles the wrapping `<form>`). Enter submits.



Storybook · Header/Controls › AI Tool Input

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
state	`"trigger" | "input"`	`trigger`	`trigger` is the entry point; `input` is the open prompt field.
value	`string`	—	Prompt text (input state). Omit to let the field own it.
defaultValue	`string`	—	Initial prompt text when uncontrolled.
onValueChange	`((value: string) => void)`	—	—
onSubmit	`((value: string) => void)`	—	Input state: called with the prompt when the user presses Enter.
placeholder	`string`	`Ask anything…`	Input state: placeholder. Default "Ask anything...".
inputLabel	`string`	`Ask the assistant`	Input state: accessible name. Default "Ask the assistant".
label	`string`	`Ask AI`	Trigger state: accessible name when there is no visible text. Default "Ask AI".

09 · HEADER & MENUS

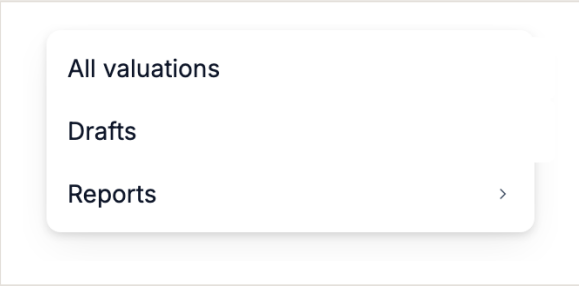
MenuPanel

REACT MenuPanel FILE src/components/header/Menus.tsx

MenuPanel — the floating surface a dropdown's rows sit on.

Tokens: Background/Surface, corner Semantic: Radius/S, elevation Elevation/Overlay. This is the shared shell behind Company Dropdown, the Captable sub-menu and the Valuations sub-menu.

Accessibility: `kind="navigation"` (default) is a labelled `<nav>`; `kind="menu"` / `"listbox"` implement the APG composite keys (one tab stop, Up/Down/Home/End, type-ahead, Esc and Tab call `onClose`).



Storybook · Header/Menus › Navigation

PROPS

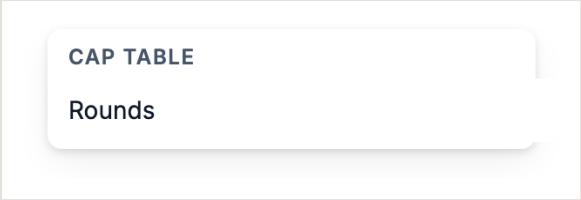
PROP	TYPE	DEFAULT	DESCRIPTION
`label`	`string`	`Companies`	Accessible name of the panel. Provide this or `aria- labelledby`.
`kind`	`"menu" | "listbox" | "navigation"`	`navigation`	See {@link MenuPanelKind}. Default `navigation`.
`initialFocus`	`boolean`	`false`	`menu` / `listbox` only. Moves focus onto the current (else first) row when mounted and returns it to the opener on unmount. Opt-in so a panel rendered statically in a gallery never steals focus.
`onClose`	`(() => void)`	—	`menu` / `listbox` only. Called on Esc, Tab and (with `initialFocus`) an outside click.

09 · HEADER & MENUS

MenuGroupLabel

REACT MenuGroupLabel FILE src/components/header/Menus.tsx

A group label inside a MenuPanel. Presentational on its own; inside a `menu` / `listbox` panel, wrap the group's rows in {@link MenuGroup} so the label names them.



Storybook · Header/Menus › Group Label

PROPS

No component-specific props; accepts the native element attributes.

---

09 · HEADER & MENUS

MenuGroup

REACT MenuGroup FILE src/components/header/Menus.tsx

A labelled `role="group"` of rows inside a `menu` / `listbox` MenuPanel.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
label	ReactNode	Companies	The visible group heading. Rendered as a MenuGroupLabel and used as the group's name.

10

COMPONENTS · 8 COMPONENTS

NAVIGATION

FIGMA	VARIANTS	REACT	FILE
Tab Item	State	= `TabItem`	`navigation/Tabs.ts x`
Tabs	—	= `Tabs`	`navigation/Tabs.ts x`
Breadcrumb Item	State	⊕ `Breadcrumb` (`items[]`)	`navigation/Breadcr umb.tsx`
Breadcrumb	—	= `Breadcrumb`	`navigation/Breadcr umb.tsx`
Page Item	State	= `PageItem`	`navigation/Paginat ion.tsx`
Pagination	—	= `Pagination` (+ `rowsPerPage` / `rowsPerPageOptions` / `onRowsPerPageChange` — a labelled rows-per-page Select)	`navigation/Paginat ion.tsx`
Step	State	= `Step`	`navigation/Stepper .tsx`
Stepper	—	= `Stepper`	`navigation/Stepper .tsx`

10 · NAVIGATION

TabItem

REACT TabItem FILE src/components/navigation/Tabs.tsx FIGMA Tab Item VARIANTS State

Tab Item — one tab in a Tabs bar.

Keep labels to one or two words and never let them wrap.

Accessibility: the drawn control is under 44px, so the target is carried by padding rather than the label box. `aria-selected` is always present (`true` / `false`). Keyboard handling lives on the parent `Tabs` .

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`active`	`boolean`	—	Exactly one tab is active at all times. Inactive tabs are `tabIndex=-1` and reached with the arrow keys.
`value`	`string`	—	Identifies the tab. Inside `Tabs`, a `TabItem` and a `TabPanel` with the same `value` are wired together (`aria-controls` ↔ `aria-labelledby`) automatically.
`controls`	`string`	—	Explicit id of the panel this tab controls. Overrides the id derived from `value`.

10 · NAVIGATION

Tabs

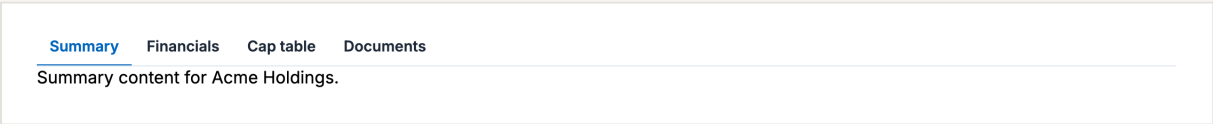
REACT Tabs FILE src/components/navigation/Tabs.tsx FIGMA Tabs

Tabs — switches the view below between sibling sections of the same object.

Use tabs only for peer views of one thing — a company's Overview, Cap table, Waterfall. Not for steps in a process; that is Stepper. Not to page through records; that is Pagination.

If the set does not fit, the information architecture is wrong — nest it rather than scrolling the bar.

Keyboard (automatic activation): Left/Right move focus *and* select the tab (the focused tab is clicked), Home/End jump to the first/last, Tab leaves the set. A name (`label` or `aria-labelledby`) is required.



Storybook · Navigation/Tabs › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`	`string`	—	—

10 · NAVIGATION

Breadcrumb

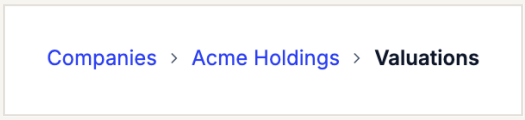
REACT Breadcrumb FILE src/components/navigation/Breadcrumb.tsx FIGMA Breadcrumb Item VARIANTS State

Breadcrumb — where this page sits in the hierarchy, and a way back up.

A breadcrumb reflects structure, not history. It is not a back button: it shows the path from the root to here, no matter how the user arrived.

In deep trails, collapse the middle rather than the ends — the root and the parent are the two the user actually needs.

Accessibility: labelled `nav`, an ordered list, the separator lives inside each `li` and is hidden from assistive technology, the last item carries `aria-current="page"`. Items with no `href` render as buttons so they stay keyboard operable.



Storybook · Navigation/Tabs › Breadcrumb

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`items` required	`BreadcrumbItemData[]`	—	Root first, current page last. The last item is always the current one.
`label`	`string`	`Breadcrumb`	Accessible name of the landmark. Default "Breadcrumb".
`renderLink`	`((props: BreadcrumbLinkRenderProps ) => ReactNode)`	—	Render links with a client router's link component. Receives the classes to apply.

10 · NAVIGATION

PageItem

REACT PageItem FILE src/components/navigation/Pagination.tsx FIGMA Page Item VARIANTS State

Page Item — one page number in a Pagination control.

The current page stays in the tab order and the accessibility tree: it is `aria-current="page"` + `aria-disabled`, not natively `disabled`, and activating it does nothing. Pass `disabled` to disable any other page.



Storybook · Navigation/Tabs › Pagination

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`page` required	`string | number`	—	—
`current`	`boolean`	—	The page being viewed. Not clickable, and the only one with a fill.

Pagination

REACT Pagination FILE src/components/navigation/Pagination.tsx FIGMA Pagination

Pagination — moves through a result set page by page.

The first and last page are always shown so the user can see how large the set is. Previous is disabled on the first page and Next on the last — never hidden: a control that disappears moves everything beside it and costs the user their aim.

For a long grid people scan rather than navigate, consider infinite scroll; pagination is for sets people cite by page.

With `rowsPerPage` + `onRowsPerPageChange` it renders a labelled rows-per-page `Select` at the start of the bar. Changing it is the caller's cue to reset `page` to 1. `ref`, `className` and other native props land on the outer element (the `nav`, or the bar when rows-per-page is shown).

`page` is controlled; omit it (use `defaultPage`) for uncontrolled use. A visually hidden status announces the page after each change.

PROPS

PROPTHE PAGE BEING VIEWED. NOT CLICKABLE, AND THE ONLY ONE WITH A FILL.	TYPEPAGINATION	DEFAULTREACT	DESCRIPTION
`pageCount` required	`number`	—	—
`page`	`number`	—	The page being viewed (controlled). Omit it and use `defaultPage` to let Pagination own it.
`defaultPage`	`number`	`1`	Initial page when uncontrolled. Default 1.
`onPageChange`	`((page: number) => void)`	—	—
`label`	`string`	`Pagination`	Accessible name of the landmark. Give each Pagination on a page its own. Default "Pagination".
`previousLabel`	`string`	`Previous page`	Accessible name of the previous button. Default "Previous page".
`nextLabel`	`string`	`Next page`	Accessible name of the next button. Default "Next page".
`statusText`	`((page: number, pageCount: number) => string)`	`(p, n) => `Page ${p} of ${n}``	Builds the text announced when the page changes. Default "Page 2 of 9".
`siblingCount`	`number`	`1`	How many pages to show either side of the current one.

PROPTHE PAGE BEING VIEWED. NOT CLICKABLE, AND THE ONLY ONE WITH A FILL.	TYPEPAGINATION	DEFAULTREACT	DESCRIPTION
`rowsPerPage`	`number`	—	Rows per page. Set it together with `onRowsPerPageChange` to render the labelled rows-per- page control beside the pages.
`rowsPerPageOptions`	`readonly number[]`	`[10, 25, 50, 100]`	The choices offered. Default 10 · 25 · 50 · 100.
`onRowsPerPageChange`	`((rows: number) => void)`	—	—
`rowsPerPageLabel`	`ReactNode`	`Rows per page`	Visible label of that control. Default "Rows per page".

10 · NAVIGATION

Step

REACT Step FILE src/components/navigation/Stepper.tsx FIGMA Step VARIANTS State

Step — one stage in a Stepper. Renders an `li`: place it inside an `ol` (`Stepper` does).

Complete shows a check rather than its number: the number has stopped being useful once the step is done. Error keeps the step reachable — it is a state, not a dead end.

Accessibility: the current step is `aria-current="step"`; complete and error are also spoken ("Completed", "Error") so state is never colour/icon only.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`index`required	`string | number`	—	The step number. Complete steps show a check instead.
`label`	`ReactNode`	`Progress`	—
`state`	`"current" | "error" | "complete" | "upcoming"`	`upcoming`	—

10 · NAVIGATION

Stepper

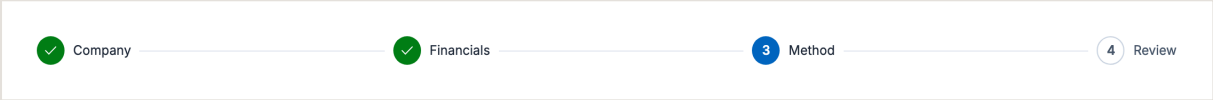
REACT Stepper FILE src/components/navigation/Stepper.tsx FIGMA Stepper

Stepper — progress through a sequence that must be completed in order.

Use for a process with a defined beginning and end — onboarding a company, running a valuation to sign-off. Not for navigation between peer views; that is Tabs.

Show every step from the start, including the ones not yet reachable: the value of a stepper is that it tells the user how much is left. Never mark a step Complete until it actually is — a stepper that lies about progress is worse than no stepper.

Accessibility: a labelled `nav` holding an ordered list, so position ("2 of 4") is announced.



Storybook · Navigation/Tabs › Stepper

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
stepsrequired	`(Omit<StepProps, "index"> & { index?: string | number | undefined; })[]`	—	—
label	`string`	`Progress`	Accessible name of the landmark. Default "Progress".

10 · NAVIGATION

TabsGroup

REACT TabsGroup FILE src/components/navigation/Tabs.tsx

Tabs Group — shares the generated id base between `Tabs` and the `TabPanel`s that sit beside it, so `value` alone wires `aria- controls` ↔ `aria-labelledby`. Optional: without it, pass `controls` / `labelledBy` explicitly.

PROPS

No component-specific props; accepts the native element attributes.

10 · NAVIGATION

TabPanel

REACT TabPanel FILE src/components/navigation/Tabs.tsx

Tab Panel — the content a tab controls. Render it inside `Tabs` ' sibling container (not inside the tablist) and set `hidden` on the inactive ones, or render only the active one. It is focusable so content without a focusable child can still be reached from the tab.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`value`	`string`	—	Matches the `value` of its `TabItem`.
`labelledBy`	`string`	—	Explicit id of the tab that labels this panel. Overrides the id derived from `value`.

11

COMPONENTS · 11 COMPONENTS

TABLE & CELLS

FIGMA	VARIANTS	REACT	FILE
cell	State × Type + 5 booleans (145)	= `Cell` (+ `label`, `tooltip`, `trailingIcon` for the Label / Tooltip / Icon Type properties)	`table/Cell.ts x`
Header	Style + 6 booleans (11)	= `ColumnHeader` (`kind`, `mark`, `label`, `tooltip`, `icon`, `input`, `action`; + `grow` / `width` / `span` column sizing; + `tone="subtle"` light header band)	`table/ColumnH eader.tsx`
Row-reading	27	⊕ `RowHeader` (the row's first cell) + `Row`	`table/RowHead er.tsx`, `table/Row.tsx`
Row-input	37	⊕ `RowHeader` (the row's first cell) + `Row`	`table/RowHead er.tsx`, `table/Row.tsx`
Content_Cell	Content	= `ContentCell`	`table/Content Cell.tsx`
Ledger	—	= `Ledger`	`table/Content Cell.tsx`
Modal_Status	State	= `ModalStatus` (+ `published` state)	`table/Status. tsx`
Valuation Status	State	= `ValuationStatus`	`table/Status. tsx`

FIGMA	VARIANTS	REACT	FILE
Footnote	Content	= `Footnote`	`table/Footnot e.tsx`
cell-icon/Dropdown, cell- icon/Calendar	—	glyphs only — drawn by `Cell`'s `icon` slot	—
—	—	+ `DataGrid`	`table/DataGri d.tsx`
Data Cell, Title Item Row, Column Header Workboard (Navigation V4 – Prism)	Type × Style	+ `Table` — composes `Cell`, `RowHeader`, `ColumnHeader`	`table/Table.t sx`

Merge. `Row-reading` and `Row-input` have the same anatomy — a mark bar, an optional bulk checkbox, the label, a trailing glyph. They are the **row header**: the first cell of a row. In code that is `RowHeader` (`type` readable · data · input · total · divider; `mark`, `bulk`, `icon`), with Reading's `group` (a left bar) and Input's `groupStart` / `groupEnd` (rules on top and underneath) as three props of one component. The row itself is `Row`; the read/edit distinction on the rest of the row lives on `Cell type`. Use `RowHeader` for the first cell of every row instead of a text `Link` or a bare `Cell`.

Addition. Figma has no single grid component — each screen assembles Header, Rows and Cells itself. `DataGrid` is that assembly, kept in one place so sticky header behaviour is not re-implemented per page.

---

11 · TABLE & CELLS

Cell

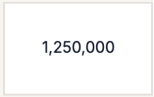
`REACT` Cell `FILE` src/components/table/Cell.tsx `FIGMA` cell `VARIANTS` State × Type + 5 booleans (145)

Cell — the atom of the data grid.

The Figma set is 145 variants: State × Type plus a set of content booleans. Here Type and State are props and the booleans are optional slots.

These cells use a 2px corner, which maps to Semantic: Radius/2XS.

Accessibility: renders `role="cell"` and belongs inside a `Row` inside a `DataGrid` (an ARIA `table`). `error`, `draft` and `total` reach assistive technology as visually hidden text after the content — `aria-selected` is not valid on a table cell, so `selected` is visual only; mark the row with `Row selected` instead.



Storybook · Data/Table/Cell › Readable

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`type`	`"group"` | `"input"` | `"readable"` | `"data"` | `"divider"`	`readable`	`readable` takes Text/Primary, `input` takes Text/Editable, and `data` takes Text/Sourced — the colour states where the number came from.
`state`	`"selected"` | `"default"` | `"hover"` | `"pressed"` | `"total"` | `"error"` | `"draft"`	`default`	—
`selected`	`boolean`	—	Draws the selection ring on top of any state. With `state="error"` the ring is red; an unselected error cell has no outline. `state="selected"` is the same as `selected` on a default cell.
`numeric`	`boolean`	—	Right-aligns and applies tabular figures. Use for every money column.
`footnote`	`ReactNode`	—	A superscript reference or currency marker. See `Footnote`.
`icon`	`ReactNode`	—	A leading glyph. Wrap it in `Icon`.
`label`	`ReactNode`	—	Figma `Label`: a small tag after the content.
`tooltip`	`ReactNode`	—	Figma `Tooltip`: explanatory text. Draws the corner indicator and shows the text on hover and keyboard focus. Never put the only copy of something here.
`trailingIcon`	`ReactNode`	—	Figma `Icon Type`: the trailing 12px glyph. `dropdown` for select and picker cells, `calendar` for date cells, or pass your own glyph.
`groupStart`	`boolean`	—	Draws the left bracket edge of a grouped run.
`groupEnd`	`boolean`	—	Draws the bottom bracket edge of a grouped run.
`span`	`number`	—	Columns this cell spans inside a DataGrid. Default 1.
`errorMessage`	`string`	—	With `state="error"`: why the value is wrong. Read out as part of the cell as hidden text, because the red tint alone never carries meaning (R8).

ColumnHeader

REACT ColumnHeader FILE src/components/table/ColumnHeader.tsx FIGMA Header
VARIANTS Style + 6 booleans (11)

Header — the data-grid column header.

Tokens: fill Background/Brand Pressed with Text/On Brand, so it reads as chrome rather than as data. `tone="subtle"` swaps to Background/Subtle with Text/Secondary over a Stroke/Strong rule. Use one tone per grid.

The header is sticky in use — render it outside the scroll container.

Accessibility: the header is a `role="columnheader"` container; when sortable the sort control is a real `<button>` inside it (Enter / Space / click), so trailing `actions` buttons are separate controls and never trigger a sort. `aria-sort` is set on the header (`none` while sortable but unsorted).



Storybook · Data/Table/ColumnHeader › Brand

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`numeric`	`boolean`	—	Right-aligns the label to sit over a numeric column.
`sort`	`"none" | "ascending" | "descending" | "asc" | "desc" | null`	—	Current sort direction, or `null` / `'none'` when this column is not the sort key. Accepts the `GridColumnHeader` vocabulary (`'ascending'` / `'descending'`) as well as `'asc'` / `'desc'`.
`onSortChange`	`((next: "asc" | "desc") => void)`	—	Makes the header sortable: a native button inside the header carries the activation. Called with the direction to sort by next.
`actions`	`ReactNode`	—	Trailing controls — a filter menu, an action button.
`grow`	`number`	—	Share of the grid's leftover width this column takes (DataGrid). Default 1.
`width`	`string`	—	A fixed column track instead of the header-sized one (DataGrid).
`span`	`number`	—	Columns this header spans. Default 1.
`tone`	`"brand" | "subtle"`	`brand`	`brand` (default) — the navy grid chrome. `subtle` — the light header band: Subtle ground, secondary text, a strong bottom rule, for tables that sit inside a card or a modal.

PROP	TYPE	DEFAULT	DESCRIPTION
kind	"default" | "action-button"	default	Figma Style. default is the navy chrome; action-button is the white header that stands in for an action, such as the trailing "add column" header.
mark	boolean	—	Figma Mark: a brand rule across the top edge, marking a period or a group boundary.
label	ReactNode	—	Figma Label: a small tag beside the title, e.g. "Projection".
tooltip	ReactNode	—	Figma Tooltip: explanatory text. Draws the corner indicator and shows the text on hover and keyboard focus. Never put the only copy of something here.
icon	ReactNode	—	Figma Icon: a trailing 12px glyph. Wrap nothing — pass the glyph itself.
input	boolean	—	Figma Input: the trailing glyph takes the accent tint, marking an editable or add-new header.
action	boolean	—	Figma Action: the header's action is engaged; draws the accent rule on the top and left edges.
selected	boolean	—	Whole column selected: Brand-subtle fill and an 8px Stroke/Brand marker on top (Figma selected top).
active	boolean	—	The column of the active cell: a 2px Stroke/Brand rule across the top.

11 · TABLE & CELLS

RowHeader

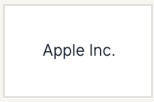
REACT RowHeader FILE src/components/table/RowHeader.tsx FIGMA Row-reading VARIANTS 27

Row Header — the first cell of a table row: its name.

Figma: **Row-reading** (1035:1911) and **Row-input** (1092:3672). They share one anatomy — a mark bar, an optional bulk checkbox, the label, a trailing glyph — and differ only in how they draw grouping, so code carries one component: Reading's group is a left bar, Input's groupStart / groupEnd are rules on top and underneath.

Use it for every row's first cell instead of a text link or a bare Cell. Tokens: Background/Surface, 26px minimum height (Sizing/Row/Compact), padding Spacing/2XS and /XS, Heading/S Light (Semi Bold for total and divider).

Accessibility: role="rowheader" — use inside a Row of a DataGrid. The bulk checkbox, link and icon button are separate native controls, each a normal tab stop.



Storybook · Data/Table/RowHeader › Readable

PROPS

PROP	TYPE	DEFAULT	SRC/COMPONENTS/TABLE/ROWHEADER.TSX
onIconClick	`(() => void)`	—	Makes the trailing glyph a button.
iconLabel	`string`	—	Accessible name of that button, e.g. "Expand Apple Inc.". Required with `onIconClick`.
children	`ReactNode`	—	The label — a name, a period, a figure. Plain text; do not pass a `Link`.
type	`"total" | "input" | "readable" | "data" | "divider"`	`readable`	`readable` → Text/Primary · `data` → Text/Sourced (pulled from a source) · `input` → Text/Editable · `total` → ruled above and below, semibold · `divider` → the shaded section band.
mark	`boolean`	—	Figma `Mark`: a 2px Stroke/Focus bar on the left edge.
bulk	`boolean`	—	Figma `Bulk`: a leading checkbox for bulk actions.
checked	`boolean`	—	Controlled state of the bulk checkbox. Omit to let it own its state.
defaultChecked	`boolean`	—	Initial state of the bulk checkbox when uncontrolled.
onCheckedChange	`((checked: boolean) => void)`	—	—
selectLabel	`string`	—	Accessible name of the bulk checkbox, e.g. "Select Apple Inc.".
group	`boolean`	—	Row-reading `Group`: a 2px Stroke/Control bar on the left edge.
groupStart	`boolean`	—	Row-input `Start Group`: a 2px Stroke/Control rule on top.
groupEnd	`boolean`	—	Row-input `End Group`: a 2px Stroke/Control rule underneath.
icon	`ReactNode`	—	Figma `Icon`: a trailing 12px glyph (the expander "+"). Pass the glyph itself.
footnote	`ReactNode`	—	A superscript reference or currency marker. See `Footnote`.
href	`string`	—	Makes the whole label navigate. The header keeps its own look — it is not drawn as a text link — and underlines on hover and focus.

PROP	TYPE	DEFAULT	SRC/COMPONENTS/TABLE/ROWHEADER.TSX
span	number	—	Columns this header spans inside a DataGrid. Default 1.
selected	boolean	—	Whole row selected: Brand-subtle fill and an 8px Stroke/Brand marker on the left (Figma `Selected left`).

11 · TABLE & CELLS

Row

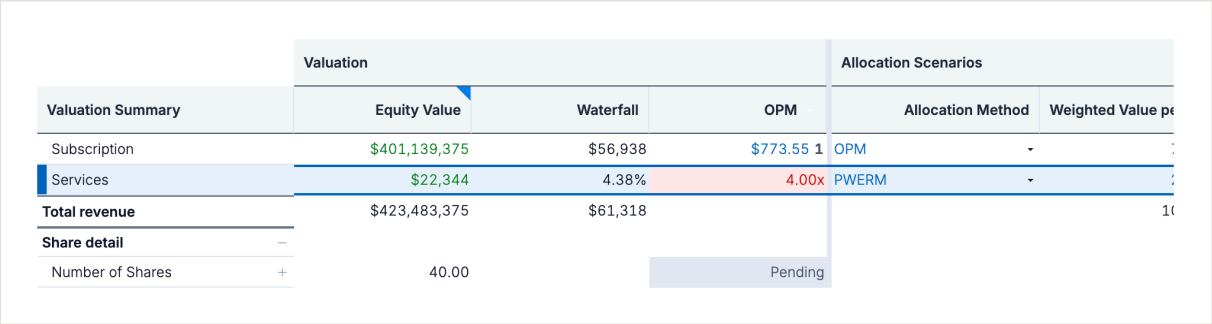
REACT Row FILE src/components/table/Row.tsx FIGMA Row-reading VARIANTS 27

Row — one line of the data grid.

Figma models this as two sets, Row-reading and Row-input, which differ only in whether the editing affordances are suppressed. Here that is the `type` of the cells inside the row, so one component covers both: use `type="readable"` for locked periods and published figures.

Both sets are composed from Cell — change Cell, not this.

Accessibility: `role="row"` — place it directly inside a `DataGrid` (an ARIA `table`). `selected` sets `aria-selected` on the row. The `divider` type is a presentational rule (`role="presentation"`), not a row of the table.



Storybook · Data/Table/Table › Row Selected

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type	`"total"` | `"input"` | `"readable"` | `"data"` | `"divider"`	readable	—
zebra	boolean	—	Alternating background. Apply on odd rows only.
group	boolean	—	A grouped run of rows under a group header.
selected	boolean	—	The row is the current selection (e.g. its row actions are open).

11 · TABLE & CELLS

ContentCell

REACT ContentCell FILE src/components/table/ContentCell.tsx FIGMA Content_Cell VARIANTS Content

Content_Cell — a cell carrying richer content than a number: a user, a label, a date or a note.

Accessibility: `role="cell"` — use it inside a `Row` of a `DataGrid`.



Storybook · Data/Table/DataGrid › Content Cells

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type	`"label"` | `"note"` | `"date"` | `"user"`	`user`	What the cell holds. Drives `data-type` and, for `label`, the accent colour.
leading	`ReactNode`	—	A leading Avatar or Icon. Make it decorative (`aria- hidden`) or give it its own accessible name — it is not named for you.

11 · TABLE & CELLS

Ledger

REACT Ledger FILE src/components/table/ContentCell.tsx FIGMA Ledger

Ledger — a ledger-entry dot, 8px.

Because it is a bare graphic it never carries meaning alone: it must be paired with a tooltip or a label, which is why `label` is required.

LINE ITEM	ACTUALS	PROJECTIONS
Line item	FY2024	FY2025E
VIP FUND		
– Revenue	4,120,000	4,980,000
Product	3,000,000	
Total	**4,120,000**	**4,980,000**

Storybook · Data/GridPatterns › Statement In Data Grid

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	`string`	—	What the dot means. A bare graphic never carries meaning alone (rule R8).

11 · TABLE & CELLS

ModalStatus

REACT ModalStatus FILE src/components/table/Status.tsx FIGMA Modal_Status VARIANTS State

Modal_Status — the workflow state of a valuation record.

Accessibility: the label carries the meaning; the colour only reinforces it (rule R8). That is why the label is never hidden.

`published` — released to the portfolio (a Debt Only valuation's terminal state). It shares Complete's positive tint and is told apart by a leading check glyph and its label.

Storybook · Data/Table/Status › Workflow

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
staterequired	`"draft"` | `"complete"` | `"review"` | `"in-process-usa"` | `"in-process- arg"` | `"final"` | `"published"`	—	—
children	`ReactNode`	—	Overrides the default label for the state.

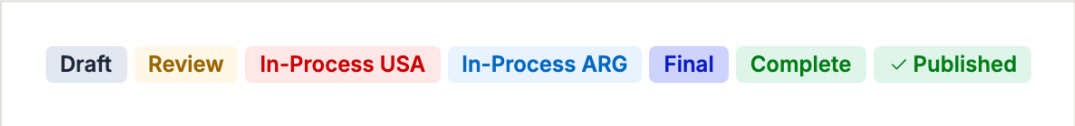
11 · TABLE & CELLS

ValuationStatus

REACT ValuationStatus FILE src/components/table/Status.tsx FIGMA Valuation Status VARIANTS State

Valuation Status — the commercial state of a deal.

Distinct from ModalStatus: that one tracks the work, this one tracks the deal. They can disagree, and both may appear on the same record.



Storybook · Data/Table/Status › All Workflow States

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
staterequired	`"in-service" | "awaiting-payment" | "complete-deal" | "cancelled"`	—	—
children	`ReactNode`	—	Overrides the default label for the state.

11 · TABLE & CELLS

Footnote

REACT Footnote FILE src/components/table/Footnote.tsx FIGMA Footnote VARIANTS Content

Footnote — a superscript reference or currency marker on a figure.

Type is Overline/S: the smallest role in the system and the only one licensed for a marker this size. Never set it below the 12px floor (rule R10) — the previous 8px treatment has been raised.

Accessibility: a static marker is plain text; to tie it to its note, give the note an `id` and set `aria-describedby` on an `interactive` footnote.



Storybook · Data/Table/Footnote › Reference

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`currency`	`string`	—	A currency marker such as `USD`, shown beside the reference.
`ai`	`boolean`	—	Marks a figure a model produced. Takes Text/AI.
`interactive`	`boolean`	—	The footnote reveals its source: the marker becomes a `<button>` inside the `<sup>`, reachable by keyboard. `onClick`, `aria-label` and `aria- describedby` (point it at the note's id) go to that button.

11 · TABLE & CELLS

DataGrid

REACT DataGrid FILE src/components/table/DataGrid.tsx

DataGrid — the layout, scroll and stickiness shell around Row and Cell.

It is a CSS grid: the header row defines the column tracks and every Row is a subgrid of them, so a body cell always fills exactly the width of its column header, and header and body cannot drift apart. Wider than its container, the grid scrolls horizontally inside itself.

Both cell models work inside it: `Cell` rows and the grid-pattern cells (`RowLabelCell`, `GridValueCell`, `InCellControl`).

Table or grid? This is a table. It renders `role="table"` with `row` / `columnheader` / `rowheader` / `cell` children, *not* `role="grid"`. The WAI-ARIA grid role promises a single tab stop, arrow-key movement between cells and Enter/F2 to edit — a model none of these cells implement, because the library draws editable-looking cells (`GridValueCell`, `InCellControl`) but never edits them: each is a plain native control (a `<button>`, or the consumer's own input) that sits in the normal Tab order and is activated with Enter / Space. Reading order and column/row header relationships come from the table roles, and sort state from `aria-sort` on the column header. If you build a genuinely editable spreadsheet on top of this, own the APG grid keyboard model (roving tabindex, arrows, Home/End, Enter/F2, Esc) and set `role="grid"` through your own wrapper — do not just pass it here.

Accessibility: **a name is required** — `label` or `aria-labelledby`.

Line item	FY2024	FY2025
Revenue	4,120,000	4,980,000 USD1
Cost of sales	1,640,000	1,910,000
Operating expenses	980,000	1,050,000
EBITDA	**1,500,000**	**2,020,000**

Storybook · Data/Table/DataGrid › Default

PROPS

PROPPROP	TYPETYPE	DEFAULTDEFAULT	DESCRIPTIONDESCRIPTION
`label`	`string`	—	—
`head`	`ReactNode`	—	The sticky header row. Render `ColumnHeader` / `GridColumnHeader` / `AddColumnHeader` instances here — one per column. Each header's `grow` / `width` / `span` props define its column track.
`groupHead`	`ReactNode`	—	An optional row of `ColumnGroupHeader`s above `head` (use their `span`).
`children`	`ReactNode`	—	The body rows. Render `Row` instances here (directly or in fragments).
`columns`	`string[]`	—	Explicit column tracks, overriding the ones derived from `head`, e.g. `['minmax(max-content, 2fr)', size.control.l]`.
`maxHeight`	`string`	—	Bound the grid's height: the body scrolls inside it and the header sticks to the grid's top. Without it the grid grows with its rows. The scrolling region is keyboard-focusable so it can be scrolled without a pointer.

11 · TABLE & CELLS

Table

REACT Table FILE src/components/table/Table.tsx

Figma Data Cell, Title Item Row, Column Header Workboard (Navigation V4 – Prism)**Variants** Type × Style

Table — the one Prism table for the whole platform.

Valuations, Waterfalls, Financials, Cap Table, Companies, Reports and the Workboard all render this component. Differences come from `columns`, `rows`, `cells` and the props — never from a fork.

It is a spreadsheet-like ARIA `grid`: 26px body rows and 40px headers, a sticky title column, one tab stop with roving focus inside, a rectangular selection drawn as ONE perimeter overlay, user-resizable columns that never go below the width their longest value needs, and an optional formula view.

For a static, read-only table with no selection use `DataGrid`.

State is derived, never stored per cell: fill comes from selection and hover, text colour from `cell.kind` only. Fill precedence is error > selected > pressed > hover > zebra > default.

Accessibility: `role="grid"`, `aria-selected` on selected cells, `aria-readonly` on readable / sourced / total / draft cells, `aria- invalid` on errors. Editable cells carry a hidden "editable" cue so colour is not the only signal. Keys: arrows move, Shift+arrows extend, Ctrl/Cmd+arrows jump, Ctrl/Cmd+Space selects the column, Shift+Space the row, Enter / F2 edits, Esc cancels then clears, Ctrl/Cmd+C copies tab-separated values.



Storybook · Data/Table/Table › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tableIdrequired	`string`	—	Stable id. Persisted column widths are stored under it.
titlerequired	`string`	—	—
columnsrequired	`TableColumnDef[]`	—	—
rowsrequired	`TableRowDef[]`	—	—
cellsrequired	`Record<string, TableCellData>`	—	Keyed `${rowId}:${columnId}`. A missing key is an empty cell.
formulaMode	`boolean`	—	Letters across the top, row numbers down the left, and an fx bar under the table while it holds a selection.
selection	`Selection`	—	Controlled selection. Indices count the *visible* rows and the data columns.
defaultSelection	`Selection`	—	—
onSelectionChange	`((selection: Selection) => void)`	—	—
columnWidths	`Record<string, number>`	—	Controlled user widths, keyed by column id (`TITLE_COLUMN_ID` for the title column).
onColumnResize	`((columnId: string, width: number) => void)`	—	Width is already clamped to the column's minimum and maximum.
onCellCommit	`((cellId: string, value: string) => void)`	—	Called with the cell's `id` and the typed text (or the chosen option).
onAddColumn	`(() => void)`	—	Shows the trailing add-column header.

PROP	TYPE	DEFAULT	DESCRIPTION
addColumnLabel	string	—	—
onAddRow	((rowId: string) => void)	—	Called by a row's + control (addable).
footnotes	Record<number, string>	—	Footnote text by number; a cell's footnote points at it with aria-describedby.
className	string	—	—

12 CHARTS

COMPONENTS · 6 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Chart Frame	Grid	⊕ chartScaffold()	charts/chartSetup.ts
Chart Legend Item	Series 1–8	= ChartLegendItem (+ ChartLegend)	charts/ChartLegend.tsx
Bar Chart	Type	= BarChart	charts/BarChart.tsx
Line Chart	Type	= LineChart	charts/LineChart.tsx
Waterfall Chart	—	= WaterfallChart	charts/WaterfallChart.tsx
Donut Chart	State	= DonutChart	charts/DonutChart.tsx
—	—	+ ChartCanvas	charts/ChartCanvas.tsx
—	—	+ useChartTokens / resolveChartTokens	charts/useChartTokens.ts
—	—	+ direct-label, connector and donut-centre plugins	charts/plugins.ts

Charts are the one group not drawn by this package: they are **Chart.js 4** configured against the Scalar tokens.

`Chart Frame` **is not a component here.** In Figma it is drawn geometry — a 416 × 192 plot area at an origin of 48, 8. With Chart.js the scaffold is scale configuration, so it is expressed as `chartScaffold(tokens, grid, format)` returning Chart.js scale options, and Chart.js owns the geometry responsively. Its `Grid` variant survives as the `grid` prop (`horizontal` · `both` · `none`).

Additions exist because canvas cannot read CSS. `ChartCanvas` owns the Chart.js lifecycle, re-resolving tokens on theme change, and the screen-reader data table. The plugins cover marks Chart.js has no native equivalent for: direct labels, waterfall connectors, and the donut's centre total.

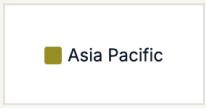
12 · CHARTS

ChartLegendItem

REACT ChartLegendItem FILE src/components/charts/ChartLegend.tsx FIGMA Chart Legend Item
VARIANTS Series 1–8

Chart Legend Item — a colour swatch plus the series name. Render inside a `ChartLegend` (a list).

A legend is mandatory whenever a chart shows two or more series: identity must never rest on colour alone. Where four or fewer series are plotted, also direct-label them on the chart. The swatch is decorative (`aria-hidden`); the text is the content. The legend is a static key, not a toggle.



Storybook · Charts/ChartLegend › Single Item

PROPS

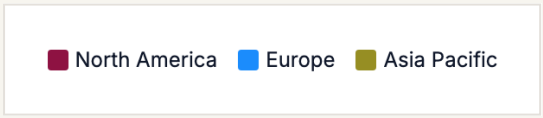
PROP	TYPE	DEFAULT	DESCRIPTION
series required	number	—	Zero-based series index. The ramp is a fixed categorical order.

12 · CHARTS

ChartLegend

REACT ChartLegend FILE src/components/charts/ChartLegend.tsx FIGMA Chart Legend Item
VARIANTS Series 1–8

Chart Legend — the list of legend items under a chart. Named "Legend" unless you pass `aria-label`.



Storybook · Charts/ChartLegend › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labels required	ReactNode[]	—	—
current	number	—	Index of the series currently emphasised on the chart; marked `aria-current`.

12 · CHARTS

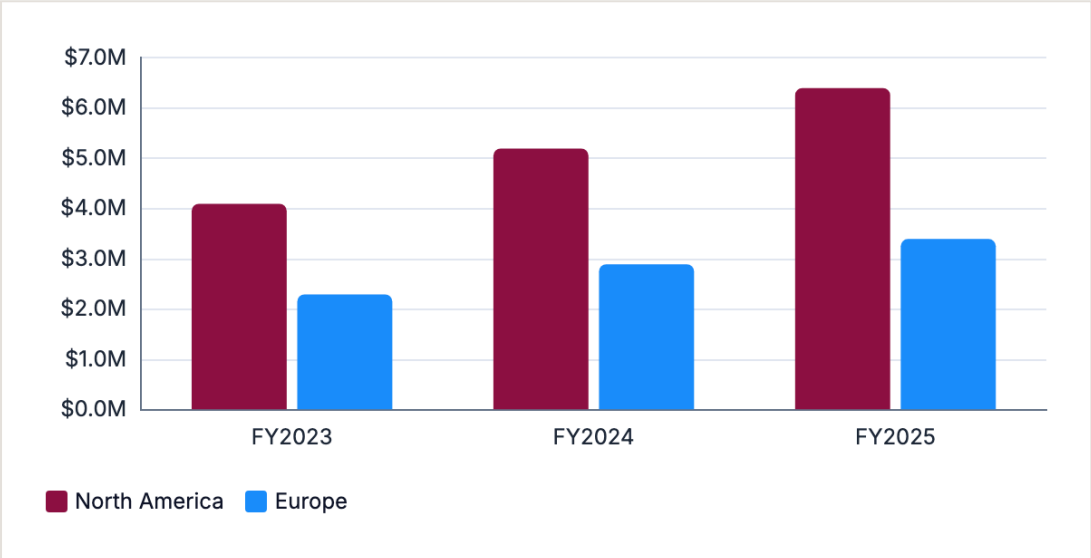
BarChart

REACT BarChart FILE src/components/charts/BarChart.tsx FIGMA Bar Chart VARIANTS Type

Bar Chart — magnitude compared across categories.

Marks follow the house spec: bars anchored to the baseline with 4px rounded tops only, and a 2px surface gap between adjacent bars and between stacked segments.

Accessibility: always ships a legend and a screen-reader data table. With three or more series it also turns on direct labels, so identity never rests on colour alone (the ramp itself is CVD-validated in CI).



Storybook · Charts/BarChart › Grouped

PROPS

PROP	TYPE	DEFAULT › CHARTS/CHARTLEGEND · STORYBOOK	DESCRIPTION PROPS
categories required	`string[]`	—	—
series required	`{ label: string; values: number[]; }[]`	—	One entry per series, each the length of `categories`.
title required	`string`	—	—
type	`"stacked" | "grouped"`	`grouped`	Grouped compares series within each category. Stacked shows composition and a total — use it only when the total is meaningful. If readers need to compare the middle segments, use small multiples: a stacked segment that does not sit on the baseline cannot be compared by eye.
grid	`"none" | "both" | "horizontal"`	`horizontal`	—
format	`((value: number) => string)`	—	—
categoryLabel	`string`	`Category`	The category axis name, used in the screen-reader table.
height	`number`	`240`	—

12 · CHARTS

LineChart

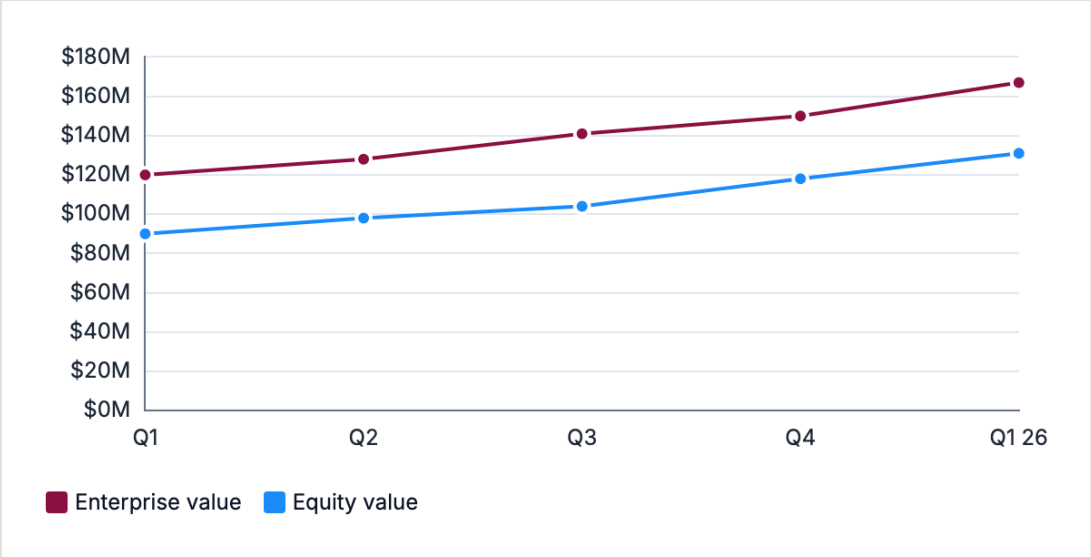
REACT LineChart FILE src/components/charts/LineChart.tsx FIGMA Line Chart VARIANTS Type

Line Chart — change over time.

Marks: 2px strokes with round caps and joins, 8px markers ringed 2px in the surface colour so overlapping points stay separable. Points sit at band centres so they align with the category labels.

Area fills are painted largest first so smaller series are never buried.

Never use two y-scales on one chart: two measures of different magnitude become two charts, or one indexed to a common base.



Storybook · Charts/LineChart › Line

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
categoriesrequired	`string[]`	—	—
seriesrequired	`{ label: string; values: number[]; }[]`	—	—
titlerequired	`string`	—	—
type	`"line" | "area"`	`line`	Area is for a single cumulative quantity or a part-to- whole over time. With more than three overlapping series it becomes unreadable — use Line.
grid	`"none" | "both" | "horizontal"`	`horizontal`	—
format	`((value: number) => string)`	—	—
categoryLabel	`string`	`Category`	—
height	`number`	`240`	—

12 · CHARTS

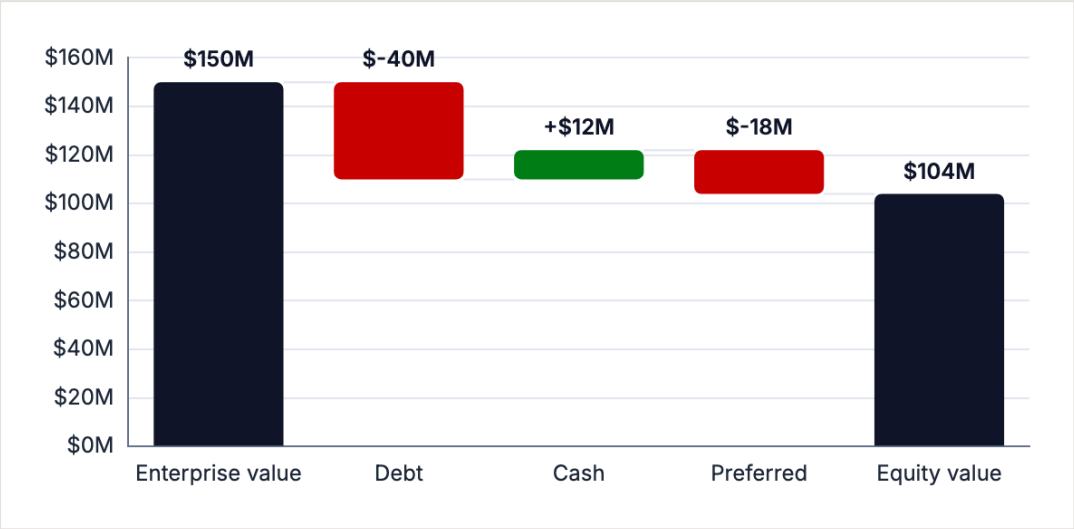
WaterfallChart

REACT WaterfallChart FILE src/components/charts/WaterfallChart.tsx FIGMA Waterfall Chart

Waterfall Chart — how a starting figure becomes an ending figure, step by step.

Totals are anchored to the baseline and rounded on top only, because they touch it. Deltas float and are rounded on all four corners, because they do not. Totals take Chart/Total; deltas take Chart/Positive or Chart/Negative.

Every bar is directly labelled — a waterfall is read as arithmetic, not as a shape, so the numbers are not optional. Connectors carry the running total between steps.



Storybook · Charts/WaterfallChart › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
stepsrequired	`WaterfallStep[]`	—	—
titlerequired	`string`	—	—
format	`((value: number) => string)`	—	—
height	`number`	`260`	—

12 · CHARTS

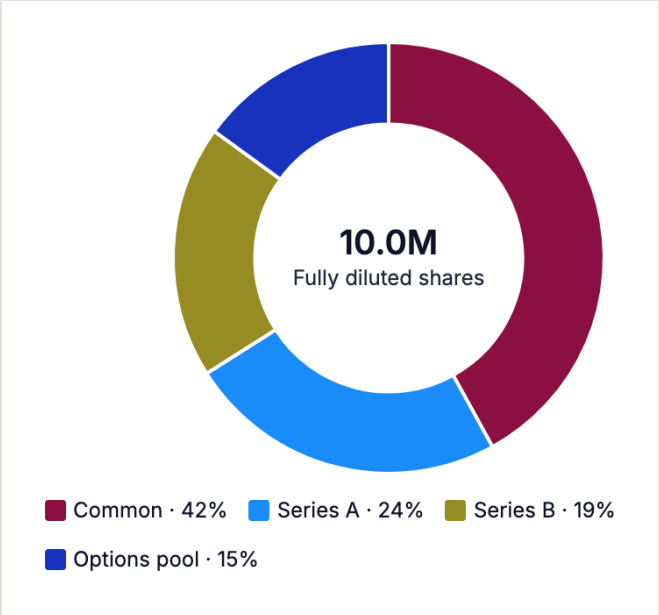
DonutChart

REACT DonutChart FILE src/components/charts/DonutChart.tsx FIGMA Donut Chart VARIANTS State

Donut Chart — part-to-whole for a single composition, with the total held in the middle.

A donut answers "what is the split". It cannot answer "how did the split change" — that is a stacked bar over time.

Segments are separated by a 2px surface gap, and every slice is labelled with its share in the legend, so identity never rests on colour.



Storybook · Charts/DonutChart › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
slicesrequired	`DonutSlice[]`	—	Eight slices is the ceiling. Past that, sort descending and fold the tail.
titlerequired	`string`	—	—
total	`string`	—	The figure held in the middle.
caption	`string`	—	—
focusedIndex	`number`	—	Keeps one series at full strength and drops the rest to their Subtle steps, for hover and for drill-down.
format	`((value: number) => string)`	—	Formats the Value column of the screen-reader table. Defaults to the raw number.
height	`number`	`260`	—

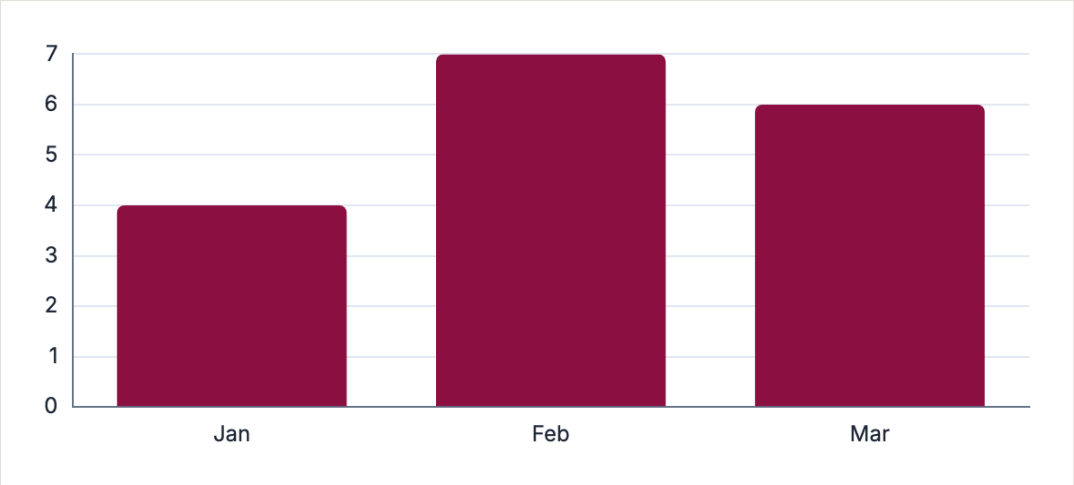
12 · CHARTS

ChartCanvas

REACT ChartCanvas FILE src/components/charts/ChartCanvas.tsx

The canvas shell every Scalar chart is drawn in.

Renders through `react-chartjs-2`, which owns the Chart.js lifecycle (create, in-place update on data/option change, destroy on unmount). This shell adds what the binding does not: re-resolving tokens when the theme changes (the chart is recreated, keyed on the token object) and the screen-reader fallback.



Storybook · Charts/ChartCanvas › Custom Chart

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
build required	`(tokens: ChartTokens) => ChartConfiguration<any, any[], unknown>`	—	Builds the Chart.js config from resolved tokens. Called on every render; the chart is updated in place whenever the data or options it returns change, and recreated only when the theme tokens or chart type change.
title required	`string`	—	The accessible name of the chart. A canvas is opaque to assistive technology, so this is the only thing a screen reader gets from the drawing itself.
table	`ChartDataTable`	—	The same numbers as a table, rendered visually hidden beside the canvas and referenced from the canvas with `aria-describedby`. Without it the data is unreachable without sight. Charts in this package always supply one.
height	`number`	`240`	—

13 SUMMARY CARD

COMPONENTS · 2 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Card	—	= `Card`	`card/Card.tsx`
Card_item	—	= `CardItem`	`card/Card.tsx`

13 · SUMMARY CARD

Card

REACT Card FILE src/components/card/Card.tsx FIGMA Card

Card — a summary panel for one company or fund.

The card is a surface, not a button. If the whole card is clickable, the affordance still needs a named action inside it. With a `title` the card is a named region (`aria-labelledby` the heading). `asChild` renders the single child element (e.g. an `<article>` or `<li>`) as the card root, with the header and item list placed inside it.



Storybook · Core/Card › Default

PROPS

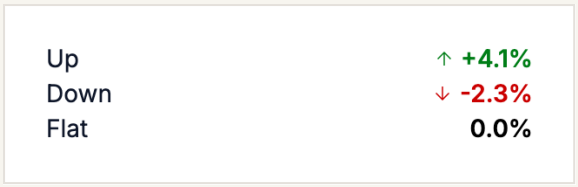
PROP	TYPE	DEFAULT	DESCRIPTION
`title`	`ReactNode`	—	—
`headingLevel`	`1` | `2` | `3` | `4` | `5` | `6`	`3`	The title's heading level (`h1`–`h6`). Default 3.
`tag`	`ReactNode`	—	A Chip or Badge sitting beside the title.
`action`	`ReactNode`	—	The named action. A card is a surface, not a button.
`className`	`string`	—	—
`asChild`	`boolean`	`false`	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

13 · SUMMARY CARD

CardItem

REACT CardItem FILE src/components/card/Card.tsx FIGMA Card_item

Card_item — one label-and-value row inside Card.



Storybook · Core/Card › Item Trends

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`**required**	`ReactNode`	—	—
`value`**required**	`ReactNode`	—	—
`trend`	`"up"` | `"down"`	—	Direction of change. Colour never carries the direction on its own — the arrow glyph and the sign do (rule R8). The arrow is decorative; the direction is also announced as visually hidden text (`trendLabel`).
`trendLabel`	`string`	—	Spoken direction. Defaults to "Increase" / "Decrease" — override to localise.
`className`	`string`	—	—

14 COMPONENTS · 4 COMPONENTS FEEDBACK

FIGMA	VARIANTS	REACT	FILE
Alert	Style	= `Alert`	`feedback/Alert.tsx`
Toast	Style × State	= `Toast` (+ + `ToastViewport`)	`feedback/Toast.tsx`
Progress Bar	Type	= `ProgressBar`	`feedback/ProgressBar.tsx`
Skeleton	Type	= `Skeleton`	`feedback/Skeleton.tsx`

14 · FEEDBACK

Alert

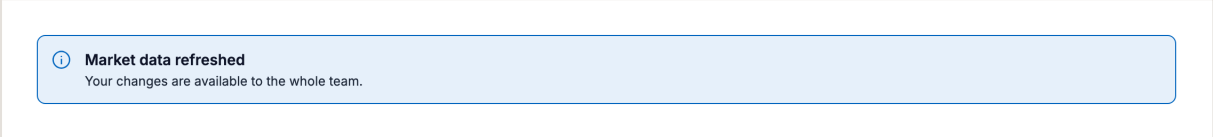
REACT Alert FILE src/components/feedback/Alert.tsx FIGMA Alert VARIANTS Style

Alert — a message about the state of the page or the task, shown inline.

Alert is inline and stays until the condition changes. Toast is transient and floats. If the user must act on it, it is an Alert; if it is an acknowledgement they can miss, it is a Toast.

Say what happened and what to do next. "Import failed" is half a message.

Accessibility: `negative` is `role="alert"` (assertive); the rest are `role="status"` (polite). Override with the `role` prop. The glyph is decorative; the tone is conveyed by the glyph shape and the words.



Storybook · Feedback/Alert › Info

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tone	`"positive"` | `"warning"` | `"negative"` | `"info"`	—	The kind of message. Default `info`.
style	`CSSProperties | AlertTone`	—	@deprecated Use `tone`. A string value is read as the tone for backwards compatibility; a `CSSProperties` object is the normal HTML `style`.
title	`ReactNode`	—	—
icon	`ReactNode`	—	Overrides the default glyph. The icon is never optional.
actions	`ReactNode`	—	Trailing actions — usually one Button.
onDismiss	`(() => void)`	—	Renders a dismiss button that calls this.
dismissLabel	`string`	`Dismiss`	Accessible name of the dismiss button. Default "Dismiss".

14 · FEEDBACK

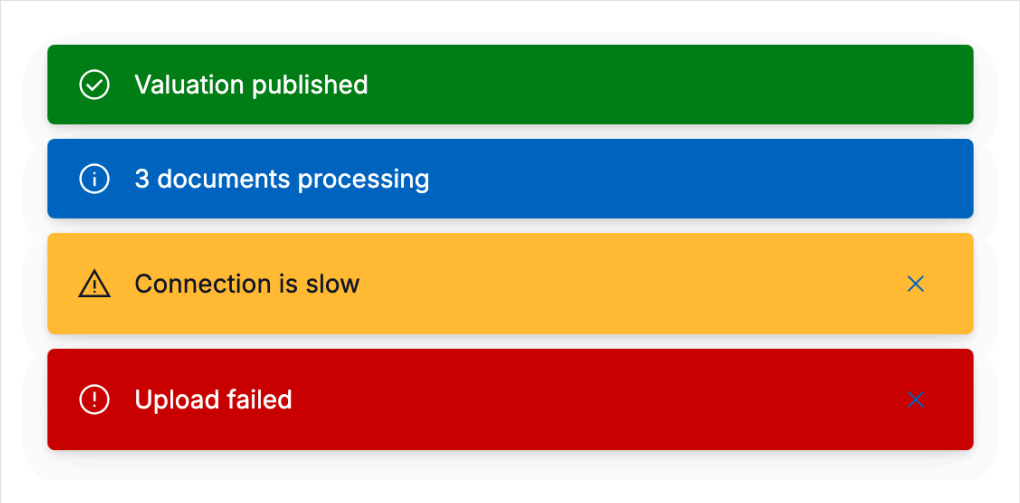
Toast

REACT Toast FILE src/components/feedback/Toast.tsx FIGMA Toast VARIANTS Style × State

Toast — a transient, system-initiated message.

A toast reports something that already happened. It never asks a question and never holds the only copy of information the user needs — it disappears.

Accessibility: pair the colour with an icon and words (rule R8). `negative` toasts are `role="alert"`, the rest `role="status"`. For reliable announcement mount the toasts inside a `ToastViewport`, which is a live region that exists before they arrive. Hover exists so the dismiss timer can pause — keep that behaviour when you wire the timer (the library ships no timer).



Storybook · Feedback/Alert › Toast

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`tone`	`"positive" | "warning" | "negative" | "info"`	—	The kind of message. Default `positive`.
`style`	`CSSProperties | ToastTone`	—	@deprecated Use `tone`. A string value is read as the tone for backwards compatibility; a `CSSProperties` object is the normal HTML `style`.
`icon`	`ReactNode`	—	—
`onDismiss`	`(() => void)`	—	Renders a dismiss button that calls this.
`dismissLabel`	`string`	`Dismiss`	Accessible name of the dismiss button. Default "Dismiss".

14 · FEEDBACK

ToastViewport

REACT ToastViewport FILE src/components/feedback/Toast.tsx FIGMA Toast VARIANTS Style × State

Fixed-position stack for toasts. Render one per application. It is the persistent live region (`aria-live="polite"`, additions only) that makes toasts mounted later get announced. Override the name with `aria-label`.

PROPS

No component-specific props; accepts the native element attributes.

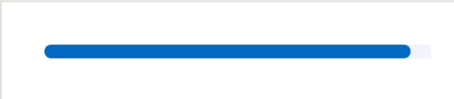
14 · FEEDBACK

ProgressBar

REACT ProgressBar FILE src/components/feedback/ProgressBar.tsx FIGMA Progress Bar VARIANTS Type

Progress Bar — how far along a task is.

`role="progressbar"` with `aria-valuemin/max/now`; indeterminate omits `aria-valuenow`. The sliding animation is switched off under prefers-reduced-motion.



Storybook · Feedback/Alert › ProgressBar

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
value	number	—	Percentage complete, 0–100. Omit it for an indeterminate bar. Use determinate whenever the total is known. A fake determinate bar that stalls at 90% destroys trust faster than an honest indeterminate one.
label	string	—	—

14 · FEEDBACK

Skeleton

REACT Skeleton FILE src/components/feedback/Skeleton.tsx FIGMA Skeleton VARIANTS Type

Skeleton — the shape of content that has not loaded yet.

A skeleton must match the layout it replaces — same line count, same widths, same positions. If the real content lands somewhere else, the page jumps and the skeleton has made things worse.

Use only for first loads of a known shape. For a refresh of content already on screen, keep the old content and show progress instead of blanking it.

Under 300ms, show nothing at all — a flash of skeleton reads as a glitch.

Accessibility: each block is `aria-hidden`, so it is silent on its own. Put `aria-busy="true"` on the region that is loading (and a visually hidden "Loading…" status, or use `SkeletonGroup`) so screen-reader users know. The pulse is switched off under prefers-reduced-motion.

Storybook · Feedback/Alert › Skeleton

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type	"title" | "text" | "circle" | "block"	text	—
width	string | number	—	—
height	string | number	—	—

14 · FEEDBACK

SkeletonGroup

REACT SkeletonGroup FILE src/components/feedback/Skeleton.tsx

Skeleton Group — wraps several skeletons in one `aria-busy` region with a single polite "Loading..." announcement. Replace it with the real content when the data arrives.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
label	string	Loading…	Text announced while loading. Default "Loading...".

15 ACCORDION & DRAWER

COMPONENTS · 6 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Accordion Item	State	= `AccordionItem`	`disclosure/Accordion.tsx`
Accordion	—	= `Accordion`	`disclosure/Accordion.tsx`
Drawer	Side	= `Drawer`	`disclosure/Drawer.tsx`
Workspace Drawer	State	= `WorkspaceDrawer` (sticky to the viewport bottom)	`disclosure/WorkspaceDrawer.tsx`
Workspace Drawer Tab	State	= `WorkspaceDrawerTab`	`disclosure/WorkspaceDrawer.tsx`
Data Review Card	State	= `DataReviewCard`	`disclosure/DataReviewCard.tsx`

15 · ACCORDION & DRAWER

AccordionItem

REACT AccordionItem FILE src/components/disclosure/Accordion.tsx FIGMA Accordion Item VARIANTS State

Accordion Item — one collapsible section.

The whole header row is the target, not just the chevron, and it carries the 44px minimum.

Accessibility: the header is a button inside a heading (`headingLevel`), with `aria-expanded` and `aria-controls` (the latter only while the panel exists).

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
open	boolean	—	Controlled open state. Wins over the parent Accordion's `value`.
defaultOpen	boolean	false	—
onOpenChange	((open: boolean) => void)	—	—
disabled	boolean	—	—
value	string	—	Identifies the item to a parent `Accordion` that is controlling (or limiting to one) the open sections. Ignored when the item is used on its own.
headingLevel	1 | 2 | 3 | 4 | 5 | 6	3	Level of the heading that wraps the header button, so it joins the outline. Default 3.
keepMounted	boolean	false	Keep the panel in the DOM (hidden) while closed. Default false: it unmounts.
panelRegion	boolean	true	Expose the open panel as a `region` landmark labelled by its header. Default true; turn it off in long accordions, where a dozen landmarks are noise.

15 · ACCORDION & DRAWER

Accordion

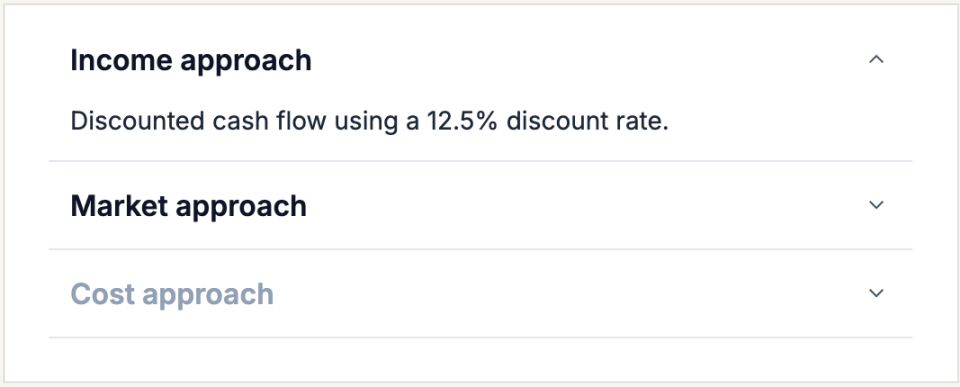
REACT Accordion FILE src/components/disclosure/Accordion.tsx FIGMA Accordion

Accordion — a stack of sections the reader opens as needed.

Use for secondary detail on a dense page — assumption notes, methodology, audit history. Never hide anything required to complete the task behind a collapsed section.

Allow more than one section open at a time unless space genuinely forbids it; auto-closing the previous section steals content the reader may still be using. If every section ends up open, the content wanted a page.

Keyboard: with focus on a header, ↑/↓ move to the previous/next header and Home/End to the first/last (APG accordion). Tab still visits every header. Pass `value` on the items and `type` / `value` / `defaultValue` here to coordinate them; plain items keep their own state.



Storybook · Overlays/Accordion › Multiple

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type	"multiple" | "single"	multiple	multiple (default) lets any number of sections be open; single keeps one open at a time. Only items that carry a value take part.
value	string[]	—	Controlled list of open item values.
defaultValue	string[]	[]	—
onValueChange	((value: string[]) => void)	—	—

15 · ACCORDION & DRAWER

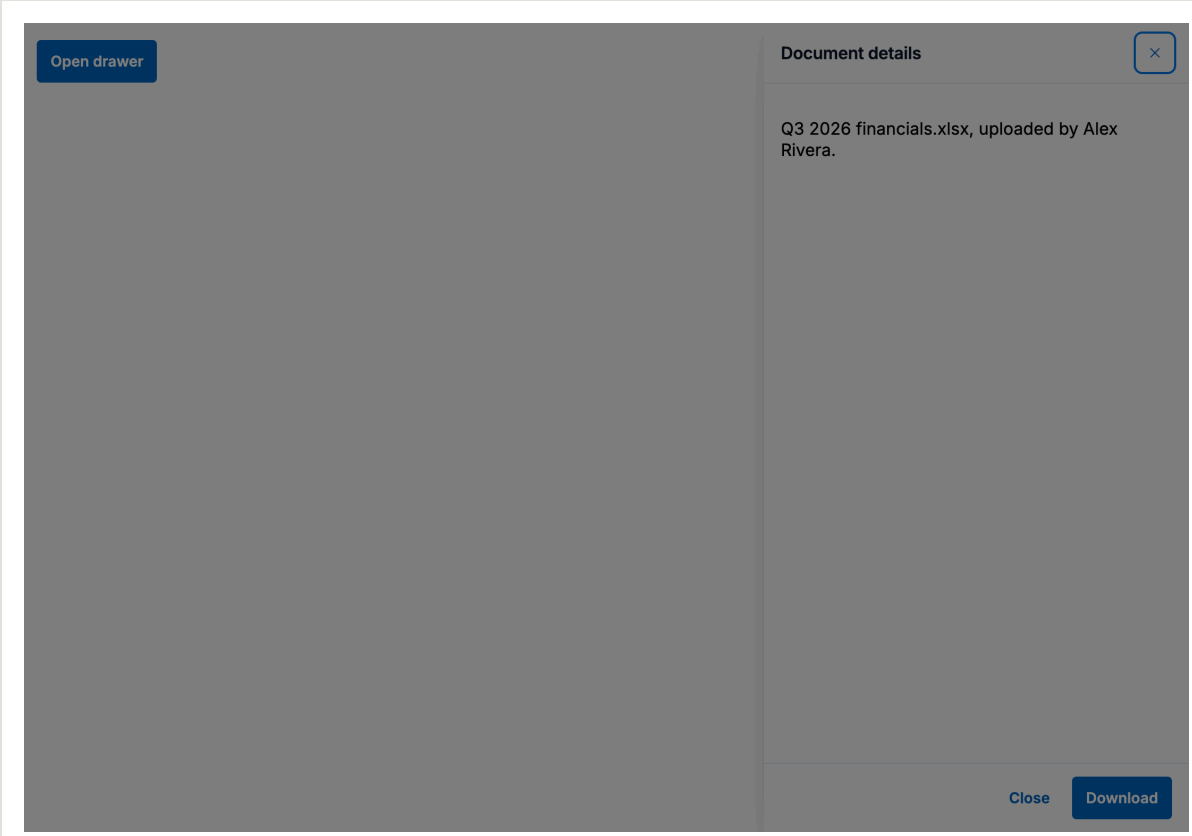
Drawer

REACT Drawer FILE src/components/disclosure/Drawer.tsx FIGMA Drawer VARIANTS Side

Drawer — a panel that slides in over the page for a focused task, without losing the context behind it.

A drawer keeps the page behind visible on purpose. If the task needs the user's whole attention, or must be finished before anything else, use a Modal instead.

Accessibility: `role="dialog"` labelled by its title. On open, focus moves into the drawer; Escape closes it; on close focus returns to whatever opened it (`restoreFocus`). With `modal` (default) Tab is trapped; with `modal={false}` there is no scrim and Tab and outside clicks leave it.



Storybook · Overlays/Accordion › Drawer

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
open required	`boolean`	—	—
onClose required	`() => void`	—	—
title	`ReactNode`	—	—
footer	`ReactNode`	—	Real Button instances above a divider.
side	`"bottom" | "left" | "right"`	`right`	Right is the default, for detail and edit flows. Left is for navigation. Bottom is for narrow viewports.
modal	`boolean`	`true`	Modal (default): a scrim covers the page, Tab is trapped inside, the page does not scroll and `aria- modal` is true. Non-modal: no scrim, Tab can leave the drawer and a click outside closes it.
initialFocus	`RefObject<HTMLElement | null>`	—	Element to focus on open. Defaults to the first tabbable control (the Close button).
triggerRef	`RefObject<HTMLElement | null>`	—	The control that opened the drawer, so a click on it is not an "outside" click.
restoreFocus	`boolean`	—	Return focus to the opener on close. Default true.

15 · ACCORDION & DRAWER

WorkspaceDrawer

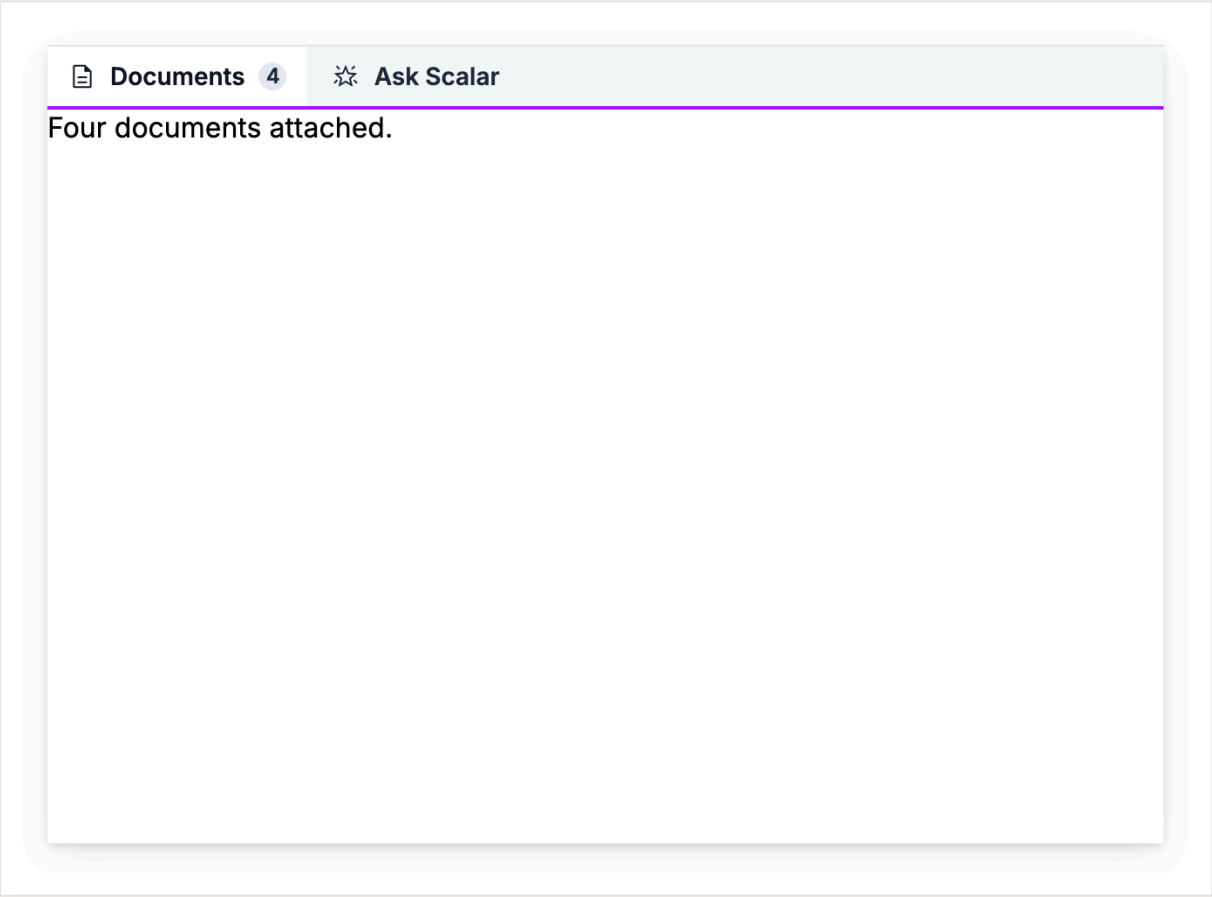
REACT WorkspaceDrawer FILE src/components/disclosure/WorkspaceDrawer.tsx FIGMA Workspace Drawer
VARIANTS State

Workspace Drawer — the docked panel at the foot of a data page, where documents, notes and model output are worked through without leaving the table above.

It is sticky to the bottom of the viewport (or of its nearest scroll container): render it as the last child of the page's scrolling column and the content above scrolls behind it. 40vh, or 60vh when `expanded`.

Notes, Sheets and Documents are content slots: swap the children for that tab's own view rather than forking the drawer.

Accessibility: a labelled region; the tab strip is a `tablist` with roving focus (←/→, Home/End move focus; Enter/Space activates, via the tab's `onClick`). The body is a `tabpanel` labelled by the active tab. It does not trap focus or take it on mount — it is docked page chrome, not an overlay.



Storybook · Overlays/Accordion › WorkspaceDrawer

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tabs	ReactNode	—	WorkspaceDrawerTab instances. Exactly one is active.
expanded	boolean	—	Expanded trades page height for drawer height.
docked	boolean	—	Docked: the tab strip only, with no content area — the resting state at the foot of a page (Figma "Workspace Drawer · docked"). Opening a tab swaps it for the open drawer.
tabsLabel	string	Workspace	Accessible name of the tab strip. Default "Workspace". The drawer itself is named by `aria- label` (default "Workspace").

15 · ACCORDION & DRAWER

WorkspaceDrawerTab

REACT WorkspaceDrawerTab FILE src/components/disclosure/WorkspaceDrawer.tsx FIGMA Workspace Drawer Tab
VARIANTS State

Workspace Drawer Tab — one tab in the docked workspace drawer.

Inside a `WorkspaceDrawer` it is a roving tab stop (only the active tab is in the Tab order; arrows move between tabs) and the active tab controls the drawer's tabpanel.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	ReactNode	—	—
icon	ReactNode	—	—
count	number	—	A queue length, not a notification. Hide it when nothing is waiting.
active	boolean	—	—
ai	boolean	—	Reserved for a tab whose content is model-generated. The one place in the drawer chrome that carries the solid Background/AI, so the purple always means "a model produced this".

15 · ACCORDION & DRAWER

DataReviewCard

REACT DataReviewCard FILE src/components/disclosure/DataReviewCard.tsx FIGMA Data Review Card
VARIANTS State

Data Review Card — one model-extracted value waiting on a human decision.

The card never asserts a value on its own: it shows what was extracted, the document and sentence it came from, and why. Accept and reject live in the title row so the decision is always one click from the evidence.

Accept and reject are Secondary icon buttons: the system has no soft-filled icon button, so positive and negative read as a coloured outline rather than the green and red fills used elsewhere.

Accessibility: an `article`; selection is a toggle button (`aria-pressed`), not `aria-selected`, which is invalid on an article.



Storybook · Overlays/Accordion › DataReviewCard

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
sourceFile	ReactNode	—	The document the value came from.
quote	ReactNode	—	The sentence the value was extracted from.
rationale	ReactNode	—	Why the model believes the value.
selected	boolean	—	Whether this card is the one being inspected. Controlled only: the parent owns which card is selected. Exposed as `aria-pressed` on the title button.
onSelect	(() => void)	—	Makes the card selectable. The title becomes a real button (Enter / Space) whose hit area stretches over the whole card, so the mouse target is unchanged and the keyboard has a way in.
onAccept	(() => void)	—	—

PROP	TYPE	DEFAULT	DESCRIPTION
onReject	(() => void)	—	—
actions	ReactNode	—	Extra controls rendered after Accept / Reject in the title row.

16 MODALS

COMPONENTS · 4 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Column Item	State	= `ColumnItem`	`modals/ColumnPicker.tsx`
Column Title	State	= `ColumnTitle`	`modals/ColumnPicker.tsx`
Search	State	= `ModalSearch`	`modals/ColumnPicker.tsx`
Add_Column_Modal	State	⊕ `Modal` + the three above	`modals/Modal.tsx`
—	—	+ `Modal` (the reusable shell; `size` s · m · l · xl)	`modals/Modal.tsx`

Note. `Add_Column_Modal` is an assembled screen in Figma, composed from Column Title, Column Item, Search and Button. In code it is that same composition — build it from `Modal` plus those three, rather than importing a one-off. The `Modal` shell is the reusable part and is new in the code layer.

16 · MODALS

ColumnItem

REACT ColumnItem FILE src/components/modals/ColumnPicker.tsx FIGMA Column Item VARIANTS State

Column Item — a selectable column in the Add Column modal.

It is a toggle button (`aria-pressed`), not a listbox option: several can be picked at once and each is its own tab stop, so no arrow-key model is owed.

PROPS

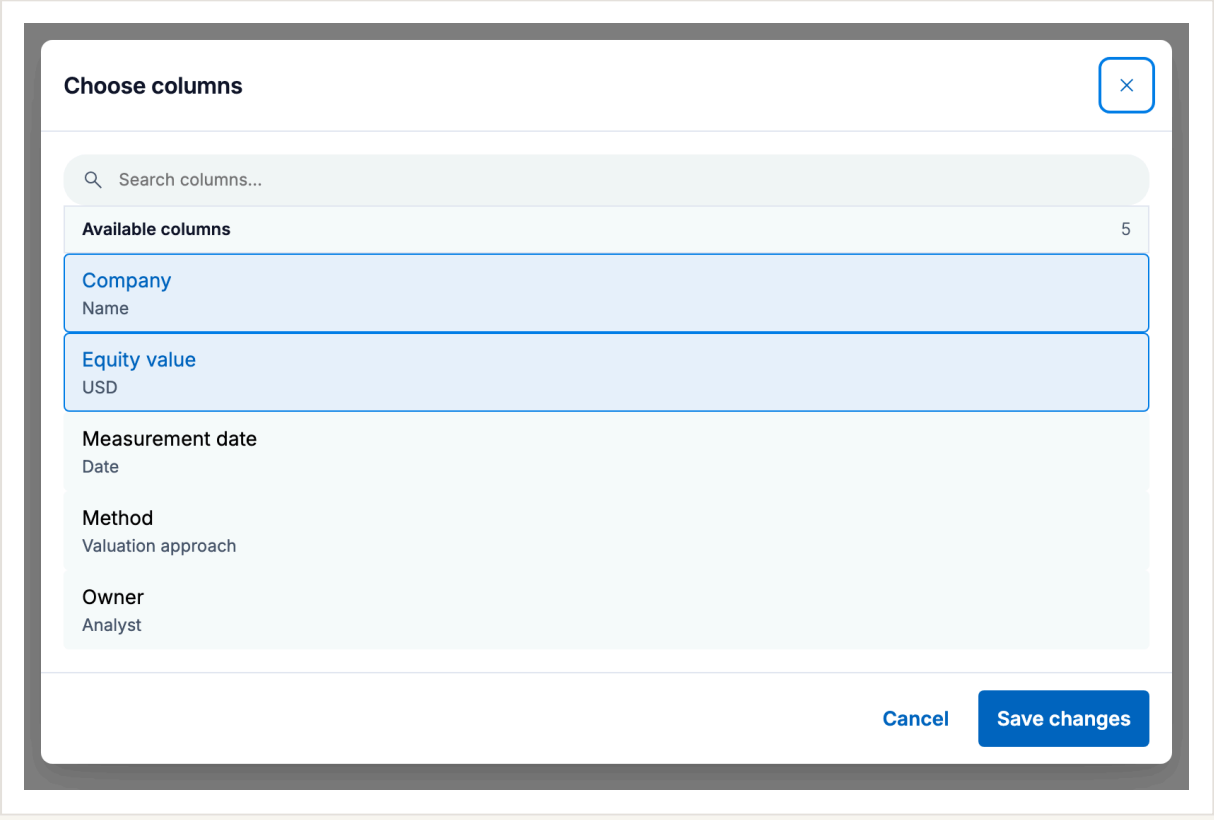
PROP	TYPE	DEFAULT	DESCRIPTION
`label`required	ReactNode	—	—
`subText`	ReactNode	—	The unit or type, e.g. "Currency".
`selected`	boolean	—	Whether the column is picked. Exposed as `aria-pressed`.

16 · MODALS

ColumnTitle

REACT ColumnTitle FILE src/components/modals/ColumnPicker.tsx FIGMA Column Title VARIANTS State

Column Title — the header cell of a selectable column list (a group of `ColumnItem` toggles).



Storybook · Overlays/Modal › Column Picker

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`title`required	ReactNode	—	—
`count`	ReactNode	—	e.g. "15 items".

16 · MODALS

ModalSearch

REACT ModalSearch FILE src/components/modals/ColumnPicker.tsx FIGMA Search VARIANTS State

Search — the search field inside a modal.

Search filters in place; it does not submit. If a query needs submitting, pair it with a Button. The input is named "Search" unless you pass an `aria-label` / `aria-labelledby`; `ref` points at the input.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`className`	`string`	—	Class for the wrapper.

16 · MODALS

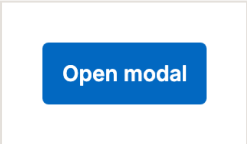
Modal

REACT Modal FILE src/components/modals/Modal.tsx FIGMA Add_Column_Modal VARIANTS State

Modal — the dialog shell.

Accessibility: a modal traps focus, closes on Escape (top-most only), locks page scroll and returns focus to whatever opened it. `onClose` may be an inline function; changing it never moves focus. The scrim is a sibling, never a child. `ref`, `id`, `aria-*` and `data-*` land on the dialog element.

Use a modal when the task needs the user's whole attention or must be finished before anything else. Otherwise use a Drawer, which keeps the page behind visible.



Storybook · Overlays/Modal › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
open required	`boolean`	—	—
onClose required	`() => void`	—	—
title	`ReactNode`	—	The heading. It names the dialog (`aria-labelledby`), whatever node it is.
footer	`ReactNode`	—	—
dismissOnScrimClick	`boolean`	`true`	Whether clicking the scrim dismisses. Turn it off when state is unsaved.
size	`"s"` | `"m"` | `"l"` | `"xl"`	`m`	Width. `s` for a short confirm-style form, `m` (default) for most tasks, `l` for a two-column form or a small table, `xl` for a grid or a document preview.
role	`"alertdialog"` | `"dialog"`	`dialog`	`alertdialog` for a confirmation that interrupts the user. Default `dialog`.
initialFocus	`RefObject<HTMLElement | null>`	—	What receives focus on open. Default: the first tabbable control.

17

COMPONENTS · 5 COMPONENTS

GLOBAL SEARCH

FIGMA	VARIANTS	REACT	FILE
Global Search	State	= `GlobalSearch`	`search/GlobalSearch.tsx`
Search Result Row	Type × State (27)	= `SearchResultRow`	`search/SearchResultRow.tsx`
Search Section Header	Type (9)	= `SearchSectionHeader`	`search/SearchSectionHeader.tsx`
Search Scope Chip	Type (5)	= `SearchScopeChip`	`search/SearchScopeChip.tsx`
Key Hint	—	= `KeyHint`	`search/KeyHint.tsx`

17 · GLOBAL SEARCH

GlobalSearch

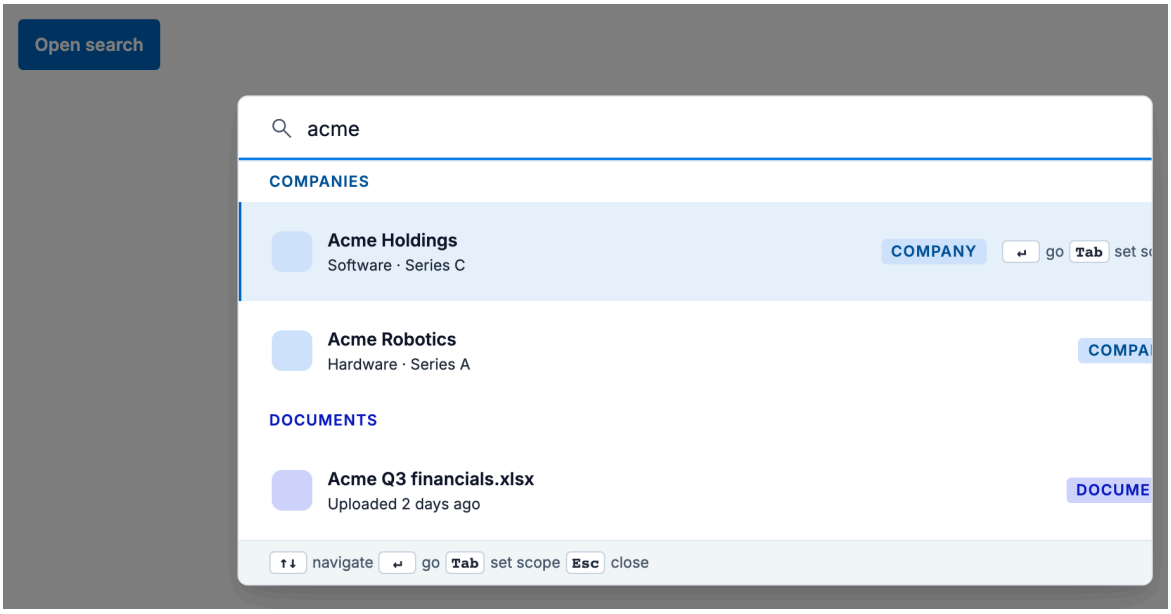
REACT GlobalSearch FILE src/components/search/GlobalSearch.tsx FIGMA Global Search VARIANTS State

Global Search — the command palette for the whole product.

One overlay, opened from anywhere with ⌘K, that searches every entity the user can reach and runs commands against them. It is not a filter and not a page search: it crosses firms, companies, documents, versions and settings in a single list.

Scope is a stack, not a filter. Tab pushes the highlighted result onto the scope, Backspace pops it. The chips in the bar are that stack, outermost first. Do not show more than three: past Firm → Company → Document the user has lost the thread.

Accessibility: a modal `dialog` (focus is trapped, the page does not scroll). Focus moves to the search input on open and returns to whatever opened the palette on close. The input is a labelled `combobox`; ↑↓ move the highlight without moving focus out of it (`aria-activedescendant`), and the highlighted row scrolls into view. Result count is announced through a polite live region. Escape closes from any scope depth. `open` and `query` are controlled.



Storybook · Navigation/GlobalSearch › Interactive

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
open required	`boolean`	—	—
onClose required	`() => void`	—	—
query required	`string`	—	—
onQueryChange required	`(query: string) => void`	—	—

PROP	TYPE	DEFAULT	DESCRIPTION
resultsrequired	`SearchResult[]`	—	Results in relevance order. They are grouped into contiguous runs by type in the order they arrive — never re-sorted alphabetically.
scopes	`SearchScope[]`	`[]`	The scope stack, outermost first.
onScopesChange	`((scopes: SearchScope[]) => void)`	—	—
onSelect	`((result: SearchResult) => void)`	—	—
footer	`ReactNode`	—	Shown in the footer, e.g. the current firm.
renderResult	`((result: SearchResult, state: { id: string; selected: boolean; select: () => void; }) => ReactNode)`	—	Replaces the default result row. Render a `SearchResultRow` (or your own option) and spread `id` onto it so `aria-activedescendant` can find it.
aria-label	`string`	`Global search`	Accessible name of the dialog. Default "Global search".

17 · GLOBAL SEARCH

SearchResultRow

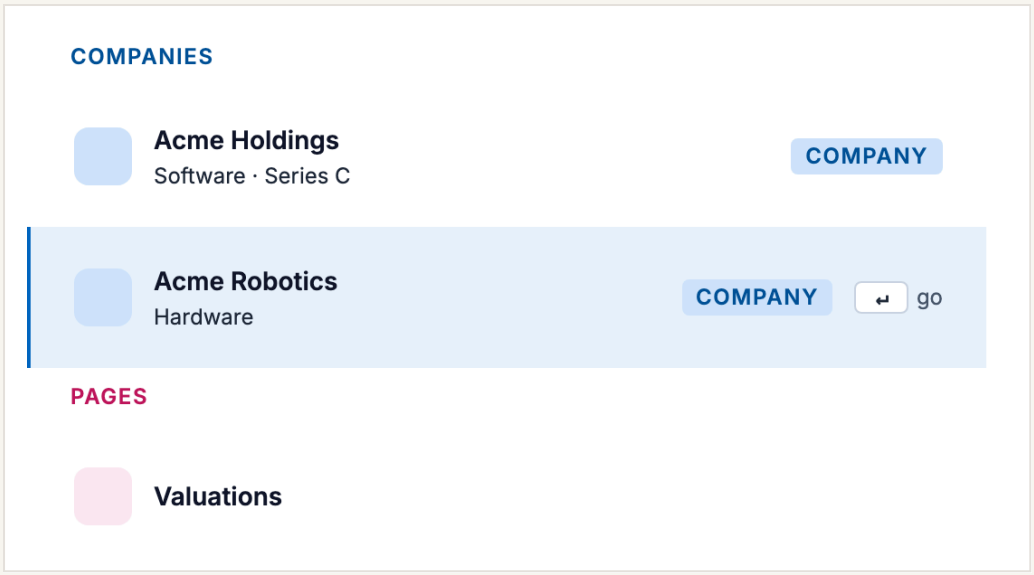
REACT SearchResultRow FILE src/components/search/SearchResultRow.tsx FIGMA Search Result Row

Variants Type × State (27)

Search Result Row — one hit inside Global Search.

Semantic colour states what the result is. It never states whether the user may open it.

Accessibility: an option in the Global Search listbox. Focus stays in the input (`aria-activedescendant`), so the row is a `div` with `tabIndex={-1}` — clickable, not a tab stop. The row is 62px and clears Target/Minimum. Selection is carried by the 2px Stroke/Brand edge and the visible key hints as well as the fill, so it survives greyscale and colour-vision deficiency (rule R8).



Storybook · Navigation/GlobalSearch › Result Rows

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
typerequired	`"document"` | `"page"` | `"version"` | `"neutral"` | `"firm"` | `"company"` | `"measurement-date"` | `"firm-action"` | `"company-action"`	—	The PRISM contract. Moves as one piece: it sets the tile fill, the glyph tint and the badge together. Never mix them — a row tinted Entity/Company carrying a DOCUMENT badge tells the user something false.
titlerequired	`ReactNode`	—	—
subtitle	`ReactNode`	—	—
icon	`ReactNode`	—	The glyph inside the tile. Swap it freely; the type keeps the colour.
selected	`boolean`	—	—
hints	`ReactNode`	—	Key hints shown on the selected row.
hideBadge	`boolean`	—	Hides the type badge. The badge is what keeps the colour honest.

17 · GLOBAL SEARCH

SearchSectionHeader

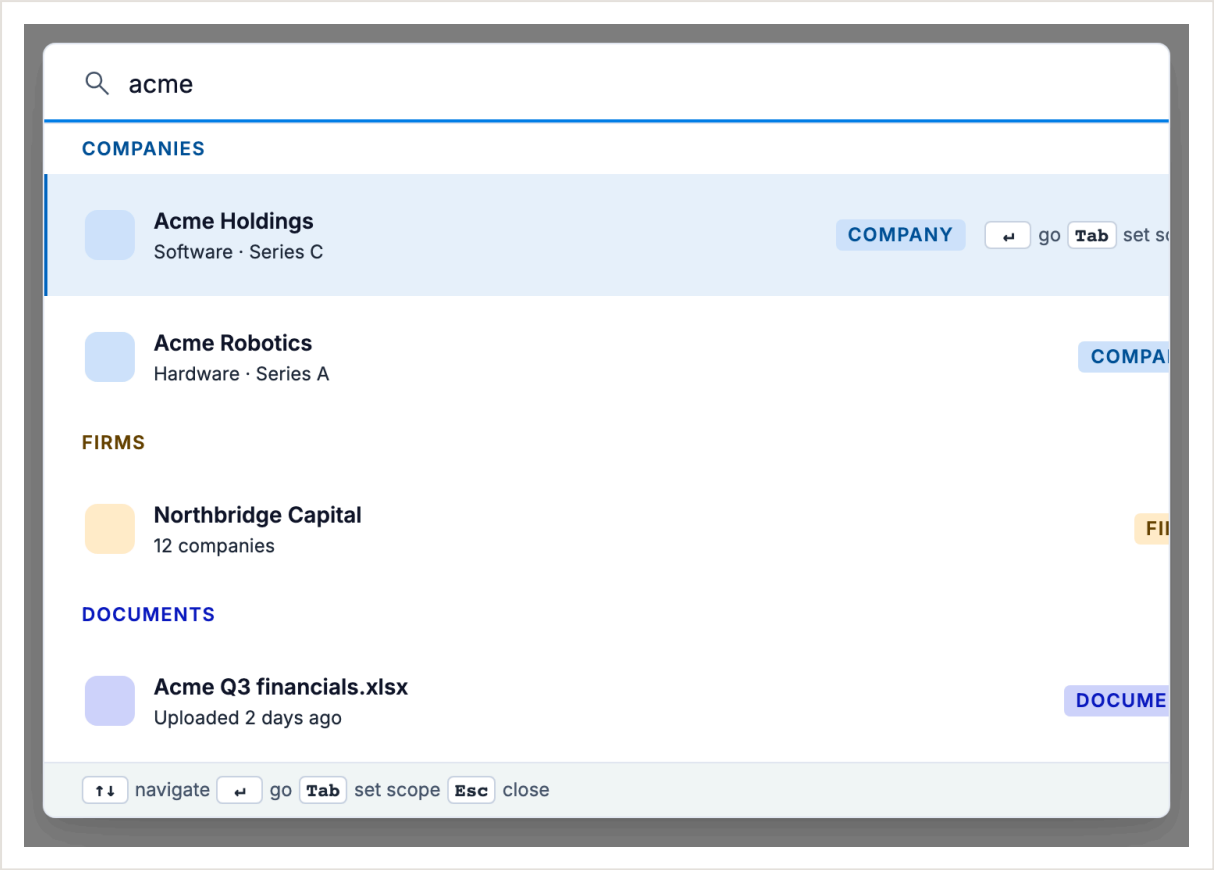
REACT SearchSectionHeader FILE src/components/search/SearchSectionHeader.tsx FIGMA Search Section Header

Variants Type (9)

Search Section Header — the group label above a run of results.

One header per contiguous run of a single type. Order runs by relevance, not alphabetically, and never interleave types beneath one header. The header's type must match the type of every row under it.

Accessibility: give it an `id` and point the group's `aria-labelledby` at it — this label is the accessible name of the group. Keep it a plain plural noun — "COMPANIES", not "3 RESULTS".



Storybook · Navigation/GlobalSearch › With Results

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type required	`"document"` | `"page"` | `"version"` | `"neutral"` | `"firm"` | `"company"` | `"measurement-date"` | `"firm-action"` | `"company-action"`	—	—

17 · GLOBAL SEARCH

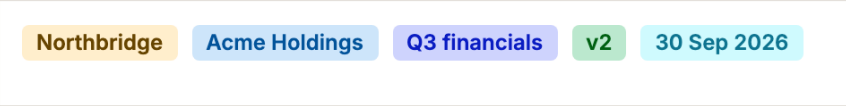
SearchScopeChip

REACT SearchScopeChip FILE src/components/search/SearchScopeChip.tsx FIGMA Search Scope Chip
VARIANTS Type (5)

Search Scope Chip — one level of the Global Search scope stack.

Chips read left to right, outermost scope first. They are added only by Tab and removed only by Backspace: a breadcrumb of where the search is pointed, not a filter control the user clicks.

Accessibility: the chip is not a button and takes no focus. A visually hidden prefix ("Scoped to Company:") tells a screen reader what the chip means. If it is ever made click-removable, the target must reach Target/Minimum.



Storybook · Navigation/GlobalSearch › Scope Chips

PROPS

PROP	TYPE	DEFAULTSTORYBOOK · NAVIGATION/GLOBALSEARCH › WITH RESULTS	DESCRIPTIONPROPS
typerequired	`"document"` | `"version"` | `"firm"` | `"company"` | `"measurement-date"`	—	—
srLabel	`string`	—	Screen-reader prefix that makes the scope audible. Default "Scoped to {Type}:" (e.g. "Scoped to Company:").

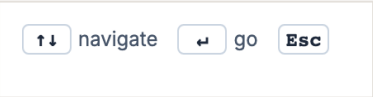
17 · GLOBAL SEARCH

KeyHint

REACT KeyHint FILE src/components/search/KeyHint.tsx FIGMA Key Hint

Key Hint — a keycap paired with the action it performs.

Accessibility: decorative, and hidden from assistive technology. The shortcut it advertises must also exist somewhere the keyboard and a screen reader can reach — a hint that renders only on hover is not documentation (rule R8).



Storybook · Navigation/GlobalSearch › Key Hints

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
keyGlyphrequired	`string`	—	The key, as its real glyph — ↑↓ ↵ Tab ⌫ Esc. Never a spelled-out name.
label	`string`	—	A lower-case verb phrase: "go", "set scope".

18 TEMPLATE

COMPONENTS · NO COMPONENTS

`Page Template · Data Sheet` is an assembled page, not a component. Build it from `PrimaryMenu`, `CompanyInfo`, `SecondaryMenu`, `TertiaryMenu` and `DataGrid`.

20 MENUS & ACTIONS

COMPONENTS · 11 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Menu Item	State × Tone (7)	= `MenuItem` (+ + `MenuDivider`, + `MenuSubItems`)	`actions-menus/Menu.tsx`
Context Menu	Header (2)	= `ContextMenu` (`heading` prop)	`actions-menus/Menu.tsx`
User Menu	Firm Settings Expanded (2)	= `UserMenu` (expansion lives on `MenuItem expanded`)	`actions-menus/Menu.tsx`
Split Button	Style × Tone × State (16)	= `SplitButton` (+ `menuPlacement` bottom · top)	`actions-menus/Actions.tsx`
FAB	State (4)	= `Fab`	`actions-menus/Actions.tsx`
Speed Dial Item	State (3)	= `SpeedDialItem`	`actions-menus/Actions.tsx`
Speed Dial	—	= `SpeedDial`	`actions-menus/Actions.tsx`
Segment	Content × State (8)	⊕ `SegmentedControl` (`options[]`)	`actions-menus/Actions.tsx`
Segmented Control	Content × Items (4)	= `SegmentedControl`	`actions-menus/Actions.tsx`
View Tab	State (3)	= `ViewTab`	`actions-menus/Actions.tsx`
View Tab Bar	—	= `ViewTabBar`	`actions-menus/Actions.tsx`

MenuItem

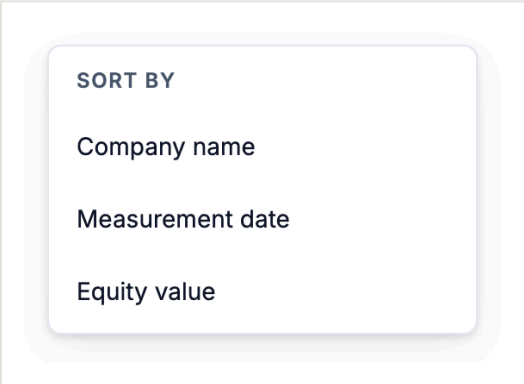
REACT MenuItem FILE src/components/actions-menus/Menu.tsx FIGMA Menu Item VARIANTS State × Tone (7)

Menu Item — one action row in a Context Menu, User Menu or dropdown.

Figma: `20 · Menus & Actions / Menu Item` (State × Tone). Hover is a CSS state; Selected is `selected`; Disabled is the native attribute.

Tokens: hover Background/Subtle · selected Background/Brand Subtle + Text/Brand · destructive Text/Negative, hover Background/Negative Subtle.

Accessibility: items are `tabindex=-1`; the parent menu (`ContextMenu`, `UserMenu`, or `useMenuKeys`) owns the roving focus. Use `asChild` to render a router link: `<MenuItem asChild><Link to="/x">Go</Link></MenuItem>`.



Storybook · Menus/ContextMenu › Selectable Items

PROPS

PROP20 · MENUS & ACTIONS (11)	TYPEFIGMA	DEFAULTVARIANTS	DESCRIPTIONREACT
`icon`	`ReactNode`	—	Leading glyph. Pass an SDS_Main icon wrapped in `Icon`.
`tone`	`"default"` | `"destructive"`	`default`	`destructive` is for irreversible actions (Delete, Revoke). Always last, after a divider.
`selected`	`boolean`	—	The item is the current/selected option. Exposed as `menuitemradio` + `aria-checked` (pass `role="menuitemcheckbox"` for independent toggles).
`hasSubmenu`	`boolean`	—	Draws a trailing chevron for an item that opens a submenu or expands in place. Sets `aria- haspopup`.
`expanded`	`boolean`	—	For `hasSubmenu` items that expand inline: whether the children are showing.
`disabled`	`boolean`	—	—

PROP20 · MENUS & ACTIONS (11)	TYPEFIGMA	DEFAULTVARIANTS	DESCRIPTIONREACT
`onClick`	`MouseEventHandler<HTMLElement>`	—	—
`href`	`string`	—	Renders an anchor. Ignored when `disabled`.
`className`	`string`	—	—
`asChild`	`boolean`	—	Render the single child element instead of the component's own tag, merging props onto it — use it to make a router link look like a Button: `<Button asChild><Link to="/x">Go</Link></Button>`.

20 · MENUS & ACTIONS

MenuDivider

REACT MenuDivider FILE src/components/actions-menus/Menu.tsx FIGMA Menu Item
VARIANTS State × Tone (7)

Horizontal rule between Menu Item groups. Destructive items sit after one.

PROPS

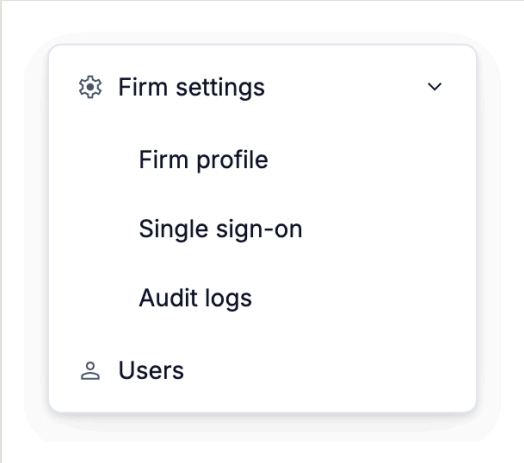
PROP	TYPE	DEFAULT	DESCRIPTION
`className`	`string`	—	—

20 · MENUS & ACTIONS

MenuSubItems

REACT MenuSubItems FILE src/components/actions-menus/Menu.tsx FIGMA Menu Item
VARIANTS State × Tone (7)

Children of an inline-expanding Menu Item (Firm settings → Firm profile, SSO…). Place directly after the parent item.



Storybook · Menus/ContextMenu › Inline Submenu

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`	`string`	`Account`	Accessible name of the group — usually the parent item's label ("Firm settings").
`className`	`string`	—	—

---

20 · MENUS & ACTIONS

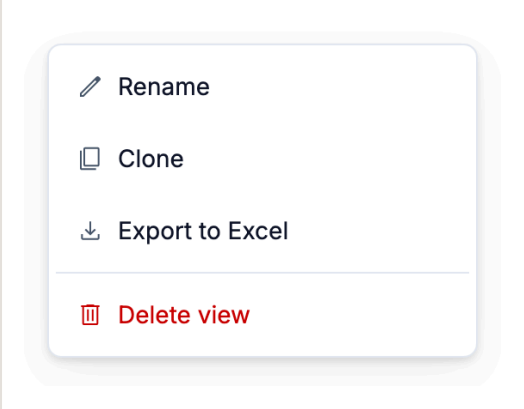
ContextMenu

REACT ContextMenu FILE src/components/actions-menus/Menu.tsx FIGMA Context Menu VARIANTS Header (2)

Context Menu — the floating menu opened by a kebab (⋮) or meatball (•••) trigger: page actions, saved-view actions, per-row and per-column actions.

Surface: Background/Surface Raised + Elevation/Overlay. Compose from `MenuItem` and `MenuDivider`. Positioning and open state belong to the trigger; this is the surface only.

Keyboard: Up/Down, Home/End, type-ahead, Right/Left for inline submenus, Esc and Tab call `onClose`. Pass `autoFocus` when the menu is mounted on open.



Storybook · Menus/ContextMenu › Default

PROPS

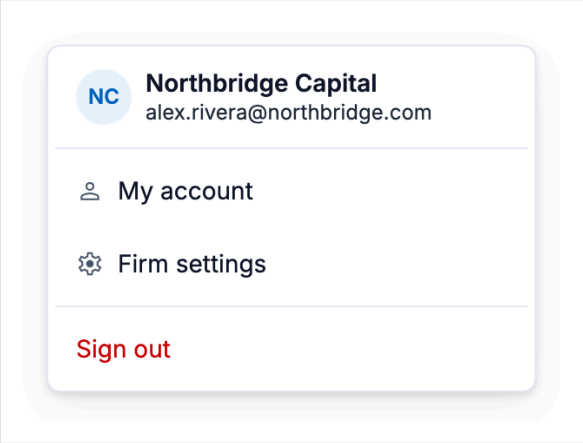
PROP	TYPE	DEFAULT	DESCRIPTION
heading	ReactNode	—	Optional overline heading above the items.
label	string	Account	Accessible name — what the menu acts on ("Firm summary actions").
className	string	—	—
onClose	((reason: MenuCloseReason) => void)	—	Called when the user presses Esc or Tab inside the menu. The menu does not own open state: close it here and return focus to the trigger.

20 · MENUS & ACTIONS

UserMenu

REACT UserMenu FILE src/components/actions-menus/Menu.tsx FIGMA User Menu
VARIANTS Firm Settings Expanded (2)

User Menu — account menu opened from the Avatar at the far right of the Primary Menu. Header, then account and firm administration links, then Sign out after a divider. Same keyboard model as `ContextMenu`.



Storybook · Menus/ContextMenu › User menu

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
namerequired	ReactNode	—	Firm or user shown in the header.
onClose	((reason: MenuCloseReason) => void)	—	Called when the user presses Esc or Tab inside the menu. The menu does not own open state: close it here and return focus to the trigger.
detail	ReactNode	—	Secondary line — usually the signed-in email.

PROP	TYPE	DEFAULT	DESCRIPTION
avatarSrc	string	—	—
initials	string	—	—
avatarAlt	string	—	Describes the avatar picture for assistive tech. Defaults to empty (the name is shown beside it).
label	string	Account	Accessible name of the menu. Default "Account".
children	ReactNode	—	MenuItem / MenuDivider children. End with Sign out after a divider.
className	string	—	—

20 · MENUS & ACTIONS

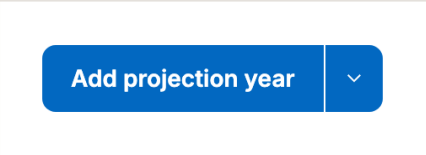
SplitButton

REACT SplitButton FILE src/components/actions-menus/Actions.tsx FIGMA Split Button
VARIANTS Style × Tone × State (16)

Split Button — primary action plus a caret that opens alternatives ("+ Add projection year ▾" → Add historical year; "Report ▾" → Validate, Preview, Finalize, Send). The two halves are separate hit targets.

Figma: `Split Button` (Style × Tone × State, 16 variants). Hover and Open map to CSS and `menuOpen`.

Accessibility: opening moves focus to the first menu item; Esc, outside click and Tab close it and focus returns to the caret. ArrowDown on the caret opens the menu. The caret is `aria-controls` the menu popover.



Storybook · Menus/Actions › Split Primary

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
menuLabelrequired	string	—	Accessible name of the caret half ("More save options").
variant	"primary" | "secondary"	primary	—
tone	"main" | "positive"	main	`positive` is the green Save used on edit screens.
leadingIcon	ReactNode	—	—

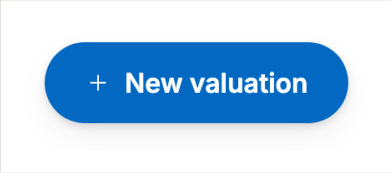
PROP	TYPE	DEFAULT	DESCRIPTION
onClick	`() => void`	—	The main action.
onMenuToggle	`() => void`	—	Fires when the caret is activated, and when Esc / outside click / Tab ask to close an open menu (so a consumer that toggles its own state keeps working). Prefer `onOpenChange`.
onOpenChange	`(open: boolean) => void`	—	Reports the next open state: caret click, ArrowDown on the caret, Esc, outside click, Tab.
menuOpen	`boolean`	—	Whether that menu is showing. Flips the caret. Omit to let the component own it.
defaultMenuOpen	`boolean`	`false`	Initial open state when uncontrolled.
disabled	`boolean`	—	—
menu	`ReactNode`	—	Render the Context Menu here; it is positioned under the caret.
menuPlacement	`"bottom"` | `"top"`	`bottom`	Where the menu opens. `top` for a button at the foot of a page or drawer. Default `bottom`.
className	`string`	—	—

20 · MENUS & ACTIONS

Fab

REACT Fab FILE src/components/actions-menus/Actions.tsx FIGMA FAB VARIANTS State (4)

FAB — the page's single floating "add" action, bottom-right, above the Workspace Drawer. Never destructive. With `hasMenu` it opens a Speed Dial. Inside a `SpeedDial` trigger it is `aria-controls` the dial's menu automatically.



Storybook · Menus/Actions › FAB

PROPS

PROP	REQUIRED	TYPE	DEFAULT	STRING DESCRIPTION —
`icon`		`ReactNode`	—	—
`hasMenu`		`boolean`	—	Shows a caret; the FAB then opens a Speed Dial.

PROP	REQUIRED	TYPE	DEFAULT	STRING DESCRIPTION–
`open`		`boolean`	—	Speed Dial is showing — the icon becomes a close glyph. Omit to let the FAB own it.
`defaultOpen`		`boolean`	`false`	Initial open state when uncontrolled.
`onOpenChange`		`((open: boolean) => void)`	—	Reports the next open state when `hasMenu` and the FAB is clicked.
`onClick`		`((event: MouseEvent<HTMLButtonElement, MouseEvent>) => void)`	—	—
`disabled`		`boolean`	—	—
`fixed`		`boolean`	`true`	Fixes the button to the bottom-right of the viewport. Default true.
`className`		`string`	—	—

20 · MENUS & ACTIONS

SpeedDialItem

REACT SpeedDialItem FILE src/components/actions-menus/Actions.tsx FIGMA Speed Dial Item
VARIANTS State (3)

One option in a Speed Dial stack. Disabled when the option is not available yet. Focus is managed by the parent `SpeedDial`.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`onClick`	`((event: MouseEvent<HTMLButtonElement, MouseEvent>) => void)`	—	—
`disabled`	`boolean`	—	—
`className`	`string`	—	—

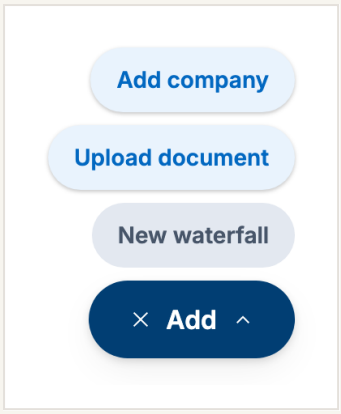
20 · MENUS & ACTIONS

SpeedDial

REACT SpeedDial FILE src/components/actions-menus/Actions.tsx FIGMA Speed Dial

Speed Dial — the FAB expanded into its actions. Render over a `Scrim`; clicking the scrim or the FAB closes it.

Keyboard (menu pattern): Up/Down, Home/End, type-ahead; Esc closes and returns focus to the FAB; Tab closes.



Storybook · Menus/Actions › Speed dial

PROPS

PROPTYPE	TYPEDEFAULT	DEFAULTDESCRIPTION	DESCRIPTIONONCLICK
`children` required	`ReactNode`	—	SpeedDialItem children, nearest-first = most used.
`trigger` required	`ReactNode`	—	The FAB that opened it (rendered `open`).
`label` required	`string`	—	—
`onClose`	`((reason: MenuCloseReason) => void)`	—	Esc / Tab inside the dial. Close it here; focus has already been moved back to the FAB on Esc.
`autoFocus`	`boolean`	—	Focus the first item on mount. Off by default so statically rendered dials never steal focus.
`className`	`string`	—	—

20 · MENUS & ACTIONS

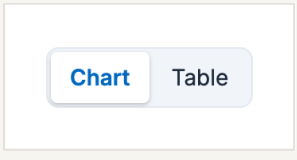
SegmentedControl

REACT SegmentedControl FILE src/components/actions-menus/Actions.tsx FIGMA Segment

VARIANTS Content × State (8)

Segmented Control — mutually exclusive view toggle: Chart | Table, Client | Internal, a date snapshot | Latest, and the icon-only list | calendar switch. Exactly one segment is selected.

Generalises `ToolSwitch`, which stays for the Valuations | Workboard switch. Radiogroup pattern: one tab stop (the selected segment, else the first enabled); arrow keys move focus and selection; Home/End jump.



Storybook · Menus/Actions › Segmented

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
optionsrequired	`readonly SegmentOption<V>[]`	—	—
labelrequired	`string`	—	Accessible name of the group ("View").
value	`string`	—	Selected value. Omit to let the control own it (see `defaultValue`).
defaultValue	`string`	—	Initial value when uncontrolled. Defaults to the first enabled option.
onChange	`((value: V) => void)`	—	—
className	`string`	—	—

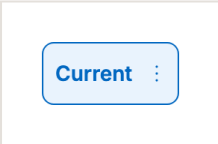
20 · MENUS & ACTIONS

ViewTab

REACT ViewTab FILE src/components/actions-menus/Actions.tsx FIGMA View Tab VARIANTS State (3)

View Tab — a saved view, scenario or note tab (Firm summary ⋮, Current, Note 1 ⋮, Backsolve ⋮, a closable company filter).

Structure: the kebab and close buttons are siblings of the tab inside a plain wrapper (so the tablist only ever owns tabs), and are out of the tab order (`tabindex=-1`); the `ViewTabBar` is the single tab stop. From the tab, Shift+F10 / the Menu key triggers `onMenu` and Delete triggers `onClose`.



Storybook · Menus/Actions › View tab (single)

PROPS

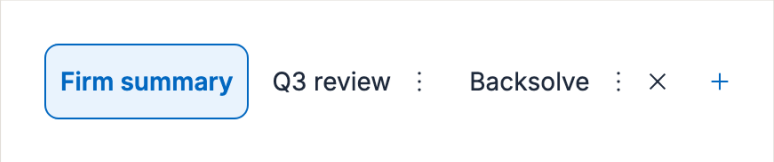
PROP	TYPE	DEFAULT	DESCRIPTION
selected	boolean	—	—
onSelect	(() => void)	—	—
onMenu	(() => void)	—	Shows the kebab; open a Context Menu (Rename, Clone, Delete) from it. Also bound to Shift+F10 / the Menu key on the tab.
onClose	(() => void)	—	Shows a close button for a temporary tab. Also bound to Delete on the tab.
label	string	—	Name used in the kebab / close labels ("Firm summary options"). Defaults to the text of `children`.
tabId	string	—	`id` of the tab button, so a panel can reference it with `aria-labelledby`.
panelId	string	—	`id` of the panel this tab controls (`aria-controls`).
className	string	—	—

20 · MENUS & ACTIONS

ViewTabBar

REACT ViewTabBar FILE src/components/actions-menus/Actions.tsx FIGMA View Tab Bar

Row of View Tabs with a trailing "+" that creates a new view. The tablist element is a zero-size owner (`aria-owns`) of the tab buttons. Tabs pattern: one tab stop (the selected tab), Left/Right and Home/End move focus between tabs, Enter / Space select (manual activation).



Storybook · Menus/Actions › View tabs

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	string	—	Accessible name of the tab list.
onAdd	(() => void)	—	Creates a new view. Omit to hide the add button.
addLabel	string	Add view	—
className	string	—	—

21 FORM PATTERNS

COMPONENTS · 13 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Floating Label Field	Type × State (12)	⊕ `FloatingLabelInput` + `FloatingLabelSelect`	`form- patterns/Fields.tsx`
Number Field	Adornment × State (8)	= `NumberField` (`suffix` switches the adornment; + `label floating label)`	`form- patterns/Fields.tsx`
Time Field	State (3)	= `TimeField`	`form- patterns/Fields.tsx`
Tag Input	State (3)	= `TagInput`	`form- patterns/Fields.tsx`
Copy Field	Type × State (4)	= `CopyField` (`secret`)	`form- patterns/Fields.tsx`
Combobox Option	Selection × State (8)	= `ComboboxOption`	`form- patterns/Pickers.tsx`
Show More Row	—	= `ShowMoreRow`	`form- patterns/Pickers.tsx`
Combobox Panel	Footer (2)	= `ComboboxPanel`	`form- patterns/Pickers.tsx`
Repeatable Row	Layout (2)	= `RepeatableRow` (the fields are children)	`form- patterns/Pickers.tsx`
Dropzone	State (4)	= `Dropzone` (Hover = drag-over, CSS; + `prompt` override)	`form- patterns/Pickers.tsx`
Slider	State (3)	= `Slider`	`form- patterns/Pickers.tsx`
Inline Edit	State (4)	= `InlineEdit`	`form- patterns/Pickers.tsx`
Inline Picker	State (3)	= `InlinePicker`	`form- patterns/Pickers.tsx`
—	—	+ `SelectMenu` + `SelectMenuOption` — the open listbox surface of a select	`form- patterns/Pickers.tsx`
—	—	+ `ImageCropField` — logo / avatar preview, zoom Slider, delete	`form- patterns/Pickers.tsx`

Merge. The two Floating Label types are separate exports because an `<input>` and a `<select>` take different props; the label float is CSS (`:placeholder-shown`), so both work controlled or uncontrolled.

21 · FORM PATTERNS

FloatingLabelInput

REACT FloatingLabelInput FILE src/components/form-patterns/Fields.tsx FIGMA Floating Label Field

Variants Type × State (12)

Floating Label Field (Type=Input) — outlined field whose label rests inside the box when empty and floats onto the top border once focused or filled. The dominant field style in Firm Settings, Account, Daily NAV and every Create/Add modal. Height = Target/Minimum (44).

Use `FormField` + `Input` in dense, in-table forms; use this for page-level and modal forms. The float is pure CSS (`:placeholder-shown`), so it works controlled or uncontrolled. `ref` and rest props land on the `<input>`; `className` on the wrapper. A consumer `aria-describedby` is merged with the helper text id. `required` adds a visual asterisk to the label.



Storybook · Forms/Patterns › Floating Input

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`required	`string`	—	—
`helperText`	`ReactNode`	—	Helper text under the field. In `state="error"` it is announced (`role="alert"`).
`state`	`"disabled"` | `"default"` | `"error"`	`default`	`error` turns the outline and helper negative. Say what is wrong and how to fix it.
`className`	`string`	—	Lands on the outer wrapper, not on the control.

21 · FORM PATTERNS

FloatingLabelSelect

REACT FloatingLabelSelect FILE src/components/form-patterns/Fields.tsx FIGMA Floating Label Field

Variants Type × State (12)

Floating Label Field (Type=Select). The label is always floated, because a native select always shows a value — include an empty first option for "nothing chosen yet". Above ~8 options use `ComboboxPanel` instead. `ref` and rest props land on the `<select>`; `className` on the wrapper.



Storybook · Forms/Patterns › Floating Select

PROPS

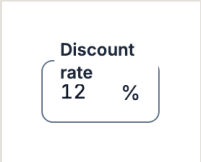
PROP	TYPE	DEFAULT	DESCRIPTION
`label`**required**	`string`	—	—
`helperText`	`ReactNode`	—	Helper text under the field. In `state="error"` it is announced (`role="alert"`).
`state`	`"disabled"` | `"default"` | `"error"`	`default`	`error` turns the outline and helper negative. Say what is wrong and how to fix it.
`className`	`string`	—	Lands on the outer wrapper, not on the control.

21 · FORM PATTERNS

NumberField

REACT NumberField FILE src/components/form-patterns/Fields.tsx FIGMA Number Field
VARIANTS Adornment × State (8)

Number Field — numeric entry. With `suffix`: a unit adornment (thresholds %, multiples). Without: an up/down stepper for integer counts (window days, SFTP port). Give it `label` for a floating label that sits with FloatingLabelInputs in a page or modal form, or wrap it in a FormField in a dense form (or pass `aria-label`). Controlled (`value`) or uncontrolled (`defaultValue`). `ref` and rest props land on the `<input>`; `className` on the wrapper. The stepper buttons are outside the tab order — ↑/↓ on the input are the keyboard path.



Storybook · Forms/Patterns › Number

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`value`	`number | ""`	—	Controlled value. `''` is an empty field. Omit it (and use `defaultValue`) for uncontrolled use.
`defaultValue`	`number | ""`	—	Initial value when uncontrolled.
`onChange`	`((value: number | "") => void)`	—	—

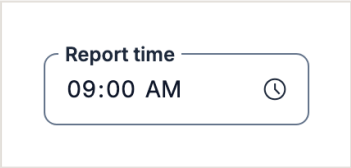
PROP	TYPE	DEFAULT	DESCRIPTION
suffix	ReactNode	—	Unit adornment (%, x, bps). Mutually exclusive with the stepper; announced as part of the field description.
min	number	—	—
max	number	—	—
step	number	1	—
state	"disabled" | "default" | "error"	default	—
unitLabel	string	value	Name for the increment / decrement buttons ("trading days").
label	ReactNode	—	Floating label on the top border, matching a filled FloatingLabelInput. A number field always shows its value slot, so the label is always floated. Wired to the input with `htmlFor`. Without it, give the field an `aria-label` or wrap it in a `FormField`.

21 · FORM PATTERNS

TimeField

REACT TimeField FILE src/components/form-patterns/Fields.tsx FIGMA Time Field VARIANTS State (3)

Time Field — time-of-day entry (report send time). Native `type="time"`, so the browser provides the picker and locale formatting. Put the firm time zone in the helper text. Controlled or uncontrolled (native). `ref` and rest props land on the `<input>`; `className` on the wrapper.



Storybook · Forms/Patterns › Time

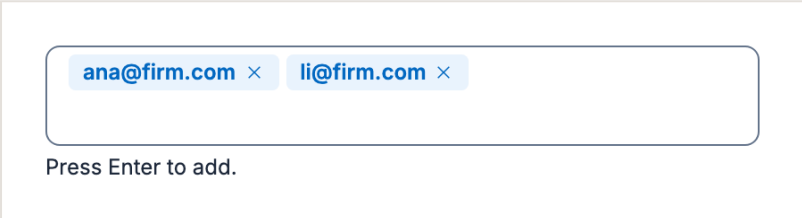
PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
state	"disabled" | "default" | "error"	default	—
label	ReactNode	—	Floating label, as on `NumberField`. Without it, give the field an `aria-label` or wrap it in a `FormField`.
icon	ReactNode	—	Replaces the clock glyph.

TagInput

REACT TagInput FILE src/components/form-patterns/Fields.tsx FIGMA Tag Input VARIANTS State (3)

Tag Input — free text that becomes a removable Chip on Enter, comma or paste (email domains, document names to request). Backspace on an empty entry removes the last chip. A rejected value shows its reason as visible text under the field (`role="alert"`), and additions and removals are announced through a live region. `ref` is the entry `<input>`; `className` and rest props land on the bordered group. Controlled (`values`) or uncontrolled (`defaultValues`).



Storybook · Forms/Patterns › Tags

PROPS

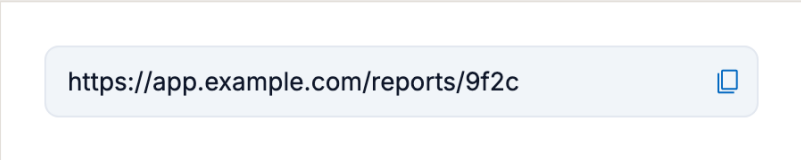
PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	`string`	—	Accessible name of the group and the entry field ("Associated email domains").
values	`readonly string[]`	—	Controlled tags. Omit (and use `defaultValues`) for uncontrolled use.
defaultValues	`readonly string[]`	—	—
onChange	`((values: string[]) => void)`	—	—
validate	`((value: string) => string)`	—	Rejects a value before it becomes a chip. Return an error string to block it.
prefix	`ReactNode`	—	—
placeholder	`string`	—	—
helperText	`ReactNode`	—	Visible hint below the field ("Press Enter to add"). Linked to the entry with `aria-describedby`.
state	`"disabled" | "default" | "error"`	`default`	—
required	`boolean`	—	—
id	`string`	—	Id of the entry `<input>` (for an external `<label htmlFor>`).
disabled	`boolean`	—	Disables the entry and the chips' remove buttons.

21 · FORM PATTERNS

CopyField

REACT CopyField FILE src/components/form-patterns/Fields.tsx FIGMA Copy Field
VARIANTS Type × State (4)

Copy Field — a read-only value the user copies into another system (SSO metadata / ACS URLs, SCIM base URL, API tokens). The value is a read-only `<input>` named by `label`; the copy button stays mounted (focus is kept) and the result ("Copied" / "Couldn't copy") is announced through a persistent live region and shown for 2s. Never editable. `ref` is the value `<input>`; `className` and rest props land on the wrapper.



Storybook · Forms/Patterns › Copy

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	string	—	—
labelrequired	string	—	Accessible name ("SCIM endpoint base URL").
secret	boolean	—	Masks all but the first `revealPrefix` characters, with a reveal toggle.
revealPrefix	number	10	—

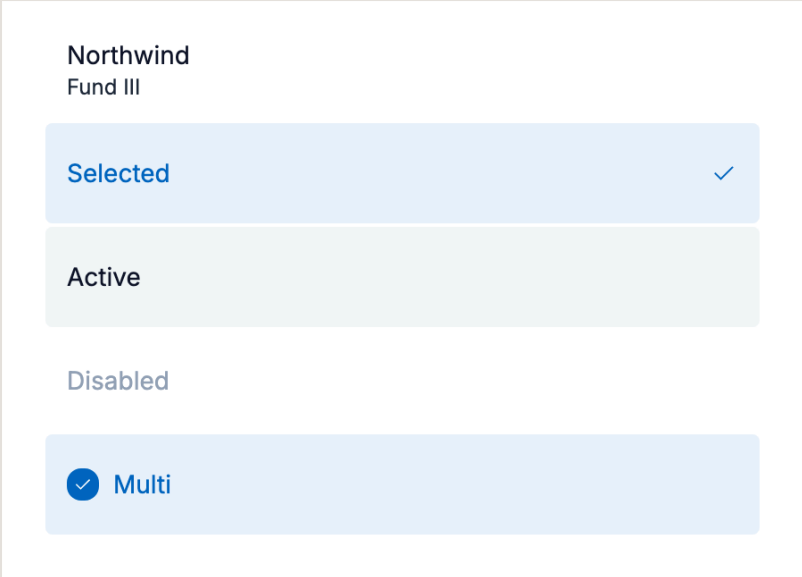
21 · FORM PATTERNS

ComboboxOption

REACT ComboboxOption FILE src/components/form-patterns/Pickers.tsx FIGMA Combobox Option

Variants Selection × State (8)

Combobox Option — one result row inside a Combobox Panel. Not focusable on its own: the owning input points `aria- activedescendant` at its `id`. Mouse press does not move focus off the input.



Storybook · Forms/Patterns › Combobox Options

PROPS

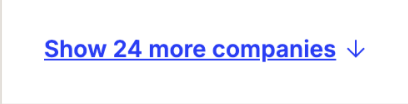
PROP	TYPE	DEFAULT	DESCRIPTION
`detail`	`ReactNode`	—	Secondary line — fund, date, ticker.
`selected`	`boolean`	—	—
`selection`	`"single" | "multi"`	`single`	`multi` draws a leading checkbox; several options can be selected.
`active`	`boolean`	—	Keyboard-highlighted option (aria-activedescendant target).
`disabled`	`boolean`	—	—
`onSelect`	`(() => void)`	—	—

21 · FORM PATTERNS

ShowMoreRow

REACT ShowMoreRow FILE src/components/form-patterns/Pickers.tsx FIGMA Show More Row

Show More Row — reveals the next page of a truncated list ("Show 24 more companies").



Storybook · Forms/Patterns › Show More

PROPS

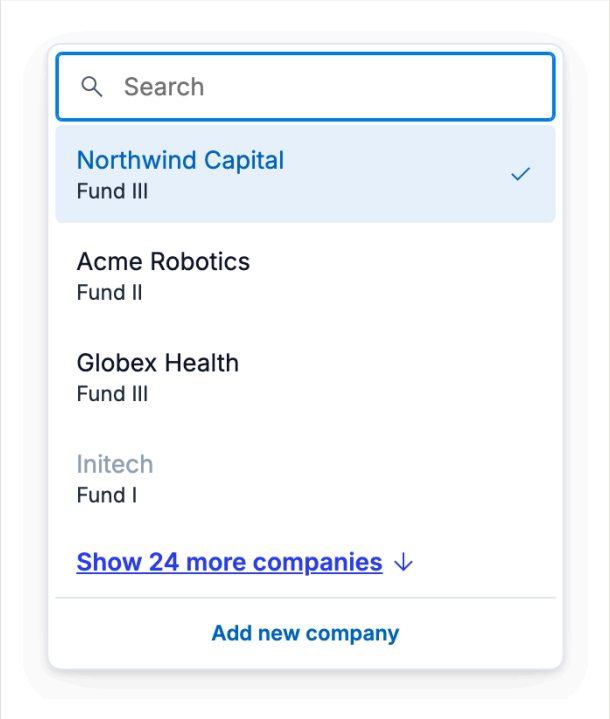
No component-specific props; accepts the native element attributes.

ComboboxPanel

REACT ComboboxPanel FILE src/components/form-patterns/Pickers.tsx FIGMA Combobox Panel
VARIANTS Footer (2)

Combobox Panel — searchable dropdown: search field, filtered options, an optional Show More Row and create action. Use instead of `Select` whenever the list can exceed ~8 items (companies, measurement dates, versions, folders, people). Filtering is the caller's job — pass the already-filtered `items`.

Keyboard (focus stays in the input, `aria-activedescendant` follows): ↑/↓ move (disabled options skipped, active option scrolled into view), Enter selects, Esc clears the query, then calls `onEscape`. Home/End keep their text-caret meaning. `ref` is the search `<input>`; `className` and rest props land on the panel root. The panel is a surface: the trigger owns open state, so `aria-expanded` is always true while it is rendered.



Storybook · Forms/Patterns › Combobox

PROPS

PROPCOMBOBOXPANEL	TYPEREACT	DEFAULT	DESCRIPTION
`items` required	`readonly ComboboxItem[]`	—	—
`onSelect` required	`(value: string) => void`	—	—
`query` required	`string`	—	—
`onQueryChange` required	`(query: string) => void`	—	—
`label` required	`string`	—	Accessible name of the list ("Companies").

PROPCOMBOBOXPANEL	TYPEREACT	DEFAULT	DESCRIPTION
`value`	`string | readonly string[]`	—	Selected value(s). An array switches the panel to multi-select (or set `multiple`).
`multiple`	`boolean`	—	Multi-select without passing an array (e.g. nothing selected yet).
`searchPlaceholder`	`string`	`Search`	—
`searchLabel`	`string`	—	Accessible name of the search field. Default: `label`.
`showMore`	`{ label: ReactNode; onClick: () => void; }`	—	"Show 24 more companies" — omit when everything is showing.
`footer`	`ReactNode`	—	Footer create action ("+ Add new company", "+ Save as new version").
`renderOption`	`((item: ComboboxItem) => ReactNode)`	—	Custom option content (replaces the label text; `detail` is still shown).
`autoFocus`	`boolean`	—	Focus the search field on mount. Opt-in: a panel rendered statically must not steal focus.
`onEscape`	`(() => void)`	—	Called on Escape (after a non-empty query has been cleared). The trigger usually closes the panel here.

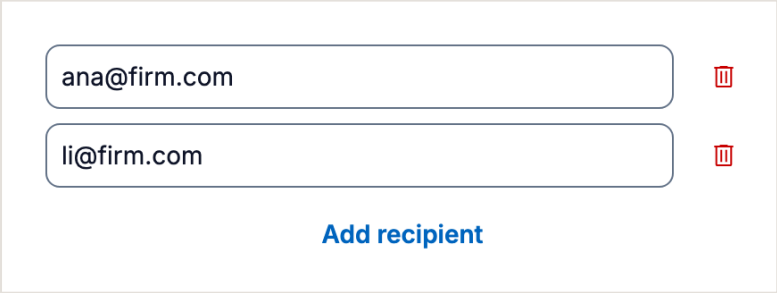
21 · FORM PATTERNS

RepeatableRow

REACT RepeatableRow FILE src/components/form-patterns/Pickers.tsx FIGMA Repeatable Row

VARIANTS Layout (2)

Repeatable Row — one row of a list editor (recipients, domains, group → role mappings). Stack rows and end the list with a tertiary "+ Add " Button. Pass `index` or a specific `removeLabel` so the remove buttons are distinguishable to a screen reader. `ref` and rest props land on the row.



Storybook · Forms/Patterns › Repeatable

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`children`required	`ReactNode`	—	The field(s) of this row: one input, or input → select for a mapping.
`onRemove`required	`() => void`	—	—
`removeDisabled`	`boolean`	—	Keep a required list from reaching zero rows.
`removeLabel`	`string`	`Remove image`	Name of the remove button. Make it specific ("Remove recipient ana@firm.com"); default derives from `index`.
`index`	`number`	—	Zero-based position, used for the default remove name ("Remove row 2") and the row's group name.

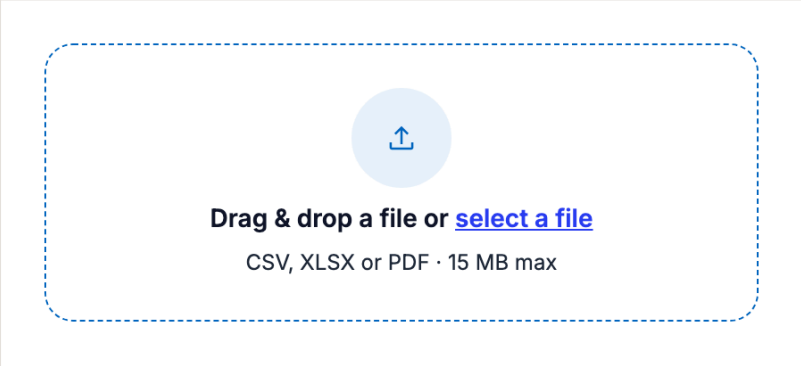
---

21 · FORM PATTERNS

Dropzone

REACT Dropzone FILE src/components/form-patterns/Pickers.tsx FIGMA Dropzone VARIANTS State (4)

Dropzone — drag & drop or browse. Report templates, documents, logos. Hover is set while a file is dragged over. The browse button is present in every state (default prompt, "Choose another file" while uploading, "Try again" after an error), so a keyboard user can always pick a file. `ref` and rest props land on the zone (`role="group"`).



Storybook · Forms/Patterns › Dropzone Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`onFiles`required	`(files: File[]) => void`	—	—
`hint`required	`ReactNode`	—	Always state accepted types and size: "CSV, XLSX or PDF · 15 MB max". Linked to the browse button.
`accept`	`string`	—	Accepted types, as for `<input accept>` (".csv,.xlsx,.pdf").

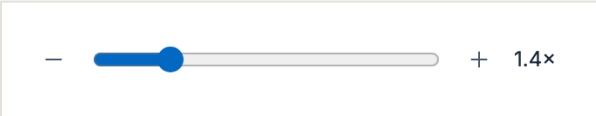
PROP	TYPE	DEFAULT	DESCRIPTION
multiple	boolean	—	—
state	"default" | "error" | "uploading"	default	—
progress	number	—	0–100 while uploading. Omit for an indeterminate bar.
message	ReactNode	—	Uploading label or error reason.
progressLabel	string	Upload progress	Accessible name of the progress bar. Default "Upload progress".
disabled	boolean	—	Ignores drops and disables the browse button.
prompt	ReactNode | ((browse: (label: ReactNode) => ReactNode) => ReactNode)	—	Replaces the default prompt ("Drag & drop a file or select a file") in the default state. Pass a render function to keep the browse link: `prompt= {(browse) => <>Drop a logo or {browse('choose an image')}</>}`.

21 · FORM PATTERNS

Slider

REACT Slider FILE src/components/form-patterns/Pickers.tsx FIGMA Slider VARIANTS State (3)

Slider — a continuous value in a range (logo cropper zoom). Native range input, so ←/→/↑/↓ (one step), Home/End, PageUp/PageDown (larger step) work, controlled or uncontrolled. Pass `aria-valuetext` when the number needs a unit. Never for financial values — use `NumberField`. `ref` and rest props land on the `<input>`; `className` on the wrapper.



Storybook · Forms/Patterns › Slider

PROPS

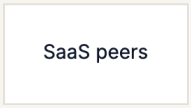
PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	string	—	Accessible name ("Zoom").
minIcon	ReactNode	—	—
maxIcon	ReactNode	—	—
readout	ReactNode	—	Visible current value ("1.4×"), shown after the track. Pair it with `aria-valuetext` for the spoken form.

21 · FORM PATTERNS

InlineEdit

REACT InlineEdit FILE src/components/form-patterns/Pickers.tsx FIGMA Inline Edit VARIANTS State (4)

Inline Edit — click-to-edit text inside a layout (comp group name, a new security name in a column header). Enter / blur commits, Escape cancels (a cancelled edit is never committed), and focus returns to the trigger after Enter or Escape. The external `value` is read when an edit starts. `ref` points at whichever element is showing (trigger button or input); rest props land on it too.



Storybook · Forms/Patterns › Inline Edit Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`placeholder` **required**	`string`	—	Link-styled prompt shown while empty ("Enter name").
`label`**required**	`string`	—	Accessible name ("Comp group name").
`value`	`string`	—	Controlled value. Omit (and use `defaultValue`) to let the component own it.
`defaultValue`	`string`	—	—
`onCommit`	`((value: string) => void)`	—	Fires with the new text when an edit is committed (Enter or blur) and it differs.
`invalid`	`boolean`	—	Marks the edit field `aria-invalid`.
`disabled`	`boolean`	—	—
`readOnly`	`boolean`	—	Shows the value but cannot be edited.

21 · FORM PATTERNS

InlinePicker

REACT InlinePicker FILE src/components/form-patterns/Pickers.tsx FIGMA Inline Picker
VARIANTS State (3)

Inline Picker — link-styled dropdown inside text or a key-value list ("09/21/2026 | V-2 ▾", in-cell "Select option ▾"). Opens a MenuPanel, SelectMenu or Combobox Panel — point `aria-controls` at it. The accessible name is "{label}: {visible value}". In a form, use `Select`. `ref` and rest props (`disabled`, `aria-controls`, …) land on the button.

09/21/2026 | V-2

Storybook · Forms/Patterns › Inline Picker Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
childrenrequired	ReactNode	—	Main value ("09/21/2026").
labelrequired	string	—	Accessible name prefix ("Previous versions"). The visible value is appended, so the name contains the visible text.
secondary	ReactNode	—	Optional second value after a separator ("V-2").
open	boolean	false	—

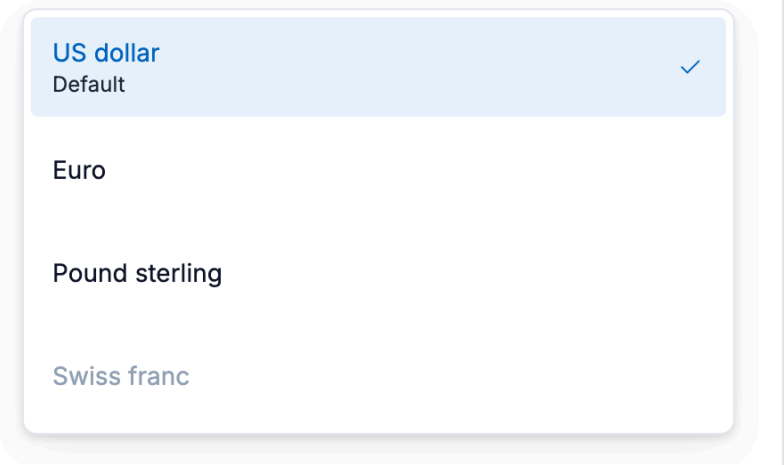
21 · FORM PATTERNS

SelectMenu

REACT SelectMenu FILE src/components/form-patterns/Pickers.tsx

Select Menu — the open listbox surface of a select: a raised panel of options with a selected check, disabled options and optional descriptions, plus optional search and footer slots.

The listbox owns its keyboard model (focus stays on the listbox, `aria-activedescendant` follows): ↑/↓ move, Home/End jump, typing jumps to the next option starting with those letters, Enter / Space choose the active option, Esc calls `onEscape`. Pass your own `aria-activedescendant` to take over highlighting. The trigger still owns open state, positioning and outside-click dismissal. `ref` and rest props (`id`, `onKeyDown`, `tabIndex`) land on the `role="listbox"` element; `className` on the outer wrapper. For a searchable list that can exceed ~8 items, prefer `ComboboxPanel`.



Storybook · Forms/Patterns › Select menu

PROPS

PROP	TYPESTORYBOOK · FORMS/PATTERNS › INLINE PICKER DEFAULT	DEFAULTPROPS	DESCRIPTIONPROP
childrenrequired	ReactNode	—	SelectMenuOption instances (MenuGroupLabel / MenuDivider between groups are fine).
labelrequired	string	—	Accessible name of the listbox ("Currency").
search	ReactNode	—	Optional slot above the list — a search input for a long list. The caller filters.
footer	ReactNode	—	Optional slot below the list — a create action ("+ Add role").
multiselectable	boolean	—	Several options can be selected at once.
onEscape	(() => void)	—	Called on Escape — the trigger usually closes the menu and returns focus here.
className	string	—	Lands on the outer wrapper.

21 · FORM PATTERNS

SelectMenuOption

REACT SelectMenuOption FILE src/components/form-patterns/Pickers.tsx

Select Menu Option — one choice in a Select Menu. The selected option carries a trailing check as well as the tint, so selection is never colour alone (R8). A disabled option stays visible and is skipped. It is not focusable itself — the `SelectMenu` listbox points `aria-activedescendant` at it, and an `id` is generated when you do not pass one.

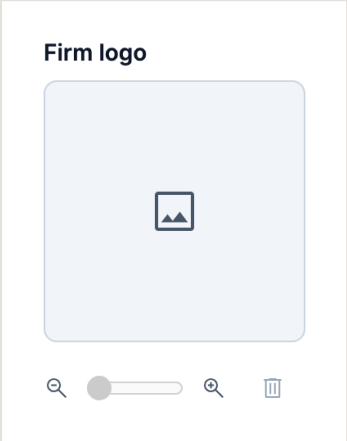
PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
description	ReactNode	—	Optional second line explaining the choice ("Visible to every analyst").
icon	ReactNode	—	Leading glyph, passed through `Icon`.
selected	boolean	—	—
disabled	boolean	—	—
active	boolean	—	Keyboard-highlighted option — set it only when you drive `aria-activedescendant` yourself; `SelectMenu` does it otherwise.
onSelect	(() => void)	—	—

ImageCropField

REACT ImageCropField FILE src/components/form-patterns/Pickers.tsx

Image Crop Field — preview and framing for an uploaded logo or avatar: a framed preview well, a zoom Slider between zoom- out / zoom-in glyphs, and a delete ButtonIcon. Pair it with a Dropzone for the upload itself; zoom and disabled-while-empty are the only behaviour it owns. While empty the controls are disabled and a screen-reader hint says why. `ref` and rest props land on the group.



Storybook · Forms/Patterns › Image Crop

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label` required	`string`	—	Visible label and the group's accessible name ("Firm logo").
`zoom` required	`number`	—	Scale applied to the preview, 1 = fit.
`onZoomChange` required	`(zoom: number) => void`	—	—
`src`	`string`	—	The uploaded image. Omit to show the placeholder well.
`alt`	`string`	—	Alternative text for the preview. Default: the label.
`zoomMin`	`number`	`1`	—
`zoomMax`	`number`	`3`	—
`zoomStep`	`number`	`0.1`	—
`onRemove`	`(() => void)`	—	Removes the image. Omit to hide the delete button.
`removeLabel`	`string`	`Remove image`	—

PROP	TYPE	DEFAULT	DESCRIPTION
`shape`	`"square"` | `"wide"`	`square`	`square` for an avatar or mark, `wide` for a wordmark logo. Default `square`.
`placeholderIcon`	`ReactNode`	—	Glyph for the empty well. Default Material `image`.

22 GRID PATTERNS

COMPONENTS · 10 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Row Label Cell	Type × Expand (8)	= `RowLabelCell`	`grid- patterns/Grid.tsx`
Grid Value Cell	Kind × State (12)	= `GridValueCell`	`grid- patterns/Grid.tsx`
In-cell Control	Type × State (9)	= `InCellControl`	`grid- patterns/Grid.tsx`
Column Group Header	Style × Divider (4)	= `ColumnGroupHeader`	`grid- patterns/Grid.tsx`
Grid Column Header	Sort × State (9)	= `GridColumnHeader` (+ `grow` / `width` / `span`, `trailing`, `editable` date column)	`grid- patterns/Grid.tsx`
Collapsed Column Rail	State (2)	= `CollapsedColumnRail`	`grid- patterns/Grid.tsx`
Add Column Header	State (2)	= `AddColumnHeader`	`grid- patterns/Grid.tsx`
Grid Column Divider	Type (2)	= `GridColumnDivider`	`grid- patterns/Grid.tsx`
Chart Hover Card	—	= `ChartHoverCard`	`grid- patterns/Grid.tsx`
Cell History Popover	—	= `CellHistoryPopover` (chart is a child)	`grid- patterns/Grid.tsx`
—	—	+ `TaskPill` — pending-task pill in a grid row	`grid- patterns/Grid.tsx`

These sit **alongside** `Cell`, `Row` and `ColumnHeader`, not in place of them: use them for the financial-statement grids that need row hierarchy, value provenance and in-cell editing. See known gap 8.

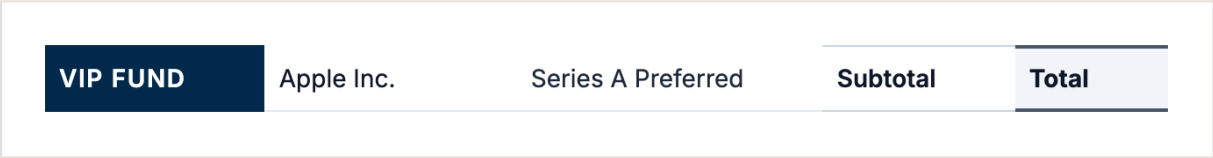
RowLabelCell

REACT RowLabelCell FILE src/components/grid-patterns/Grid.tsx FIGMA Row Label Cell
VARIANTS Type × Expand (8)

Row Label Cell — the first (sticky) column of a financial statement, cap table or allocation grid. Carries the hierarchy the base `Cell` cannot: Line Item (optionally expandable), Child (indented), Subtotal, Total and Group Header.

Tokens: Group Header = Background/Group Header + Text/On Group Header (stays navy in both modes); Total = Background/Subtle with Stroke/Strong rules.

Accessibility: `role="rowheader"` inside a `Row` of a `DataGrid` (an ARIA table). The expand control is a native `<button>` named "Expand " with `aria-expanded`; it is a normal tab stop — there is no tree-grid arrow-key model, because the table is not a `treegrid`.



Storybook · Data/GridPatterns › Row Label Types

PROPS

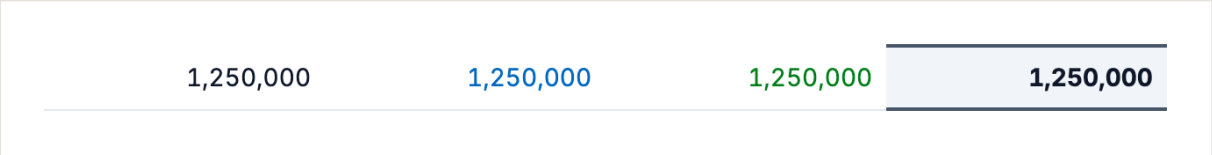
PROP	TYPE	DEFAULT	DESCRIPTION
expanded	`boolean`	—	Makes the row expandable; `true` = children showing.
onToggle	`(() => void)`	—	Called when the toggle is pressed. Required with `expanded`.
span	`number`	—	—
style	`CSSProperties`	—	—
type	`"total"` | `"line-item"` | `"child"` | `"subtotal"` | `"group-header"`	`period`	Row hierarchy. `group-header` is the full-width band (VIP Fund, Holding Co.).
toggleLabel	`string`	—	Names the row in the toggle's accessible name ("Expand Apple Inc."). Defaults to `children` when that is a string.
className	`string`	—	—

GridValueCell

REACT GridValueCell FILE src/components/grid-patterns/Grid.tsx FIGMA Grid Value Cell
VARIANTS Kind × State (12)

Grid Value Cell — value cell for editable financial grids. Right-aligned, tabular figures. Error always carries a message (colour alone never carries meaning, R8); Placeholder is an empty required input ("ENTER DATA"); Not Applicable is shaded and non- interactive.

Accessibility: `role="cell"` inside a `Row` of a `DataGrid`. With `onClick` the value is a native `<button>` (one tab stop per actionable cell); without it the cell is static text. Editing is the consumer's job — render an input as `children`. `data-state` / `data-kind` expose the state for styling hooks.



Storybook · Data/GridPatterns › Value Kinds

PROPS

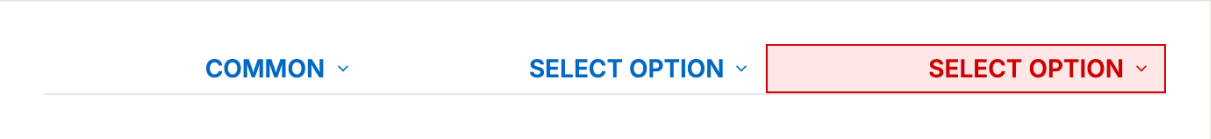
PROPPROP	TYPETYPE	DEFAULTDEFAULT	DESCRIPTIONDESCRIPTION
`span`	`number`	—	—
`style`	`CSSProperties`	—	—
`onClick`	`MouseEventHandler<HTMLButtonElement>`	—	Makes the value activatable (open its history, select it). The content is then wrapped in a native `<button>` inside the cell, so Enter / Space work.
`kind`	`"total" | "calculated" | "editable" | "sourced"`	`calculated`	calculated → Text/Primary · editable → Text/Editable · sourced → Text/Sourced · total → bold, ruled.
`state`	`"default" | "placeholder" | "error" | "focused" | "not- applicable"`	`default`	—
`hasComment`	`boolean`	—	Draws the corner flag used for cell notes.
`commentLabel`	`string`	`Has note`	Accessible name of the note flag. Default "Has note".
`errorMessage`	`string`	—	Required with `state="error"` — explain why (R8). Shown as the cell's title and read out as its description.
`placeholderText`	`string`	`Enter data`	What `state="placeholder"` shows. Default "Enter data".
`className`	`string`	—	—

InCellControl

REACT InCellControl FILE src/components/grid-patterns/Grid.tsx FIGMA In-cell Control
VARIANTS Type × State (9)

In-cell Control — borderless picker inside a grid cell: Select (security type, allocation method), Date (opens DatePicker), Currency (company currency pill). Text/Editable marks it as user input.

Accessibility: a `role="cell"` wrapper holds a native `<button>`, so the control keeps its button semantics. Its name is `label` plus the visible value ("Security type Common"), which satisfies label-in-name. Pass `aria-controls` (the popup's id) through; the ref and rest props go to the button. Focus return after the popup closes is the popup owner's job.



Storybook · Data/GridPatterns › In Cell Select

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	string	—	Accessible name of the field ("Security type"); the current value is appended to it.
type	"date" | "select" | "currency"	period	—
state	"default" | "error"	default	`error` paints the cell negative; pass `errorMessage` — colour alone never carries meaning (R8).
errorMessage	string	—	—
span	number	—	—
style	CSSProperties	—	Style of the cell wrapper (the control fills it).
children	ReactNode	Add column	Current value, or omit for the "Select option" prompt.
open	boolean	—	Whether the popup is open. Omit when no popup is wired — `aria-expanded` is then not announced.
className	string	—	—

22 · GRID PATTERNS

ColumnGroupHeader

REACT ColumnGroupHeader FILE src/components/grid-patterns/Grid.tsx FIGMA Column Group Header

Variants Style × Divider (4)

Column Group Header — second header tier spanning a run of columns (Projections over FY2025–27, LTM/NTM, Previous valuation…). Repeat the period rule down the body with `GridColumnDivider`.

Accessibility: `role="columnheader"` with `aria-colspan`, in the group row of a `DataGrid`.



Storybook · Data/GridPatterns › Group Headers

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
span required	`number`	—	Number of columns spanned — sets `aria-colspan`; width comes from layout.
style	`CSSProperties`	—	—
styleVariant	`"default"` | `"emphasis"`	`default`	`emphasis` for Projections-style groups.
periodDivider	`boolean`	—	Draws the 3px Stroke/Brand rule between actuals and projections.
className	`string`	—	—

22 · GRID PATTERNS

GridColumnHeader

REACT GridColumnHeader FILE src/components/grid-patterns/Grid.tsx FIGMA Grid Column Header
VARIANTS Sort × State (9)

Grid Column Header — interactive header for configurable portfolio grids: drag handle, sort indicator and an optional filter. Long labels truncate; put the full label in a Tooltip. (The old resize edge was decorative and has been removed — column widths come from the DataGrid tracks.)

Accessibility: `role="columnheader"` with `aria-sort` when sortable; sort, filter and reorder are separate native buttons.



Storybook · Data/GridPatterns › Column Header Interactive

PROPS

PROPSTORYBOOK · DATA/GRIDPATTERNS › GROUP HEADERS	TYPEPROPS	DEFAULTPROP	DESCRIPTIONTYPE
`grow`	`number`	—	Share of the grid's leftover width this column takes (DataGrid). Default 1.
`width`	`string`	—	A fixed column track instead of the header-sized one (DataGrid).
`span`	`number`	—	—
`style`	`CSSProperties`	—	—
`label`	`string`	—	Plain-text name of the column, used by the filter and reorder buttons ("Filter Revenue"). Defaults to `children` when that is a string.
`trailing`	`ReactNode`	—	Trailing content after the label, e.g. an editable-date marker.
`sort`	`"none" | "ascending" | "descending"`	`none`	—
`onSort`	`(() => void)`	—	Cycles none → ascending → descending. Omit for an unsortable column.
`draggable`	`boolean`	—	Shows the drag handle. On its own the handle is a decorative affordance — wire pointer reordering in the grid and pair it with `onMove`.
`onMove`	`((direction: "left" | "right") => void)`	—	Keyboard (and single-pointer) alternative to dragging, WCAG 2.5.7. With `draggable`, the handle becomes a button: focus it and press ArrowLeft / ArrowRight to move the column one place.
`onFilter`	`(() => void)`	—	Shows the in-header filter button.
`selected`	`boolean`	—	Active column (blue top tab). Exposed as `aria-current`.
`numeric`	`boolean`	—	—
`editable`	`boolean`	—	The column's values are user-editable dates: the label turns editable blue and a calendar glyph trails it.
`className`	`string`	—	—

22 · GRID PATTERNS

CollapsedColumnRail

REACT CollapsedColumnRail FILE src/components/grid-patterns/Grid.tsx FIGMA Collapsed Column Rail
VARIANTS State (2)

Collapsed Column Rail — vertical pill standing in for hidden columns ("+ 8 columns"). Its name is the visible text plus a hidden "show hidden columns" hint, so the visible label is contained in the name.



Storybook · Data/GridPatterns › Collapsed Rail

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
count required	`number`	—	Number of hidden columns.
onExpand required	`() => void`	—	—
className	`string`	—	—

22 · GRID PATTERNS

AddColumnHeader

REACT AddColumnHeader FILE src/components/grid-patterns/Grid.tsx FIGMA Add Column Header
VARIANTS State (2)

Add Column Header — trailing pseudo-header that opens the Add Columns modal. The ref and rest props go to the `columnheader` wrapper; the button inside is the control (`aria-haspopup="dialog"`, `aria-expanded` while `selected`).



Storybook · Data/GridPatterns › Add Column

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
onClickrequired	`() => void`	—	—
selected	`boolean`	—	The add-column affordance is the active target (e.g. its picker is open).
controls	`string`	—	The id of the dialog the button opens (`aria-controls`).
width	`string`	—	—
grow	`number`	—	—
style	`CSSProperties`	—	—
className	`string`	—	—

22 · GRID PATTERNS

GridColumnDivider

REACT GridColumnDivider FILE src/components/grid-patterns/Grid.tsx FIGMA Grid Column Divider
VARIANTS Type (2)

Grid Column Divider — full-height vertical rule between column groups. Purely decorative: hidden from assistive technology.



Storybook · Data/GridPatterns › Column Dividers

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
type	`"period"` | `"pinned"`	period	`period` = actuals | projections (Stroke/Brand); `pinned` = before a pinned total column (Stroke/Strong).

22 · GRID PATTERNS

ChartHoverCard

REACT ChartHoverCard FILE src/components/grid-patterns/Grid.tsx FIGMA Chart Hover Card

Chart Hover Card — hover read-out for Line / Bar / Waterfall charts. The swatch is passed in resolved (canvas colours come from `useChartTokens`).

Accessibility: `role="tooltip"` is a pointer read-out — the same data is in the chart's table alternative. To tie it to its trigger, give it an `id` and set `aria-describedby` on the trigger; the owner dismisses it on Escape.



Storybook · Data/GridPatterns › Hover Card

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`heading`required	`ReactNode`	—	The x value ("Dec 31, 2024").
`rows`required	`readonly ChartHoverRow[]`	—	One entry per series.

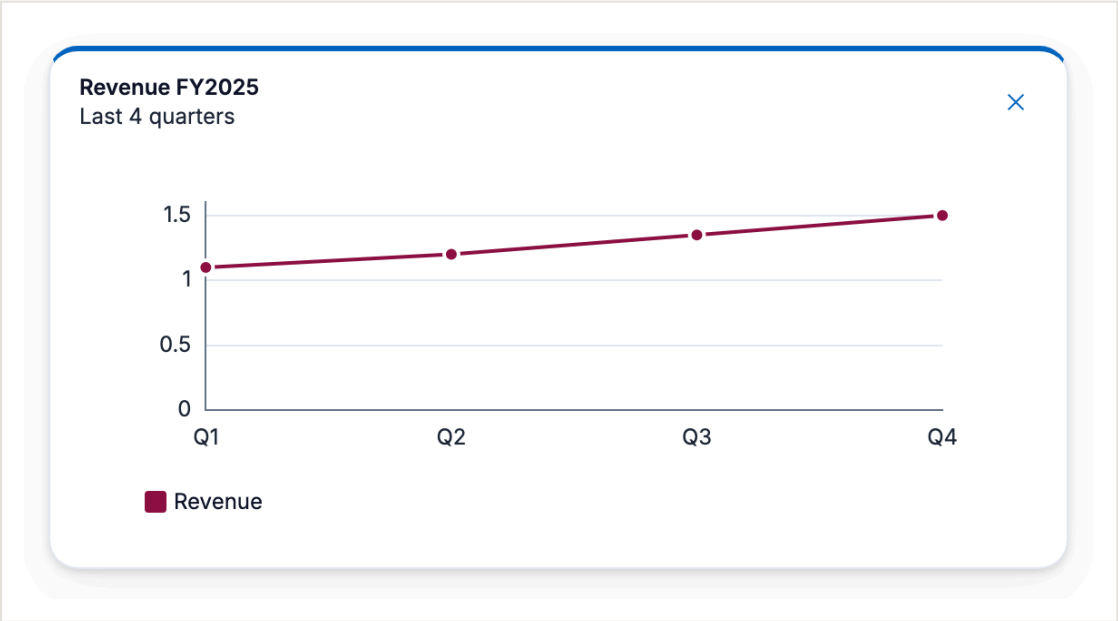
22 · GRID PATTERNS

CellHistoryPopover

REACT CellHistoryPopover FILE src/components/grid-patterns/Grid.tsx FIGMA Cell History Popover

Cell History Popover — opened from the trend icon on a value cell; shows that value over time. The caller owns positioning (anchor below-right of the cell) and mounts it only while open.

Accessibility: a non-modal `role="dialog"` named by its title (`aria-labelledby`). Focus moves in on mount (to the close button), Escape or a click outside calls `onClose`, Tab can leave, and focus returns to the opener when it unmounts.



Storybook · Data/GridPatterns › History Popover

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
onCloserequired	() => void	—	—
subtitle	ReactNode	—	—
children	ReactNode	Add column	The chart — normally a `LineChart`.
closeLabel	string	Close	Accessible name of the close button. Default "Close".
autoFocus	boolean	true	Move focus into the popover on mount and return it to the opener on unmount. The popover is mounted only while open, so this defaults to `true`; pass `false` to render it statically (a gallery) without taking focus. Escape and outside clicks only close it while this is on.

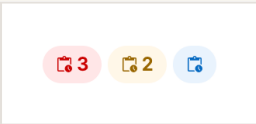
22 · GRID PATTERNS

TaskPill

REACT TaskPill FILE src/components/grid-patterns/Grid.tsx

Task Pill — a compact tinted pill in a grid row flagging pending tasks: an icon plus an optional count. The tint is `bg.*Subtle` with its matching text, and the icon and number carry the meaning with the label as the accessible name, so colour is never the

Dense grid furniture: 24px tall (Target/Dense), the documented exception to the 44px target. The ref points at the root, a `<button>` with `onClick`, otherwise a `<span role="img">`.



Storybook · Data/GridPatterns › Task Pill Tones

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label`**required**	`string`	—	Accessible name, **required** — the glyph and count are not the name ("3 overdue tasks", "Review requested"). Also the pill's tooltip text.
`count`	`number`	—	Pending-task count. Omit for a single marker with no number.
`tone`	`"warning"` | `"negative"` | `"brand"`	`warning`	`negative` overdue / blocked · `warning` due soon / needs review · `brand` open. Default `warning`.
`icon`	`ReactNode`	—	Glyph, passed through `Icon`. Default Material `pending_actions`.
`onClick`	`MouseEventHandler<HTMLButtonElement>`	—	Makes the pill a button (open the task list).

23 FILES & DOCUMENTS

COMPONENTS · 7 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
File Type Badge	Type (7)	= `FileTypeBadge` (+ + `fileKindOf`)	`files/Files.tsx`
File Row	State (3)	= `FileRow`	`files/Files.tsx`
Tree Item	Type × Level × State (12)	= `TreeItem`	`files/Files.tsx`
Page Stepper	—	= `PageStepper`	`files/Files.tsx`
Zoom Control	—	= `ZoomControl` (Material `zoom_in` / `fit_screen`)	`files/Files.tsx`
Document Viewer Header	—	= `DocumentViewerHeader`	`files/Files.tsx`
Scroll Hint Pill	Direction (2)	= `ScrollHintPill`	`files/Files.tsx`

23 · FILES & DOCUMENTS

FileTypeBadge

REACT FileTypeBadge FILE src/components/files/Files.tsx FIGMA File Type Badge VARIANTS Type (7)

File Type Badge — colour-coded extension. Bound to the File/* family: colour identifies a format, never a status. The extension text is always shown — colour alone is not enough (R8).



Storybook · Files/Files › Badge Kinds

PROPS

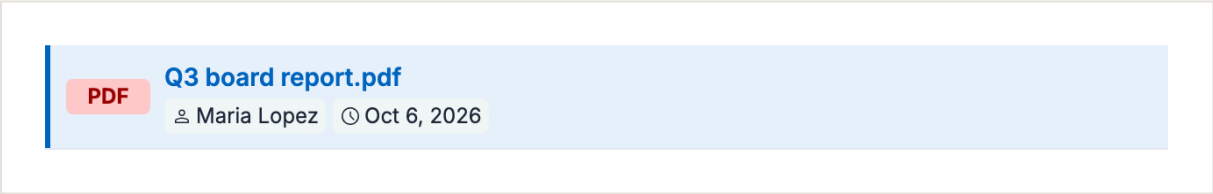
PROP	TYPE	DEFAULT	DESCRIPTION
file required	string	—	File name or extension ("report.pdf", "xlsx").

23 · FILES & DOCUMENTS

FileRow

REACT FileRow FILE src/components/files/Files.tsx FIGMA File Row VARIANTS State (3)

File Row — one document in a file list or folder.



Storybook · Files/Files › Row Selected File

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
name required	string	—	—
meta	readonly { icon?: ReactNode; label: ReactNode; }[]	—	Meta chips: uploader, date, linked area.
selected	boolean	—	The row is the open / current file. Exposed as `aria-current`. This is the "which file am I looking at" state; `checked` is the separate bulk-selection state.
checked	boolean	—	Bulk-selection checkbox, controlled. The checkbox shows when this, `defaultChecked` or `onCheckedChange` is given.

PROP	TYPE	DEFAULT	DESCRIPTION
`defaultChecked`	`boolean`	—	Bulk-selection checkbox, uncontrolled initial value.
`onCheckedChange`	`((checked: boolean) => void)`	—	—
`draggable`	`boolean`	—	Shows the drag handle (move between folders).
`onMove`	`(() => void)`	—	Keyboard / assistive alternative to dragging (WCAG 2.5.7). When given with `draggable`, the handle becomes a "Move" button that calls this.
`onOpen`	`(() => void)`	—	—
`onDownload`	`(() => void)`	—	—
`onMenu`	`(() => void)`	—	Opens a Context Menu (Rename, Move, Link to..., Delete).
`downloadIcon`	`ReactNode`	—	Glyph for the download action (default `download`), e.g. `icons.CloudDownload`.

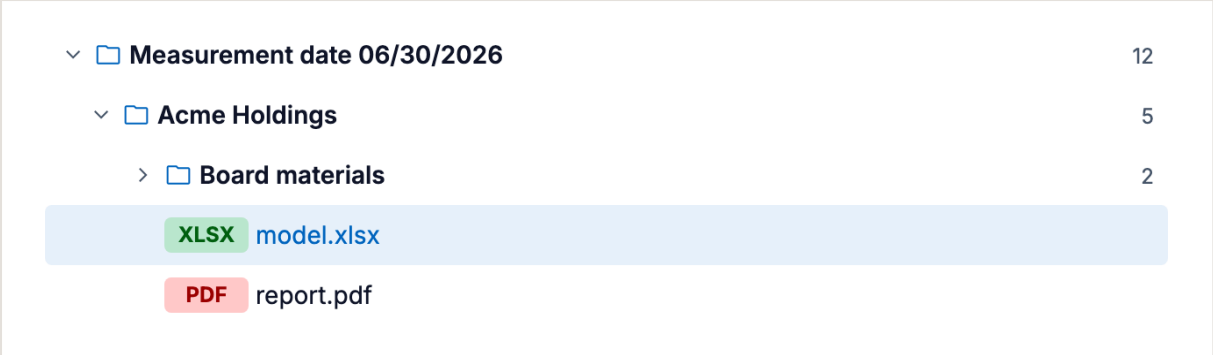
23 · FILES & DOCUMENTS

TreeItem

REACT TreeItem FILE src/components/files/Files.tsx FIGMA Tree Item VARIANTS Type × Level × State (12)

Tree Item — row of a folder/file tree (Documents: measurement date → company → subfolder → file). Render inside an element with `role="tree"` (give it an `aria-label`).

The row itself is the single focusable `treeitem` (roving tabindex: one tab stop per tree). Keyboard, per the APG tree pattern: Up/Down move between visible items, Right expands a closed folder or enters an open one, Left collapses an open folder or moves to the parent, Home/End jump to first/last, Enter/Space activate, and typing a letter jumps to the next item starting with it. `aria-level`, `aria-posinset` and `aria-setsize` are derived from the rendered tree unless you pass `posInSet` / `setSize`. A tree here is a flat run of rows linked by `level`; wrap a folder's children in `role="group"` if you nest them in the DOM.



Storybook · Files/Files › Tree

PROPS

PROP—	TYPEBULK-SELECTION CHECKBOX, CONTROLLED. THE CHECKBOX SHOWS WHEN THIS,	DEFAULT	DESCRIPTIONDEFAULTCHECKED
`type`	`"file"` | `"folder"`	`button`	—
`level`	`number`	`0`	0-based depth; sets the indent and `aria-level`.
`expanded`	`boolean`	—	Folders only.
`onToggle`	`(() => void)`	—	Folders: expand/collapse (click, Enter, Space, Left/Right).
`count`	`ReactNode`	—	Folders only — document count shown after the name.
`fileName`	`string`	—	Files only — the file name for the badge.
`selected`	`boolean`	—	—
`onSelect`	`(() => void)`	—	Files: fires on click, Enter or Space. (Folders toggle instead.)
`icon`	`ReactNode`	—	Folders only — replaces the folder glyph, e.g. `icons.Domain` for a company node.
`posInSet`	`number`	—	Overrides the position among siblings. Computed from the surrounding tree when omitted.
`setSize`	`number`	—	Overrides the sibling count. Computed from the surrounding tree when omitted.

23 · FILES & DOCUMENTS

PageStepper

REACT PageStepper FILE src/components/files/Files.tsx FIGMA Page Stepper

Page Stepper — page navigation for a paged document. Type a number and press Enter (or leave the field) to jump; out-of- range numbers clamp, non-numbers revert. The new page is announced politely and focus stays in the field.



Storybook · Files/Files › Stepper First Page

PROPS

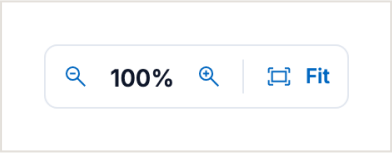
PROP	TYPE	DEFAULT	DESCRIPTION
pagerequired	number	—	—
totalrequired	number	—	—
onChangerequired	(page: number) => void	—	—

23 · FILES & DOCUMENTS

ZoomControl

REACT ZoomControl FILE src/components/files/Files.tsx FIGMA Zoom Control

Zoom Control — zoom out / level / zoom in / Fit (Material `zoom_in`, `fit_screen`). The level is announced politely.



Storybook · Files/Files › Zoom

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	number	—	Percent.
onChangerequired	(value: number) => void	—	—
onFit	(() => void)	—	—
steps	readonly number[]	[50, 75, 100, 125, 150, 200]	—

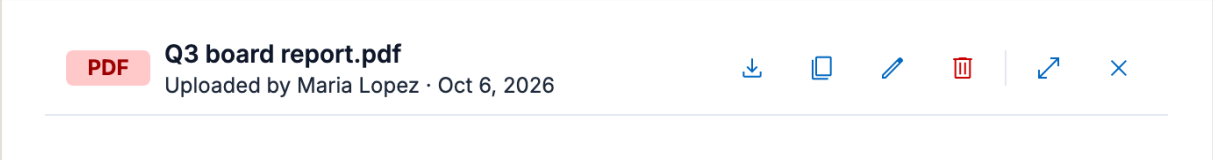
23 · FILES & DOCUMENTS

DocumentViewerHeader

REACT DocumentViewerHeader FILE src/components/files/Files.tsx FIGMA Document Viewer Header

Document Viewer Header — file badge, name (a heading) and meta, then file actions (Download, Copy link, Rename, Delete — negative) and pane actions (Expand, Close). Only the actions you pass are rendered; file actions are named

for the file ("Download report.pdf").



Storybook · Files/Files › Viewer Header

PROPS

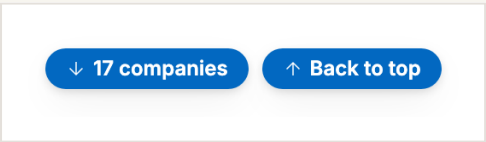
PROP	TYPE	DEFAULT	DESCRIPTION
fileNamerequired	string	—	—
meta	ReactNode	—	—
onDownload	(() => void)	—	—
onCopyLink	(() => void)	—	—
onRename	(() => void)	—	—
onDelete	(() => void)	—	—
onExpand	(() => void)	—	—
onClose	(() => void)	—	—
headingLevel	1 | 2 | 3 | 4 | 5 | 6	2	Heading level of the file name. Default 2.

23 · FILES & DOCUMENTS

ScrollHintPill

REACT ScrollHintPill FILE src/components/files/Files.tsx FIGMA Scroll Hint Pill
VARIANTS Direction (2)

Scroll Hint Pill — how much of a long list is out of view ("↓ 17 companies").



Storybook · Files/Files › Scroll Hints

PROPS

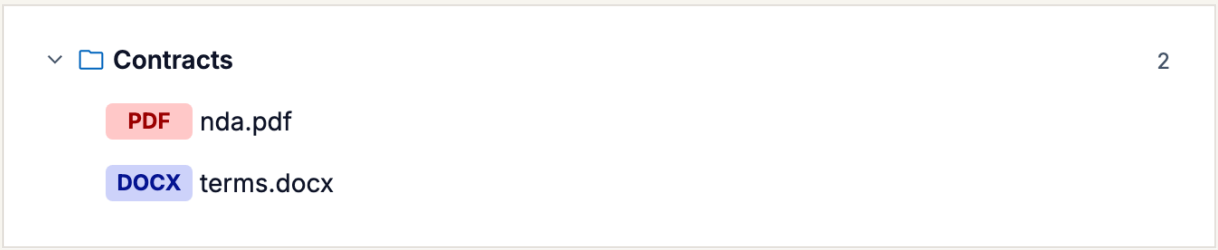
PROP	TYPE	DEFAULT	DESCRIPTION
direction	"up" | "down"	down	—

23 · FILES & DOCUMENTS

TreeGroup

REACT TreeGroup FILE src/components/files/Files.tsx

Wraps a folder's child rows when you nest them in the DOM; `role="group"` per the tree pattern.



Storybook · Files/Files › Tree Nested

PROPS

No component-specific props; accepts the native element attributes.

24 COMPONENTS · 5 COMPONENTS FEEDBACK PATTERNS

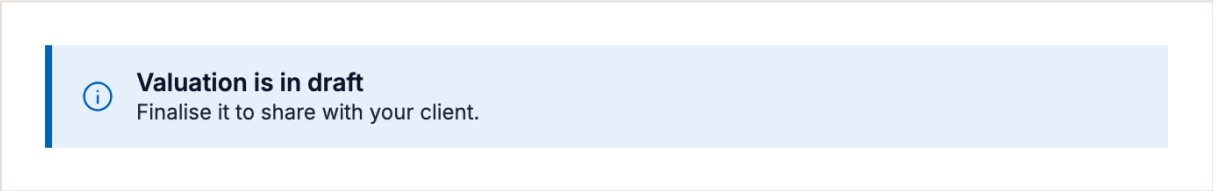
FIGMA	VARIANTS	REACT	FILE
Banner	Style (5)	= `Banner`	`feedback-patterns/Status.tsx`
Spinner	Size (3)	= `Spinner`	`feedback-patterns/Status.tsx`
Progress Ring	Value (5)	= `ProgressRing` (continuous `value` / `max`)	`feedback-patterns/Status.tsx`
Data Freshness	State (3)	= `DataFreshness`	`feedback-patterns/Status.tsx`
Save State	State (5)	= `SaveState`	`feedback-patterns/Status.tsx`

Banner

REACT Banner FILE src/components/feedback-patterns/Status.tsx FIGMA Banner VARIANTS Style (5)

Banner — page-level, full-width message under the page header. Unlike `Alert` (boxed inside a section) it spans the page, can carry an issue list that links to each problem, and one action. One Banner per page. The tone is carried by an icon and a 4px left rule, never colour alone (R8).

Accessibility: `negative` is `role="alert"`; every other tone is a polite `role="status"` (a persistent page message is not an interruption).



Storybook · Feedback/Banner › Info

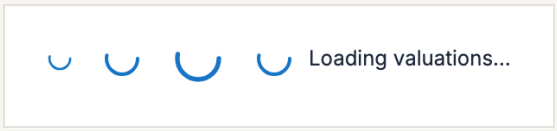
PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
tone	`"positive"` | `"warning"` | `"negative"` | `"info"` | `"neutral"`	info	—
action	ReactNode	—	A single action button ("Edit common profile").
issues	readonly BannerIssue[]	—	Error summary: each issue links to where it is.
onDismiss	(() => void)	—	—
dismissLabel	string	Dismiss	Accessible name of the dismiss button. Default "Dismiss".

Spinner

REACT Spinner FILE src/components/feedback-patterns/Status.tsx FIGMA Spinner VARIANTS Size (3)

Spinner — indeterminate loading for work with no known duration (preparing a PDF preview, refreshing market data). For layout-shaped loading use `Skeleton`; for known progress `ProgressBar` or `ProgressRing`. Pulses instead of spinning under prefers-reduced-motion.



Storybook · Feedback/Banner › Spinners

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`size`	`"s"` | `"m"` | `"l"`	`m`	—
`label`	`string`	—	Short visible label ("Preparing preview..."). Also the accessible name.

---

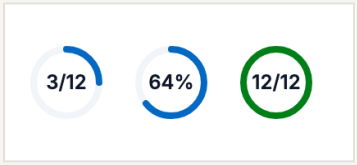
24 · FEEDBACK PATTERNS

ProgressRing

REACT ProgressRing FILE src/components/feedback-patterns/Status.tsx FIGMA Progress Ring

VARIANTS Value (5)

Progress Ring — circular completion for a count-based goal next to a page title (requests sent / answered). Diameter = Sizing/Progress Ring/M. Turns positive at 100%.



Storybook · Feedback/Banner › Progress Rings

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`value`**required**	`number`	—	Completed count.
`max`**required**	`number`	—	Total. `0` renders "0/0" and an empty ring.
`label`**required**	`string`	—	What is being counted ("information requests answered").
`display`	`"count"` | `"percent"`	`count`	Centre text: `count` ("3/5") or `percent` ("60%").

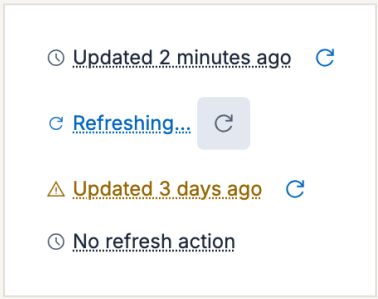
24 · FEEDBACK PATTERNS

DataFreshness

REACT DataFreshness FILE src/components/feedback-patterns/Status.tsx FIGMA Data Freshness
VARIANTS State (3)

Data Freshness — how current externally sourced data is (Daily NAV market data, Capital IQ), with a refresh action. Always include the time zone.

Accessibility: the state is spoken, not just tinted — a visually hidden "Stale." / "Refreshing." prefix sits in a polite `role="status"` span. The refresh button stays mounted while refreshing (`aria-disabled`) so keyboard focus is not dropped.



Storybook · Feedback/Banner › Freshness

PROPS

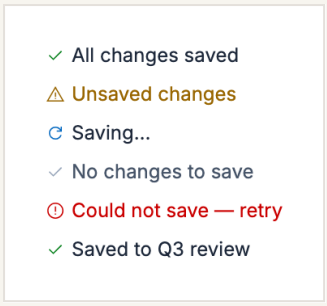
PROP	TYPE	DEFAULT	DESCRIPTION
childrenrequired	ReactNode	—	"Market data as of Sep 22, 2026, 2:00 PM CST".
state	`"current" | "refreshing" | "stale"`	current	—
onRefresh	`(() => void)`	—	—
refreshLabel	string	Refresh data	Accessible name of the refresh button. Default "Refresh data".

24 · FEEDBACK PATTERNS

SaveState

REACT SaveState FILE src/components/feedback-patterns/Status.tsx FIGMA Save State VARIANTS State (5)

Save State — inline status next to Save so the user knows whether edits are persisted. Polite `role="status"`; the `error` state is `role="alert"`.



Storybook · Feedback/Banner › Save States

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`state`	`"error" | "saved" | "unsaved" | "saving" | "no-changes"`	`current`	—
`children`	`ReactNode`	—	Overrides the default copy.

COMPONENTS · 4 COMPONENTS

25 OVERLAYS & PANELS

FIGMA	VARIANTS	REACT	FILE
Confirmation Dialog	Tone (2)	= `ConfirmationDialog` (built on `Modal`)	`overlays/Overlays.tsx`
Setting Row	State (2)	= `SettingRow`	`overlays/Overlays.tsx`
Notification Center	View (2)	= `NotificationCenter`	`overlays/Overlays.tsx`
Row Action Toolbar	—	= `RowActionToolbar`	`overlays/Overlays.tsx`

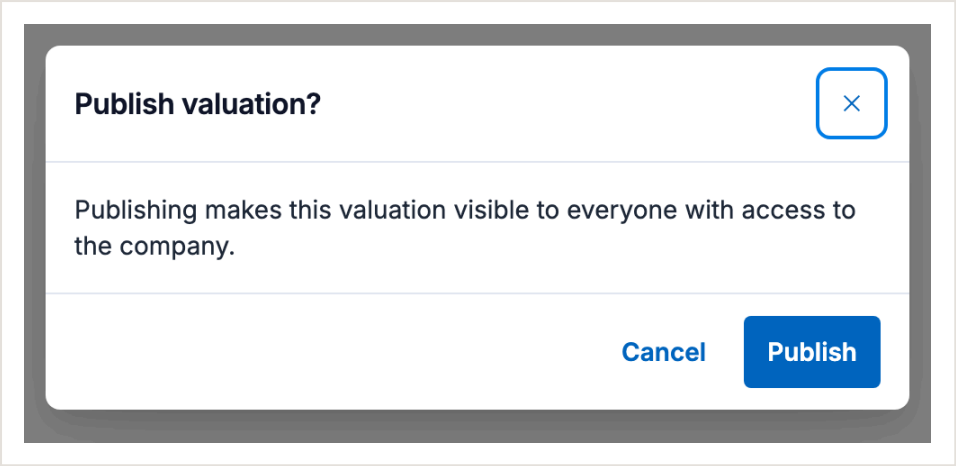
25 · OVERLAYS & PANELS

ConfirmationDialog

REACT ConfirmationDialog FILE src/components/overlays/Overlays.tsx FIGMA Confirmation Dialog
VARIANTS Tone (2)

Confirmation Dialog — confirm a consequential action (leave with unsaved changes, delete a group or view, revoke a token, apply to an open NAV day). Built on `Modal`, so it traps focus, closes on Escape and returns focus. The scrim does not dismiss it: the user must choose.

Accessibility: `role="alertdialog"` when destructive, described by its message. A destructive dialog opens on Cancel, the safe choice.



Storybook · Overlays/ConfirmationDialog › Default

PROPS

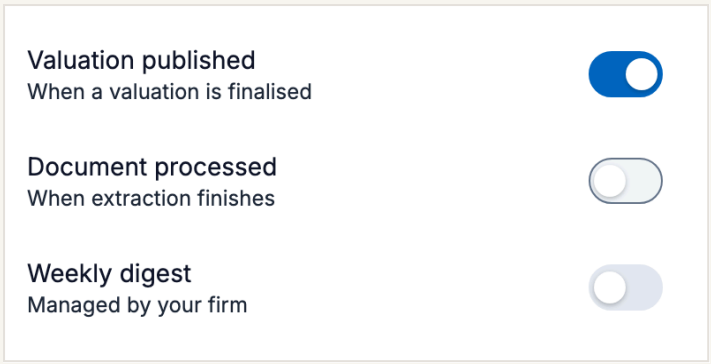
PROP	TYPE	DEFAULT	DESCRIPTION
`open` required	`boolean`	—	—
`title` required	`string`	—	A question: "Leave without saving?", "Delete this group?".
`confirmLabel` required	`string`	—	The verb, repeated: "Leave anyway", "Delete group". Never "OK".
`onConfirm` required	`() => void`	—	—
`onCancel` required	`() => void`	—	—
`cancelLabel`	`string`	`Cancel`	—
`destructive`	`boolean`	—	Destructive: negative confirm button and the warning glyph.
`detail`	`ReactNode`	—	Optional inset context ("3 cells edited · 1 method added").
`busy`	`boolean`	—	Blocks double-submits while the action runs; Escape is ignored too.

25 · OVERLAYS & PANELS

SettingRow

REACT SettingRow FILE src/components/overlays/Overlays.tsx FIGMA Setting Row VARIANTS State (2)

Setting Row — label (+ description) with a trailing Switch, for preferences that apply immediately. If the setting needs Save, use CheckboxItem. `ref` points at the switch input.



Storybook · Overlays/ConfirmationDialog › SettingRow

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	ReactNode	—	—
description	ReactNode	—	—
checked	boolean	—	Controlled value. Omit it (and use `defaultChecked`) to let the row own it.
defaultChecked	boolean	false	—
onChange	((checked: boolean) => void)	—	—
disabled	boolean	—	—
className	string	—	—

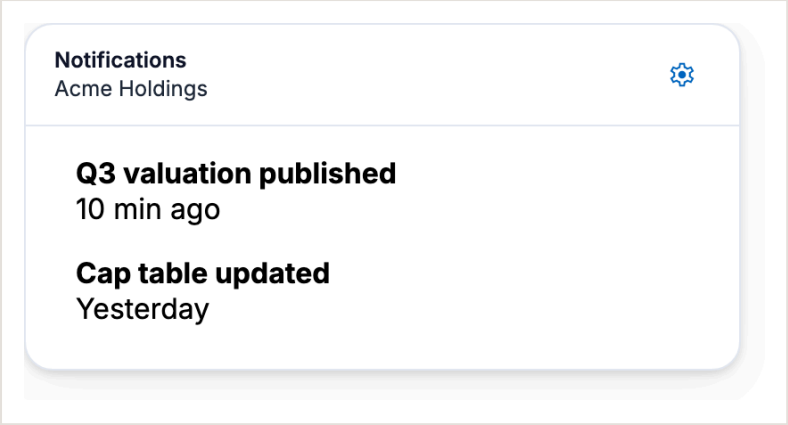
25 · OVERLAYS & PANELS

NotificationCenter

REACT NotificationCenter FILE src/components/overlays/Overlays.tsx FIGMA Notification Center
VARIANTS View (2)

Notification Center — popover under the Notification bell. The gear toggles between the inbox and notification settings.

Accessibility: a non-modal `dialog` named by its heading. Inbox items are in a polite live region so new arrivals are announced.



Storybook · Overlays/ConfirmationDialog › NotificationCenter

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`scope`	`ReactNode`	—	Scope shown under the title ("Spatical Ventures").
`view`	`"inbox" | "settings"`	—	Controlled view. Omit it (and use `defaultView`) to let the popover own it.
`defaultView`	`"inbox" | "settings"`	`inbox`	—
`onViewChange`	`((view: NotificationView) => void)`	—	—
`delivery`	`ReactNode`	—	Delivery switches (SettingRows) shown in both views.
`children`	`ReactNode`	—	Inbox items; empty renders the "all caught up" state.
`settings`	`ReactNode`	—	Settings view body: per-type SettingRows, company scope filter.
`onClose`	`(() => void)`	—	Makes it a live non-modal popover: focus moves in on mount, Escape and a click outside call `onClose`, and focus returns to the opener. Omit it when the surface is shown statically (galleries) or another container owns dismissal — nothing then takes focus.
`triggerRef`	`RefObject<HTMLElement | null>`	—	The bell that opened it, so clicking it is not an "outside" click.

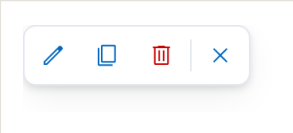
25 · OVERLAYS & PANELS

RowActionToolbar

REACT RowActionToolbar FILE src/components/overlays/Overlays.tsx FIGMA Row Action Toolbar

Row Action Toolbar — compact icon toolbar at the right edge of a selected table row. Every button is labelled. Prefer a Context Menu above four actions or when labels matter.

Keyboard: one Tab stop; Left/Right move between buttons, Home/End jump, Escape calls `onClose`.



Storybook · Overlays/ConfirmationDialog › RowActionToolbar

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`actions` **required**	`readonly RowAction[]`	—	—
`label` **required**	`string`	—	What the actions apply to ("Row actions for jane@firm.com").
`onClose`	`(() => void)`	—	Shows a Close button; Escape inside the toolbar calls it too.

26 SETTINGS & ADMIN

COMPONENTS · 13 COMPONENTS

FIGMA	VARIANTS	REACT	FILE
Key-Value Row	Layout (2)	= `KeyValueRow`	`admin/Admin.tsx`
Version History Item	State (2)	= `VersionHistoryItem` (+ `currentLabel`)	`admin/Admin.tsx`
Permission Matrix Row	Type (2)	= `PermissionMatrixRow`	`admin/Admin.tsx`
Role Selector	State (2)	= `RoleSelector`	`admin/Admin.tsx`
Profile Header	—	= `ProfileHeader`	`admin/Admin.tsx`
Filter Bar	—	= `FilterBar`	`admin/Admin.tsx`

FIGMA	VARIANTS	REACT	FILE
Directory Group	—	= `DirectoryGroup`	`admin/Admin.tsx`
Product Tile	Product (4)	= `ProductTile`	`admin/Admin.tsx`
Firm Switcher Tile	State (3)	= `FirmSwitcherTile`	`admin/Admin.tsx`
Page Task Header	—	= `PageTaskHeader`	`admin/Admin.tsx`
Code Grid	—	= `CodeGrid`	`admin/Admin.tsx`
Rich Text Toolbar	—	= `RichTextToolbar`	`admin/Admin.tsx`
App Footer	—	= `AppFooter`	`admin/Admin.tsx`

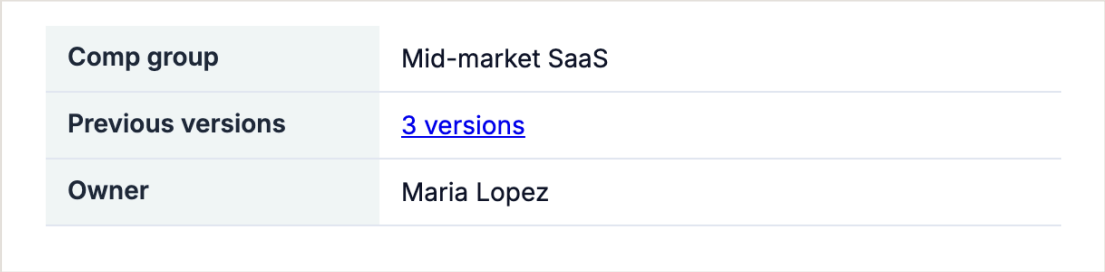
Glyphs. These components render glyphs from `icon/glyphs.tsx` (Plus, MoreVertical, Trash, Copy, Folder…), which are now aliases onto the Material Symbols set (`icons`). Every one that shows an icon also takes an icon prop — pass any `icons.*` glyph to change it. Figma still draws these from SDS_Main, so a glyph can differ between the two (known gap 4).

26 · SETTINGS & ADMIN

KeyValueRow

REACT KeyValueRow FILE src/components/admin/Admin.tsx FIGMA Key-Value Row VARIANTS Layout (2)

Key-Value Row — one pair in a description list (comp group name / previous versions, company information). Render rows inside a `KeyValueList` (`<dl>`); they stack with no gap. For a single header metric use InformationLabel.



Storybook · Admin/Admin › Key Value Inline

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`children` **required**	`ReactNode`	—	Text, a Link, an InlineEdit or an InlinePicker.
`label`	`ReactNode`	`Backup codes`	—
`layout`	`"inline"` | `"stacked"`	`inline`	`inline` = shaded key column; `stacked` = key above value for narrow cards.

26 · SETTINGS & ADMIN

VersionHistoryItem

REACT VersionHistoryItem FILE src/components/admin/Admin.tsx FIGMA Version History Item
VARIANTS State (2)

Version History Item — one entry of a settings/version timeline, newest first. Render inside a `VersionHistoryList`.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
rangerequired	ReactNode	—	Effective range ("Aug 24, 2026 5:01 PM — Present").
meta	ReactNode	—	Who and why ("Changed by Analyst · Firm template change").
current	boolean	—	Marks the live entry: shows the chip and sets `aria- current`.
currentLabel	ReactNode	Current	Text of the current-entry chip. Default "Current" ("Current · In use by 3 companies").
action	ReactNode	—	e.g. a tertiary "Restore" Button on past entries.

26 · SETTINGS & ADMIN

PermissionMatrixRow

REACT PermissionMatrixRow FILE src/components/admin/Admin.tsx FIGMA Permission Matrix Row
VARIANTS Type (2)

Permission Matrix Row — entities × access levels (Edit, View). Render inside a `PermissionMatrix`.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
label	string	Backup codes	The entity or group name. A plain string because it also names the checkboxes ("Edit VIP Fund", "View VIP Fund").
type	"group" | "entity"	button	`group` is a section row that captions the columns; `entity` carries the checkboxes.
edit	boolean	—	Edit access, controlled.
view	boolean	—	View access, controlled.
defaultEdit	boolean	false	—
defaultView	boolean	false	—
onChange	((next: { edit: boolean; view: boolean; }) => void)	—	Edit implies View: turning Edit on reports `view: true` too.

26 · SETTINGS & ADMIN

RoleSelector

REACT RoleSelector FILE src/components/admin/Admin.tsx FIGMA Role Selector VARIANTS State (2)

Role Selector — compact pill that shows and changes a user's firm role. It is a menu button (`aria-haspopup="menu"`): pair it with a MenuPanel and pass that panel's id as `aria-controls` .

Storybook · Admin/Admin › Role Selector Closed

PROPS

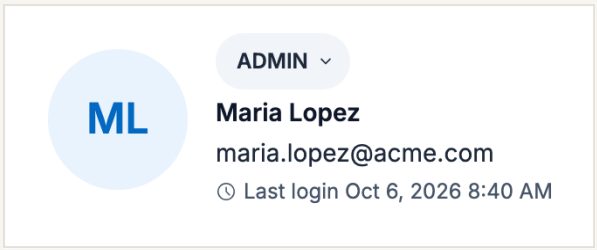
PROP	TYPE	DEFAULT	DESCRIPTION
rolerequired	string	—	The user's current role, as text.
open	boolean	false	Whether the menu it controls is open.

26 · SETTINGS & ADMIN

ProfileHeader

REACT ProfileHeader FILE src/components/admin/Admin.tsx FIGMA Profile Header

Profile Header — top of a user detail panel (User Management). The avatar is decorative; the name is the heading.



Storybook · Admin/Admin › Profile

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
name required	ReactNode	—	—
email	ReactNode	—	—
lastLogin	ReactNode	—	—
avatarSrc	string	—	—
initials	string	—	—
role	ReactNode	—	Usually a RoleSelector.
headingLevel	1 | 2 | 3 | 4 | 5 | 6	1	Heading level of the name. Default 2.

26 · SETTINGS & ADMIN

FilterBar

REACT FilterBar FILE src/components/admin/Admin.tsx FIGMA Filter Bar

Filter Bar — row of filters above a list or log (Audit logs, Documents, users). A labelled group, not a search landmark.

PROPS

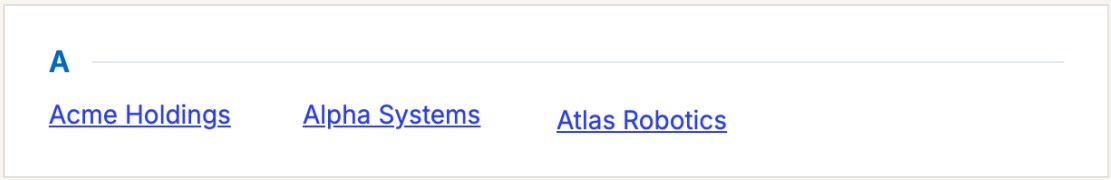
PROP	TYPE	DEFAULT	DESCRIPTION
children required	ReactNode	—	Filter controls: Select, DatePicker, Input with search, FilterDropdown.
label	ReactNode	Backup codes	Visible caption; also the group's accessible name.
action	ReactNode	—	Apply / Refresh button.

26 · SETTINGS & ADMIN

DirectoryGroup

REACT DirectoryGroup FILE src/components/admin/Admin.tsx FIGMA Directory Group

Directory Group — one letter of the A–Z company directory on the portfolio home.



Storybook · Admin/Admin › Directory

PROPS

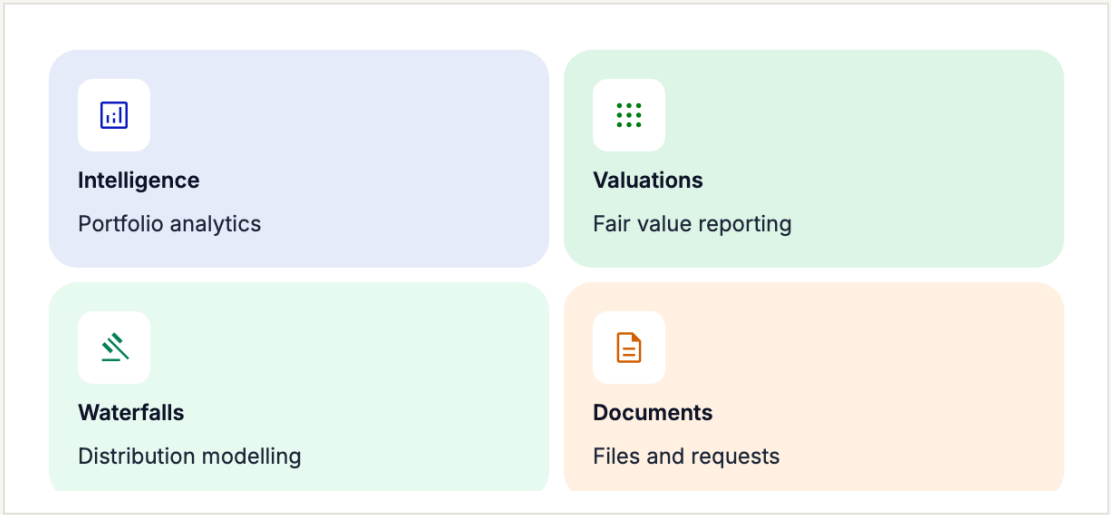
PROP	TYPE	DEFAULT	DESCRIPTION
letterrequired	`string`	—	The letter ("A").
entriesrequired	`readonly { label: ReactNode; href?: string | undefined; onClick?: (() => void) | undefined; }[]`	—	—
groupLabel	`string`	—	Accessible name of the section. Default `Companies starting with <letter>`.

26 · SETTINGS & ADMIN

ProductTile

REACT ProductTile FILE src/components/admin/Admin.tsx FIGMA Product Tile VARIANTS Product (4)

Product Tile — entry card for one Scalar product. Bound to the Product/* identity family (identity, not status). Icon well = Sizing/Icon Well/M.



Storybook · Admin/Admin › Product Tiles

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
product required	`"intelligence" | "valuations" | "waterfalls" | "documents"`	—	—
title required	`ReactNode`	—	—
icon required	`ReactNode`	—	The product's SDS_Main glyph (tinted by the tile). Decorative: the title is the name.
description	`ReactNode`	—	—
href	`string`	—	Renders an anchor when given, otherwise a button.

26 · SETTINGS & ADMIN

FirmSwitcherTile

REACT FirmSwitcherTile FILE src/components/admin/Admin.tsx FIGMA Firm Switcher Tile
VARIANTS State (3)

Firm Switcher Tile — square firm logo in the firm switcher strip.



Storybook · Admin/Admin › Firm Switcher

PROPS

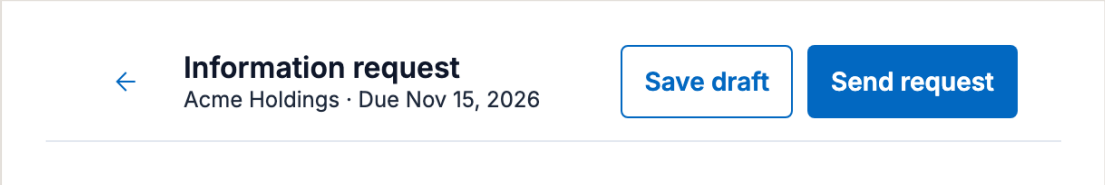
PROP	TYPE	DEFAULT	DESCRIPTION
name required	`string`	—	Firm name — the accessible name.
logoSrc	`string`	—	—
initials	`string`	—	Shown when there is no logo.
selected	`boolean`	—	The active firm. Exposed as `aria-current` (one tile of the strip is current).

26 · SETTINGS & ADMIN

PageTaskHeader

REACT PageTaskHeader FILE src/components/admin/Admin.tsx FIGMA Page Task Header

Page Task Header — header for a focused, full-page task (Information request editor). Renders a `div`, not `<header>`, so it never adds a second banner landmark.



Storybook · Admin/Admin › Task Header

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
titlerequired	ReactNode	—	—
onBackrequired	() => void	—	—
subtitle	ReactNode	—	—
backLabel	string	Back	Accessible name of the back button, ideally with its destination ("Back to requests"). Default "Back".
headingLevel	1 | 2 | 3 | 4 | 5 | 6	1	Heading level of the title. Default 1.
actions	ReactNode	—	Secondary then primary Buttons.

26 · SETTINGS & ADMIN

CodeGrid

REACT CodeGrid FILE src/components/admin/Admin.tsx FIGMA Code Grid

Code Grid — one-time secrets shown once (2FA backup codes). Masked codes read as "Hidden code"; toggling is announced.

PROPS

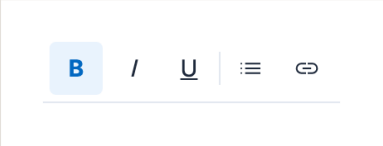
PROP	TYPE	DEFAULT	DESCRIPTION
codesrequired	readonly string[]	—	—
masked	boolean	false	Mask the codes. Show real codes only right after generation.
actions	ReactNode	—	Download / Print Buttons.
label	string	Backup codes	Accessible name of the list.

26 · SETTINGS & ADMIN

RichTextToolbar

REACT RichTextToolbar FILE src/components/admin/Admin.tsx FIGMA Rich Text Toolbar

Rich Text Toolbar — formatting bar for note editors (Workspace Drawer → Notes). An APG toolbar: one tab stop, Left/Right move between buttons (wrapping), Home/End jump to the ends. Named "Formatting" unless you pass `aria-label`.



Storybook · Admin/Admin › Rich Text

PROPS

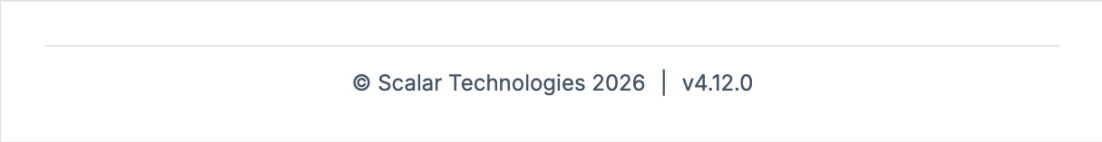
PROP	TYPE	DEFAULT	DESCRIPTION
onTogglerequired	`(format: RichTextFormat) => void`	—	—
active	`readonly RichTextFormat[]`	`[]`	—
labels	`Partial<Record<RichTextFormat, string>>`	—	Button names, for localisation. Defaults are English.

26 · SETTINGS & ADMIN

AppFooter

REACT AppFooter FILE src/components/admin/Admin.tsx FIGMA App Footer

App Footer — copyright and app version at the end of account/settings pages.



Storybook · Admin/Admin › Footer

PROPS

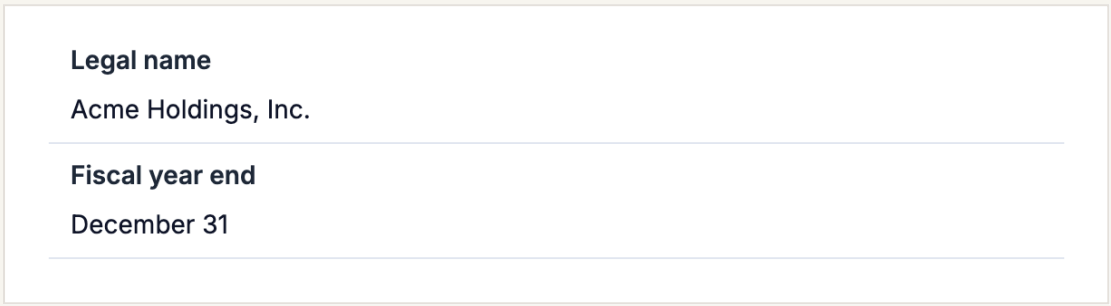
PROP	TYPE	DEFAULT	DESCRIPTION
children	`ReactNode`	—	Replaces the default copyright line.
version	`string`	—	—
year	`number`	—	Year in the default copyright line. When omitted the year is filled in after mount, so server and client markup match around New Year.

26 · SETTINGS & ADMIN

KeyValueList

REACT KeyValueList FILE src/components/admin/Admin.tsx

Key-Value List — the `<dl>` that `KeyValueRow`s belong in (a `div` row is only valid inside one).



Storybook · Admin/Admin › Key Value Stacked

PROPS

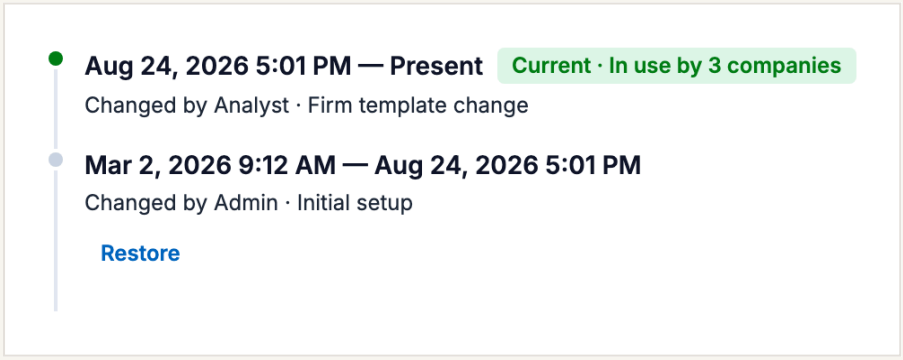
No component-specific props; accepts the native element attributes.

26 · SETTINGS & ADMIN

VersionHistoryList

REACT VersionHistoryList FILE src/components/admin/Admin.tsx

Version History List — the `<ol>` (newest first) that `VersionHistoryItem`s belong in.



Storybook · Admin/Admin › Version History

PROPS

No component-specific props; accepts the native element attributes.

26 · SETTINGS & ADMIN

PermissionMatrix

REACT `PermissionMatrix` FILE `src/components/admin/Admin.tsx`

Permission Matrix — the `role="table"` that `PermissionMatrixRow`s sit in. Carries a visually hidden header row (Name / Edit / View) so each checkbox has column context. Requires an accessible name.

PROPS

No component-specific props; accepts the native element attributes.



COMPONENTS · 4 COMPONENTS

WORKBOARD

FIGMA	VARIANTS	REACT	FILE
Action Tag	Tone × Urgency (4 × 3)	= `ActionTag`	`workboard/Workboa rd.tsx`
— (prototype `ActionsSelect`)	Tone × Urgency × Size	+ `ActionPill`	`workboard/Workboa rd.tsx`
Due Pill	Urgency × Size	= `DueDatePill`	`workboard/Workboa rd.tsx`
— (prototype `StatusDropdown`)	—	+ `StepMenu` + `StepMenuSection` / `StepMenuLink` / `StepMenuToggle` / `StepMenuAction`	`workboard/Workboa rd.tsx`

Ported from the Workboard prototype (`WORKBOARD_PROTOTYPE`, workboard-v2 Figma `mZ0cFiczRr3UCglYIO6Ot0`). Alias tones only — the proposed Blue / Pink / Cyan / Orange pill tokens were not adopted.

WORKBOARD

ActionTag

REACT ActionTag FILE src/components/workboard/Workboard.tsx FIGMA Action Tag
VARIANTS Tone × Urgency (4 × 3)

Action Tag — the read-only face of a workflow step: an open circle and the step's name in the step's tone. Use it where the step is shown but cannot be acted on (calendar cards, timeline bars, history). The acting version is `ActionPill`.

Figma: workboard-v2 `Action Tag` (7 tones × 3 urgencies; here 4 alias tones).

Accessibility: Today and Overdue append visually hidden words, so the urgency never rests on colour alone (R8).



Storybook · Workboard/Action Pill › Action Tag

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
children required	ReactNode	—	The step's name.
tone	`"positive"` | `"warning"` | `"ai"` | `"brand"`	`default`	—
urgency	`"none"` | `"today"` | `"upcoming"` | `"overdue"`	`upcoming`	Upcoming is a tinted outline, Today is the solid tone, Overdue is solid red. Default `upcoming`.
size	`"s"` | `"m"`	`m`	Default `s` (grid).

WORKBOARD

ActionPill

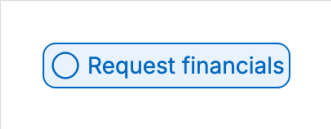
REACT ActionPill FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `ActionsSelect`)

Variants Tone × Urgency × Size

Action Pill — the step an item is on next, as a control: the circle and label tick the step off in one click, and the trailing caret opens the full step list (`StepMenu`). The same component at two sizes serves the grid's Actions column (`s`) and the detail modal's header field (`m`), so both read a row the same way.

Tokens: per tone, Background/{Tone} Subtle + Text/{Tone} + Stroke/{Tone} (upcoming), Background/{Tone} + Text/On {Tone} (today), Background/Negative + Text/On Negative (overdue).

Accessibility: a `group` named by the step and its urgency, holding two buttons: the tick and the caret (`aria-haspopup="menu"`, `aria-expanded`). Clicks inside never bubble, so the pill can sit in a clickable grid row.



Storybook · Workboard/Action Pill › Playground

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
labelrequired	`string`	—	The step the item is on next. Also builds the default accessible names.
tone	`"positive" | "warning" | "ai" | "brand"`	`default`	—
urgency	`"none" | "today" | "upcoming" | "overdue"`	`upcoming`	Urgency of the step's due date. Default `upcoming`.
size	`"s" | "m"`	`m`	`m` for a modal header field (default), `s` for a grid cell.
fullWidth	`boolean`	—	Stretch to the container, so every pill in a grid column is the same width.
onComplete	`(() => void)`	—	Ticks the step off. The circle closes to a check and the label is struck through for a beat before this runs, so the user sees what they closed. Omit to make the pill a menu trigger only.
completeOpensMenu	`boolean`	—	The step is a fork with no single next task: ticking opens the menu (`onMenuToggle`) instead of completing it.
completeLabel	`string`	—	Accessible name of the tick. Default "Mark {label} complete".
onMenuToggle	`((anchor: HTMLDivElement) => void)`	—	Opens or closes the step list. Receives the pill, to anchor the menu to.
menuOpen	`boolean`	—	Whether the step list is showing. Drives `aria-expanded` on the caret.
menuLabel	`string`	`Show all steps`	Accessible name of the caret. Default "Show all steps".
paused	`boolean`	—	The process is on hold: the circle becomes a pause mark and ticking is off.

PROP	TYPE	DEFAULT	DESCRIPTION
done	boolean	—	Every step is complete: a check, nothing left to tick.
followUp	boolean	—	A chase is standing in for the step (waiting on the client): draws a currentColor ring.
pendingMs	number	—	The last action is inside its undo window: a tint drains across the pill over this many milliseconds.

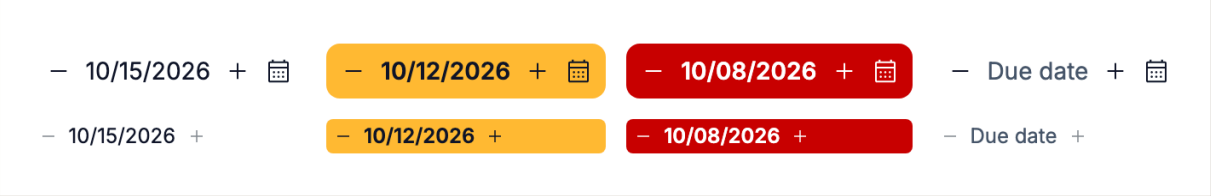
WORKBOARD

DueDatePill

REACT DueDatePill FILE src/components/workboard/Workboard.tsx FIGMA Due Pill VARIANTS Urgency × Size

Due Date Pill — the due date of an item's next step, flanked by one-day nudges. Quiet while the date is ahead, Background/Warning on the day, Background/Negative once it has passed. One control for the grid's Due Date column (`s`) and the modal header (`m`), so a date can never read two ways.

Accessibility: the nudges are labelled buttons; Today and Overdue add hidden words (R8). Clicks never bubble, so the pill can sit in a clickable row. In a grid the `s` nudges stay faint until the row is hovered or focused.



Storybook · Workboard/Action Pill › Due Date Pill

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
valuerequired	`string`	—	ISO `YYYY-MM-DD`, or `''` when no date is set.
onChangerequired	`(iso: string) => void`	—	—
size	`"s" | "m"`	`m`	`m` for a modal header field (default), `s` for a grid cell.
withPicker	`boolean`	—	Puts a native date picker behind the date and a calendar button on the trailing edge. Leave off in a grid where a click on the cell opens the item.
today	`string`	—	ISO "today" for the urgency colour. Default: the local date. Pass it for stable tests and stories.
format	`((iso: string) => string)`	—	Date display. Default `MM/DD/YYYY`.
placeholder	`string`	`Due date`	Shown when `value` is empty. Default "Due date".

WORKBOARD

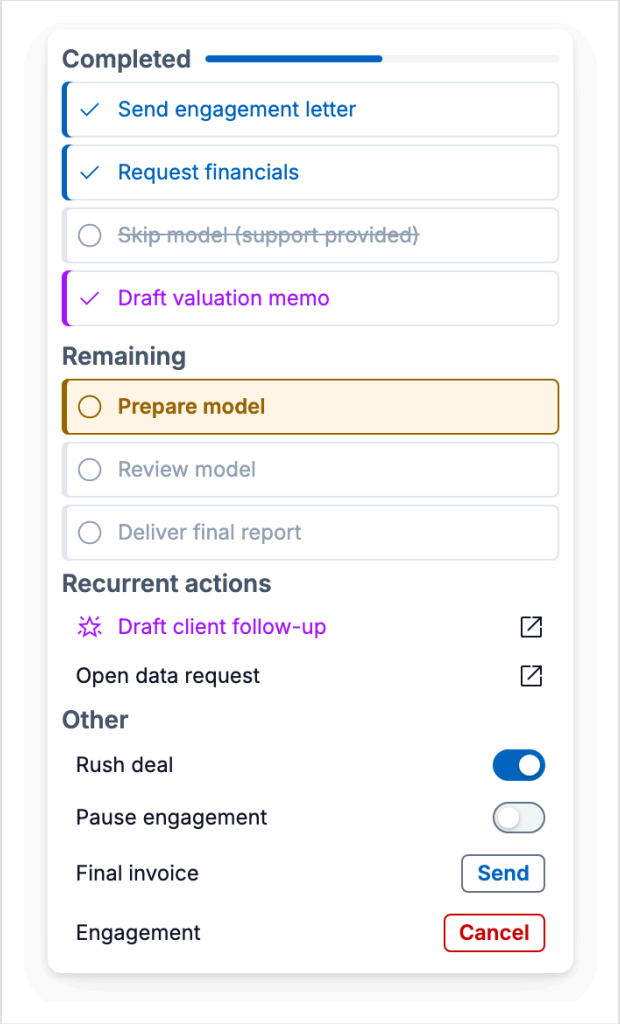
StepMenu

REACT StepMenu FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `StatusDropdown`)

Step Menu — the whole workflow behind an `ActionPill` caret: settled steps, then the remaining ones, each wearing its tone on a leading bar, plus any sections of tools, toggles and actions. The menu surface only: positioning and open state belong to the trigger.

Tokens: Background/Surface Raised + Elevation/Overlay; progress Background/Brand on Background/Subtle; the current step takes Background/{Tone} Subtle.

Keyboard: the APG menu model (`useMenuKeys`): Up/Down, Home/End, type-ahead, Esc and Tab call `onClose`. Steps are `menuitem`s; the current one carries `aria-current="step"` and every step says its state in hidden text (R8). The progress count ("2 of 6 steps completed") is the menu's description.



Storybook · Workboard/Action Pill › Step Menu

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`label` required	`string`	—	Accessible name of the menu ("Valuation status").
`steps` required	`StepMenuStep[]`	—	—
`onClose`	`((reason: MenuCloseReason) => void)`	—	Called when the user presses Esc or Tab inside the menu. The menu does not own open state: close it here and return focus to the trigger.
`onSelectStep`	`((id: string) => void)`	—	A step was picked. Picking a settled step rewinds the process to it.
`locked`	`boolean`	—	The process is on hold: every step is locked.
`progress`	`{ value: number; max: number; }`	—	Progress track. Default: `done` steps over every step that is not `skipped`.
`completedLabel`	`string`	`Completed`	Group headings. Default "Completed" / "Remaining".
`remainingLabel`	`string`	`Remaining`	—

PROP	TYPE	DEFAULT	DESCRIPTION
children	ReactNode	—	Extra sections after the steps: `StepMenuSection` holding `StepMenuLink` / `StepMenuToggle` / `StepMenuAction`.

WORKBOARD

StepMenuSection

REACT StepMenuSection FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `StatusDropdown`)

A headed run of items inside a `StepMenu`. Presentational: the items stay direct menu items for the keyboard model.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
headingrequired	ReactNode	—	Overline heading ("Recurrent actions", "Other").

WORKBOARD

StepMenuLink

REACT StepMenuLink FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `StatusDropdown`)

A tool the step list opens (a form, a draft, another page). Trailing open-in-new glyph; `href` renders an anchor.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
href	string	—	—
ai	boolean	—	Drafted by the model for the user: the AI mark and Text/AI.

WORKBOARD

StepMenuToggle

REACT StepMenuToggle FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `StatusDropdown`)

A setting that applies at once (Rush, Waiting, Pause). A `menuitemcheckbox` drawn as a small switch.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
checkedrequired	boolean	—	—
onCheckedChange required	(checked: boolean) => void	—	—

WORKBOARD

StepMenuAction

REACT StepMenuAction FILE src/components/workboard/Workboard.tsx FIGMA — (prototype `StatusDropdown`)

A one-off command from the step list (send an invoice, cancel the engagement). The whole row is the target.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
childrenrequired	ReactNode	—	What the row is about ("Final invoice").
actionrequired	string	—	The verb on the trailing chip ("Send", "Cancel").
tone	"default" | "destructive"	default	`destructive` for an irreversible action. Put it last.

COMPONENTS · REFERENCE



ADDED BY THE CODE LAYER

EXPORT	WHY
`ScalarProvider`	Carries the colour mode and the type-ramp mode. In Figma these are variable modes set on a frame.
`Icon`	Sizing and tint wrapper. Enforces the re-tint that SDS_Main glyphs require.
`icons`	The product icon set — Google Material Symbols (Outlined), generated from `icon/material- icons.json` by `npm run gen:icons`. Figma uses SDS_Main instead.
`Typography` / `Heading` / `Text` / `Label` / `Overline`	The type ramp as components. Figma expresses this as 72 text styles.

EXPORT	WHY
`Table`	The platform table: selection, resize, formula mode, keyboard and editing around `Cell` / `RowHeader` / `ColumnHeader`.
`DataGrid`	The grid shell — a CSS grid whose column tracks come from the header cells, with a sticky header under `maxHeight`.
`ChartCanvas`	Chart.js lifecycle, token re-resolution on theme change, and the screen-reader data table.
`chartScaffold` / `baseChartOptions`	The Figma Chart Frame spec as Chart.js options.
`useChartTokens` / `resolveChartTokens`	The CSS-variable → canvas bridge.
`Modal`	The reusable dialog shell behind `Add_Column_Modal`.
`ToastViewport`	The fixed stack toasts render into.
`MenuPanel` / `MenuGroupLabel`	The floating surface shared by every dropdown.
`SelectMenu` / `SelectMenuOption`	The open listbox of a select (check, disabled, description, search slot) — composed by hand on several screens.
`TaskPill`	Pending-task marker in a grid row.
`ImageCropField`	Logo / avatar upload preview with zoom.
`cx`, `useControllableState`	Utilities every component needs.

These have no Figma counterpart. They exist because code needs them and repeating them per page would guarantee drift.

ADDED BY THE CODE LAYER

ScalarProvider

REACT `ScalarProvider` FILE `src/components/../theme/ScalarProvider.tsx`

Establishes the Scalar theme. Wrap the application once.

With `viewport="auto"` the ramp follows the window width. It is read through `useSyncExternalStore`, so server rendering and hydration use `desktop` and the real width is applied right after — no hydration mismatch.

Import the stylesheet once, at the application entry point: import '@scalar/design-system/styles.css';

PROPS

PROPSCALARPROVIDER	TYPECARRIES THE COLOUR MODE AND THE TYPE-RAMP MODE. IN FIGMA THESE ARE VARIABLE MODES SET ON A FRAME.	DEFAULT	DESCRIPTIONICON
`mode`	`"light"` | `"dark"` | `"system"`	`light`	Colour mode. Defaults to `light` — pages are always light, whatever the OS says.
`viewport`	`"mobile"` | `"auto"` | `"desktop"` | `"desktop-large"`	`desktop`	Type-ramp mode. Defaults to `desktop`.
`target`	`"scope"` | `"root"`	`root`	Where the `data-theme` / `data-viewport` attributes are written. `root` (default) writes to so portalled content inherits them. `scope` renders a wrapper div instead — use it to theme part of a page. The wrapper receives `className` and any other native div props; in `root` mode no element is rendered, so `className` and native props are ignored.

ADDED BY THE CODE LAYER

Icon

REACT Icon FILE src/components/icon/Icon.tsx

Icon — sizing and tint wrapper for a glyph.

Icons imported from SDS_Main icons default to Text/On Brand (white) and are invisible on a light surface until re-tinted. This component always sets a tint, so anything rendered through it is safe by default.

Storybook · Core/Icon › Default

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
`children`	`ReactNode`	—	The glyph, as SVG body content: `<Icon> <icons.Search /></Icon>` (the product set) or `<Icon><glyphs.Check /></Icon>` (structural aliases).
`tone`	`"disabled" | "primary" | "secondary" | "positive" | "inherit" | "warning" | "negative" | "ai" | "brand" | "inverse" | "onBrand"`	`secondary`	—
`label`	`string`	—	Opt-in accessible name. Icons are decorative (`aria-hidden`) by default; one that carries meaning on its own needs a label, which switches it to `role="img"`. A consumer-supplied `aria-label` / `aria-labelledby` has the same effect.
`size`	`"xs" | "s" | "m" | "l" | "xl"`	`s`	Bound to Semantic: Sizing/Icon. Never hand-size an icon.

ADDED BY THE CODE LAYER

Typography

REACT Typography FILE src/components/typography/Typography.tsx

Typography — generic over `variant` so `step` is checked against the role.

Fair value as of September 30

Storybook · Core/Typography › Default

PROPS

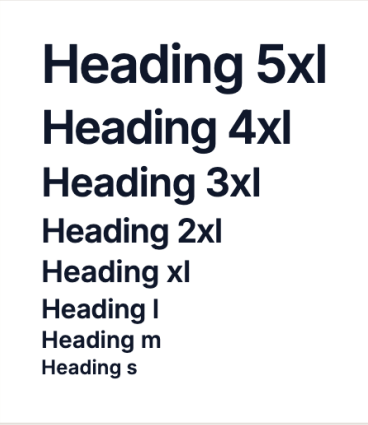
PROP	TYPE	DEFAULT	DESCRIPTION
`variant`	`"label"` | `"heading"` | `"link"` | `"text"` | `"display"` | `"overline"`	`text`	The type role. Each role exists so its metrics track its job. Named `variant` rather than `role` so the DOM `role` attribute stays free.
`step`	`"xs"` | `"s"` | `"m"` | `"l"` | `"xl"` | `"2xl"` | `"3xl"` | `"4xl"` | `"5xl"`	`s`	The step within the role, checked against the role (`overline` has only `s` and `m`). The ramp has no step below 12px except `heading.xs` (10px, data- grid chrome only — rule R10).
`weight`	`"bold"` | `"light"` | `"regular"` | `"semiBold"`	`semiBold`	Orthogonal to size: changing it never moves size or line-height (rule R7).
`tone`	`TextTone`	`tertiary`	—
`as`	`ElementType`	—	The rendered element. Pick it for document structure, not for size.
`truncate`	`boolean`	—	Truncate to a single line with an ellipsis.
`htmlFor`	`string`	—	Valid when `as="label"`, which is how `Label` and `FormField` use it.

ADDED BY THE CODE LAYER

Heading

`REACT` Heading `FILE` src/components/typography/Typography.tsx

Heading — the heading role; renders `<h{level}>` (no `role` attribute needed).



Storybook · Core/Typography › Headings

PROPS

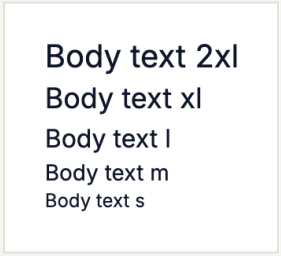
PROPTYPE	TYPEDEFAULT	DEFAULTDESCRIPTION	DESCRIPTIONVARIANT
`level`	`1` | `2` | `3` | `4` | `5` | `6`	`2`	Sets the default element (`h1`–`h6`). Document structure, not size.
`tone`	`TextTone`	`tertiary`	—
`step`	`"xs"` | `"s"` | `"m"` | `"l"` | `"xl"` | `"2xl"` | `"3xl"` | `"4xl"` | `"5xl"`	`s`	The step within the role, checked against the role (`overline` has only `s` and `m`). The ramp has no step below 12px except `heading.xs` (10px, data- grid chrome only — rule R10).
`as`	`ElementType`	—	The rendered element. Pick it for document structure, not for size.
`weight`	`"bold"` | `"light"` | `"regular"` | `"semiBold"`	`semiBold`	Orthogonal to size: changing it never moves size or line-height (rule R7).
`truncate`	`boolean`	—	Truncate to a single line with an ellipsis.
`htmlFor`	`string`	—	Valid when `as="label"`, which is how `Label` and `FormField` use it.

ADDED BY THE CODE LAYER

Text

REACT Text FILE src/components/typography/Typography.tsx

Text — the body role.



Storybook · Core/Typography › Text Steps

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tone	TextTone	tertiary	—
step	"s" | "m" | "l" | "xl" | "2xl"	s	The step within the role, checked against the role (overline has only s and m). The ramp has no step below 12px except heading.xs (10px, data- grid chrome only — rule R10).
as	ElementType	—	The rendered element. Pick it for document structure, not for size.
weight	"bold" | "light" | "regular" | "semiBold"	semiBold	Orthogonal to size: changing it never moves size or line-height (rule R7).
truncate	boolean	—	Truncate to a single line with an ellipsis.
htmlFor	string	—	Valid when as="label", which is how Label and FormField use it.

ADDED BY THE CODE LAYER

Label

REACT Label FILE src/components/typography/Typography.tsx

Label — compact UI furniture and form labels.

Renders a <label> when htmlFor is given (so the association works) and a otherwise; an orphan <label> labels nothing. Pass as to override.



Storybook · Core/Typography › Label And Overline

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tone	TextTone	tertiary	—
step	"s" | "m" | "l"	s	The step within the role, checked against the role (overline has only s and m). The ramp has no step below 12px except heading.xs (10px, data- grid chrome only — rule R10).
as	ElementType	—	The rendered element. Pick it for document structure, not for size.
weight	"bold" | "light" | "regular" | "semiBold"	semiBold	Orthogonal to size: changing it never moves size or line-height (rule R7).
truncate	boolean	—	Truncate to a single line with an ellipsis.
htmlFor	string	—	Valid when as="label", which is how Label and FormField use it.

ADDED BY THE CODE LAYER

Overline

REACT Overline FILE src/components/typography/Typography.tsx

Overline — the uppercase role. It owns its tracking; never add your own.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
tone	TextTone	tertiary	—
step	"s" | "m"	s	The step within the role, checked against the role (overline has only s and m). The ramp has no step below 12px except heading.xs (10px, data- grid chrome only — rule R10).
as	ElementType	—	The rendered element. Pick it for document structure, not for size.
weight	"bold" | "light" | "regular" | "semiBold"	semiBold	Orthogonal to size: changing it never moves size or line-height (rule R7).
truncate	boolean	—	Truncate to a single line with an ellipsis.
htmlFor	string	—	Valid when as="label", which is how Label and FormField use it.



COMPONENTS · REFERENCE

OTHER EXPORTS

OTHER EXPORTS

BodySlot

REACT BodySlot FILE src/components/template/PageTemplate.tsx

BodySlot — the placeholder that stands in for page content.

It is what `PageTemplate` renders when it is given no `children`. A real page never ships it: pass your content as the template's children and this disappears. Matches the grey `BODY-SLOT` rectangle in the Figma frame "Page Template · body space used for any component".

It carries no role or accessible name: it is scaffolding, and announcing it would be noise. Its visible text already says what it is.

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
label	ReactNode	BODY-SLOT	Overrides the placeholder text. Default: "BODY- SLOT".

OTHER EXPORTS

PageTemplate

REACT PageTemplate FILE src/components/template/PageTemplate.tsx

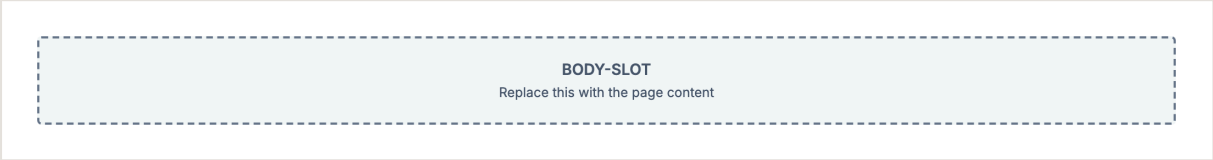
PageTemplate — the base template for every screen in the product.

Figma: "Page Template · body space used for any component" (components file `Z4MtKOfkNEzhMYJzN1q3kR`, node 1671:13280).

```
┌ navigation ──────────────────────────┐   Primary Menu (48)
│ companyInfo                          │   Company info + Secondary Menu (32)
│ ┌ sheet ─────────────────────────┐   │
│ │ subNavigation                  │   │   Tertiary Menu (40)
│ │ ┌ BODY-SLOT ──────────────┐    │   │
│ │ │ children                │    │   │   fills the remaining height
│ │ └─────────────────────────┘    │   │
│ └────────────────────────────────┘   │
│ drawer                               │   Workspace Drawer, docked (28)
└──────────────────────────────────────┘
```

Every slot is optional and independent. The body slot takes all the height the other slots leave over and scrolls on its own, so the navigation above it and the drawer below it stay put. Never restyle the slots from outside — change the component you pass in.

Landmarks: the body slot is the page's single `<main>`; the navigation and sub-navigation components you pass are expected to render their own `<nav>` / `<header>`. `ref` and native props land on the root element.



Storybook · Core/PageTemplate › Empty

PROPS

PROP	TYPE	DEFAULT	DESCRIPTION
navigation	ReactNode	—	Slot 1 — the application bar. Pass a `PrimaryMenu`.
companyInfo	ReactNode	—	Slot 2 — the company header under the bar. Pass a `CompanyInfo` (with a `SecondaryMenu` as its children).
subNavigation	ReactNode	—	Slot 3 — the sub-navigation bar at the top of the sheet. Pass a `TertiaryMenu`.
children	ReactNode	—	Slot 4 — the BODY-SLOT: the page content. Any component or composition goes here. Omit it and the template shows the `BodySlot` placeholder.
drawer	ReactNode	—	Slot 5 — the docked drawer at the foot of the page. Pass a `WorkspaceDrawer`.
fullHeight	boolean	true	Fill the viewport height (default). Set `false` when something else already owns the page height — for example when the template is nested in an app frame that renders its own full-height column.

VALUES FLOW ONE WAY.

Figma owns the values; code owns the behaviour. If a value looks wrong, fix it in Figma and regenerate. Never patch a value inside a component: the fix would reach one place instead of every place.

DOCUMENT	Scalar Design System v1.1 — Guide, Tokens & Components
PACKAGE	@scalar/design-system 1.1.0
SOURCE	Prism-v1 · de84207 · 2026-10-12 15:08:17 -0600
GENERATED	19 October 2026 · light mode throughout (rule R14)
EDITION	Editorial layout, blue edition · set in Roboto Condensed, Inter and IBM Plex Mono